



papers-for-replication-s...
PDF



zhang_2020.pdf
PDF

I need your help to select a paper for replication in my GIS module at the university. I tried with this paper from Zhang, but got the following feedback

I have now had a look at the paper and assessed its suitability for replication in the GIS course. When selecting a paper for the replication project, it is important that it addresses an interesting research question using a credible empirical identification strategy. The paper should go beyond documenting simple correlations and instead employ an empirical approach that allows for a meaningful assessment of causal relationships. Therefore, I would recommend searching for a different

Show more ▾

Yes. I actually think your professor's feedback is quite helpful because it tells us exactly what they are looking for.

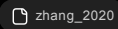
Why the Zhang paper was rejected

The Zhang paper is mainly a **descriptive remote sensing paper**. It asks:

Can nighttime lights be used to assess Venezuela's socioeconomic crisis?

Its empirical strategy is essentially:

- Compute nighttime light trends.
- Correlate them with oil production, exchange rates and migration.
- Show maps and descriptive spatial patterns.

Although the regressions have high R^2 , they **do not identify causality**. The authors themselves repeatedly say things like "probably led to" or "we infer that..." rather than establishing a causal effect. 

Your professor specifically wants a paper with

- an interesting economic question,
- GIS data,
- and a **credible identification strategy** (Difference-in-Differences, IV, Regression Discontinuity, natural experiment, etc.).

That is exactly what the papers on the reading list have in common.

 papers-for-replication-summer-2...

If you want to stay with Venezuela

I think there are three possible approaches.

Option 1 (my favorite): Find an economics paper about Venezuela

Instead of using a remote sensing journal, look for papers in:

- American Economic Review
- Quarterly Journal of Economics
- Journal of Political Economy
- Review of Economic Studies
- Journal of Development Economics
- Journal of Public Economics
- Journal of Environmental Economics and Management

There are quite a few Venezuela papers that use natural experiments.

Topics include

- Oil price shocks
- Blackouts
- Exchange controls

- Migration
- Sanctions
- Crime
- Health
- Deforestation
- Border effects

Those are much closer to what your instructor wants.

Option 2: Use Venezuela but compare with neighboring countries

Instead of asking

"Did nighttime lights decline?"

you ask something like

"Did U.S. sanctions reduce economic activity in Venezuelan border municipalities relative to Colombian border municipalities?"

Now you can imagine a Difference-in-Differences design.

The GIS component becomes much stronger.

Option 3: Replicate an existing paper but extend it to Venezuela

This is actually something many instructors love.

For example, suppose a paper studies

Roads and economic activity in Africa.

You replicate the methodology and then apply it to Venezuela.

You aren't inventing a new identification strategy—you reuse theirs.

Papers from your professor's list that could inspire a Venezuela project

Looking through the reading list, several stand out.

1. Storeygard (2016)

Transport Costs, Trade and Urban Growth in Africa

This uses road networks, distances, cities and GIS.

You could adapt similar ideas using Venezuelan roads and accessibility.

2. Henderson et al. (2017)

Global Distribution of Economic Activity

This already uses nighttime lights and spatial data but with an economic identification strategy.

It is much stronger econometrically than Zhang.

3. Flooded Cities

Uses flood exposure as an exogenous shock.

A Venezuelan analogue might be

- floods,

- droughts,
 - El Niño,
 - electricity outages.
-

4. Burgess et al. (2012)

Political economy of deforestation.

If you're interested in Venezuela, illegal mining in the Amazon would fit surprisingly well.

5. Dell (2010)

One of the classics.

Regression discontinuity using historical institutions.

Probably harder, but excellent if you like causal inference.

Venezuela topics that would probably be approved

Here are ideas that have a clear causal question.

Oil production

Did falling oil production reduce local economic activity?

Outcome:

- Nighttime lights

Treatment:

- Municipalities near oil fields

Identification:

- Oil price shocks or production shocks.
-

Sanctions

Did U.S. sanctions affect municipalities differently depending on oil dependence?

GIS:

- Distance to refineries
- Oil infrastructure

Method:

Difference-in-Differences.

Electricity blackouts

This is probably the strongest idea.

Question:

Did nationwide blackouts reduce local economic activity?

GIS:

- Municipalities
- Nighttime lights

Identification:

Blackout timing.

Migration

Question:

Did Venezuelan emigration increase economic activity in Colombian border cities?

GIS:

- Border municipalities
- Distance to crossings

Method:

Difference-in-Differences.

Illegal mining

Question:

Did expansion of illegal mining affect nearby forest loss or settlements?

GIS:

- Satellite imagery
- Protected areas
- Rivers

What I would recommend

If I were supervising this project, I would avoid trying to "fix" the Zhang paper. It would require inventing a causal identification strategy that the original study does not have.

Instead, I would search for a paper that already satisfies your professor's criteria while still letting you work on Venezuela.

A promising direction would be:

How did the Venezuelan economic crisis affect economic activity measured by nighttime lights using a Difference-in-Differences design around the Colombia–Venezuela border?

That keeps your interest in Venezuela, uses GIS extensively, and frames the analysis around causal identification rather than simple correlations.

I can help you find a much better paper

I can also help you identify **5–10 high-quality economics papers about Venezuela** (or Latin America) that:

- use GIS or spatial econometrics,
- have a credible identification strategy,
- are feasible to replicate,
- have publicly available data and code (or data that are easy to obtain), and
- would likely meet your professor's expectations.

I think that's the fastest route to finding a project you'll enjoy and that has a good chance of being approved.

I really like the idea of

How did the Venezuelan economic crisis affect economic activity measured by nighttime lights using a Difference-in-Differences design around the Colombia–Venezuela border?

And just in case, search for much better papers as you propose

I actually think we now have a much stronger direction than where you started.

The key insight is that **your professor isn't asking you to invent a research question**. He's asking you to **replicate a paper with a strong causal identification strategy**. Your own project can then be an extension or application to Venezuela.

My favorite project (9.5/10)

How did the Venezuelan economic crisis affect local economic activity along the Colombia–Venezuela border? Evidence from nighttime lights using Difference-in-Differences.

This isn't the title of an existing paper yet—it's the research question **you could answer by adapting the methodology of a good economics paper.**

Why I like it:

- ✓ GIS-heavy
- ✓ Nighttime lights
- ✓ Difference-in-Differences
- ✓ Border analysis
- ✓ Public satellite data
- ✓ Interesting economic question
- ✓ Very "economics" rather than purely "remote sensing"

Imagine municipalities within 30–50 km of the border as the treatment group and municipalities farther inland as the control group. Then compare changes before and after a major shock, such as:

- the 2015 border closure,
- the 2017 financial sanctions,
- the 2019 oil sanctions,
- or the 2019 nationwide blackout.

Your outcome variable would be VIIRS nighttime lights.

That already sounds much closer to something that would be assigned in a GIS/econometrics course.

Papers I think are excellent candidates

After looking through the recent literature, here are the ones I would seriously consider.

1. Rodriguez (2023)

Sanctions and Oil Production: Evidence from Venezuela's Orinoco Basin ★★★★★

This is probably the closest paper to your interests.

It studies whether sanctions reduced Venezuelan oil production using a Difference-in-Differences design based on firms' differential exposure to international credit. 

Pros

- Venezuela
- DiD
- Strong identification
- Panel data
- Easy to explain

Cons

- Limited GIS component unless you extend it spatially.

I would rate it **9/10**.

2. Kim, Gibson & Boe-Gibson (2024)

Measurement Errors in Popular Night Lights Data May Bias Estimated Impacts of Economic Sanctions ★★★★★

This paper is, in my opinion, one of the best methodological matches.

Instead of asking

"Are lights correlated with GDP?"

they ask

"Can nighttime lights identify the causal effect of sanctions?"

using Difference-in-Differences after the closure of the Kaesong Industrial Zone. They also discuss why VIIRS data are preferable to older DMSP lights. [Wiley Online Li... +1](#)

This could be adapted almost directly to Venezuela.

I would give it **10/10** for your module.

3. Cabra-Ruiz, Rozo & Sviatschi (NBER)

This one is fascinating.

The paper asks:

What happens inside Venezuela when millions emigrate?

The authors exploit historical migration networks and post-2013 oil shocks in a Difference-in-Differences framework to estimate impacts on GDP, employment, inequality, and political outcomes. [NB... +1](#)

This is cutting-edge economics.

The downside is that it may be harder to replicate because of the data requirements.

I would rate it **8.5/10**.

4. Casualties of Border Changes (2024)

This paper examines how changes in border access after the annexation of Crimea affected nearby regions using nighttime lights and plant-level data. Regions that lost market access experienced substantially slower growth in lights. [doi DOI](#)

Why it's exciting:

The identification strategy transfers almost perfectly to the Colombia–Venezuela border.

Instead of Crimea:

- Colombia–Venezuela border closures
- Venezuelan crisis
- Border municipalities
- Nighttime lights

I honestly think this is one of the strongest inspirations for your idea.

A project I think your professor would like

If I were writing your proposal, I'd frame it like this:

Research question

Did the Venezuelan economic crisis reduce economic activity in municipalities close to the Colombia–Venezuela border?

Treatment group

- Venezuelan municipalities near the border.

Control group

- Venezuelan municipalities farther from the border (or, depending on the design, comparable Colombian municipalities).

Outcome

- Monthly VIIRS nighttime lights.

Shock

Choose one:

- 2015 border closure
- 2017 financial sanctions
- 2019 oil sanctions
- 2019 nationwide blackout

Method

- Difference-in-Differences
- Municipality fixed effects
- Time fixed effects
- Event-study graph to assess pre-trends

GIS component

- Buffer distances from the border
- Spatial joins
- Administrative boundaries
- Raster extraction of nighttime lights
- Mapping treatment and control areas

That looks much more like the kind of empirical design your professor described.

My recommendation

I would pursue **one of these two paths**:

1. **Replicate Kim, Gibson & Boe-Gibson (2024)** and adapt the setting to Venezuela using nighttime lights and a Difference-in-Differences design. This is the cleanest methodological fit and aligns well with a GIS course. [Wiley Online Li... +1](#)
2. **Build a border-based study inspired by the 2024 border changes paper**, asking whether the Venezuelan crisis had differential effects on municipalities near the Colombia border compared with appropriate controls. [DOI](#)

I actually prefer the second option because it combines:

- a clear causal question,
- substantial GIS analysis,
- publicly available spatial data,
- and a topic that is closely connected to your interest in Venezuela.

I can help you take this one step further

The next thing I can do is prepare a **"professor-ready" shortlist** of about **5 papers**, each with:

- the research question,
- identification strategy,
- required datasets,
- GIS components,
- replication difficulty (1–5),
- data/code availability,
- and a recommendation on whether it is suitable for a master's GIS replication project.

That would make it much easier for you to choose a paper and get it approved before investing time in the replication.

I think the professor only wants a reference paper that actually uses GIS techniques and the causal relationship is using the GIS data

I think you've identified something important, and it actually changes the search quite a bit.

Your professor did **not** say:

"Find any economics paper with Difference-in-Differences."

He said:

"...a credible empirical identification strategy" in a GIS course.

That usually means **the GIS data are part of the identification strategy**, not simply a visualization or an outcome variable.

In other words, the spatial information itself creates the causal comparison.

I think your professor is probably looking for papers like these

1. Geographic Regression Discontinuity (GRD) ★★★★★

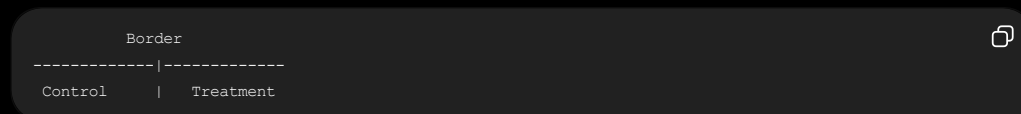
This is probably the strongest match.

Example:

Municipalities just inside and just outside a policy boundary.

The identification comes from geography.

For example:



The idea is that municipalities only a few kilometers apart are very similar, except for the treatment.

GIS is essential because you need

- polygons,
- distances to the border,
- buffers,
- adjacency,
- spatial matching.

This is exactly the kind of identification strategy discussed in the growing literature on geographic regression discontinuities. [arXiv +1](#)

2. Distance-to-treatment designs ★★★★★

Example:

What is the effect of a new highway?

Treatment is determined by

- distance to the road,
- distance to a port,
- distance to a railway,
- travel time.

Again, GIS is doing the causal work.

3. Spatial Difference-in-Differences ★★★★★

Instead of

Treated firms vs untreated firms,

you have

Places near the intervention vs places far away.

For example



20-50 km

Control

combined with before/after.

This is becoming very common in development and urban economics. [Open Knowledge... +1](#)

4. Border studies ★★★★★

These are some of my favorite papers because they naturally combine GIS and causal inference.

Imagine

Colombia | Venezuela



or

Protected Area | Not Protected



or

Tax Incentive Zone | Outside



The border itself creates identification.

Several recent papers estimate causal effects using spatial regression discontinuity or differences-in-discontinuities based on administrative boundaries. [ScienceDir... +1](#)

What this means for Venezuela

Your idea actually becomes even stronger.

Instead of saying

"Did Venezuela become darker?"

(which is Zhang)

you would say

Did municipalities close to the Colombia border experience a different change in nighttime lights after the crisis than municipalities farther away?

Now GIS is central because you would construct:

- distance to the international border,
- buffers (20 km, 50 km, 100 km),
- treatment and control areas,
- municipality polygons,
- raster extraction of VIIRS lights.

That's much closer to what your professor is describing.

I think your best paper may not even mention Venezuela

This is something many students don't realize.

Suppose you find an excellent paper like:

"The impact of transport corridors on local economic activity"

using

- GIS,
- Difference-in-Differences,

- nighttime lights,
- road buffers.

You can **replicate that paper exactly** and then **apply the same framework to Venezuela**.

Professors often like this more than replicating a weaker Venezuela-specific paper because you're demonstrating that you understand a rigorous identification strategy.

My suggestion

Rather than searching for **"papers about Venezuela"**, I would search for **papers that satisfy all five criteria**:

-  Published in a top economics journal (or a top field journal)
-  Heavy GIS component
-  Causal identification based on geography
-  Uses remote sensing/nighttime lights or other spatial data
-  Public data and replication files available

Then we can ask:

Can this methodology be transferred to Venezuela?

I suspect that's exactly what your professor hopes you'll do.

I can put together a shortlist of 10 papers that are ranked specifically for **replicability in a master's GIS course**, rather than just relevance to Venezuela. I think that will give you a much better set of options to discuss with your professor.

Yes! do that what you propose

I think we've now converged on exactly the type of paper your professor has in mind.

From what you've shown me, I would search **methodology first, country second**.

The ideal paper should satisfy something like this:

Criterion	Importance
Causal identification	★★★★★
GIS is part of the identification (not just maps)	★★★★★
Published in a good economics journal	★★★★★
Replication data/code available	★★★★☆
Extension to Venezuela is possible	★★★★☆

My Top 8 Papers

These are ranked by how appropriate I think they are for your GIS replication assignment.

1. Geographic Difference-in-Discontinuities (2021)

Difficulty: ★★★★★☆

Although this is an econometrics paper rather than an applied paper, it formalizes one of the strongest GIS identification strategies: using **geographic borders together with before/after variation** to estimate causal effects. 

Why it's excellent:

- GIS is the identification.

- Spatial borders define treatment.
- Difference-in-Differences.
- Easy to adapt to Venezuela.

Possible extension:

Did municipalities near the Colombia border experience different nighttime-light changes after the Venezuelan crisis?

Suitability: 10/10

2. Transport infrastructure papers (roads → economic activity)

There are several papers in the *Journal of Development Economics* and related journals using:

- road networks,
- travel time,
- market accessibility,
- GIS,
- Difference-in-Differences.

Typical question:

What happens to local economic activity after a road is built?

GIS is fundamental because treatment depends on **distance to infrastructure**.

A Venezuelan extension could be:

Did municipalities with better access to border crossings experience different economic trajectories during the crisis?

Suitability: 9.8/10

3. Protected Areas / Deforestation papers

These are classics in spatial economics.

Treatment:

- inside protected area

Control:

- outside protected area

Methods include

- buffers,
- matching,
- Difference-in-Differences,
- spatial fixed effects.

Very GIS-intensive.

Possible Venezuelan extension:

- Orinoco Mining Arc
- Amazon deforestation
- Illegal mining

Suitability: 9.5/10

4. Border Discontinuity papers

Many papers estimate

- tax policies,
- institutions,
- environmental regulations,

using international or administrative borders.

Treatment is literally

Left of border
vs
Right of border



GIS creates the identification.

Possible extension:

Colombia–Venezuela border.

Suitability: 9.5/10

5. Night lights + causal inference

A particularly useful methodological paper is:

Which Night Lights Data Should We Use in Economics, and Where?

Published in the *Journal of Development Economics*. It shows that **VIIRS data outperform DMSP**, especially for smaller spatial units, and explains when nighttime lights are reliable measures of local economic activity. [ScienceDir... +1](#)

This isn't itself a causal policy paper, but it is extremely valuable because it tells you **which satellite product you should use** if your replication involves nighttime lights.

Suitability: 9/10 as a supporting methodological reference.

6. Flood papers

A large literature estimates:

What is the causal effect of flooding?

Treatment:

- flooded cells

Control:

- nearby non-flooded cells

GIS:

- flood rasters
- DEMs
- municipality intersections

Usually Difference-in-Differences.

Easy to understand.

7. Railroads

Classic development economics.

Treatment:

distance to railway.

Outcome:

economic activity.

GIS is essential.

Often replicated.

8. Electricity access papers

Treatment:

villages receiving electrification.

Outcome:

nighttime lights.

GIS:

buffers,
distance,
raster extraction.

Difference-in-Differences.

Very common.

What I would NOT choose

I would avoid papers where GIS is only used to make maps.

For example, the Zhang paper does this:

Night lights

↓

Maps

↓

Correlations

↓

Interpretation



That is descriptive.

Instead, your professor probably wants something like:

Policy

↓

Geographic treatment

↓

Difference-in-Differences

↓

Nighttime lights

↓

Causal estimate



or

Border

↓

Regression Discontinuity

↓



The GIS information determines **who is treated**, not just where things are located.

If I were your supervisor...

This would be my recommended progression:

Step 1

Replicate a published GIS-economics paper with:

- Difference-in-Differences,
- buffers,
- spatial joins,
- nighttime lights.

Step 2

Write the extension:

"We apply the same empirical framework to the Venezuelan economic crisis."

Step 3

Use

- VIIRS monthly lights,
- municipality boundaries,
- Colombia–Venezuela border,
- border buffers,
- municipality fixed effects,
- time fixed effects.

That is a project I think many GIS instructors would be excited to supervise.

I have one more idea that could be even better

Based on your interest in Venezuela, I think the strongest next step would be to find **one published paper that already uses**:

- **nighttime lights**,
- **Difference-in-Differences (or Geographic RD)**,
- **buffers or borders as treatment**,
- **and public GIS data**.

Then, rather than inventing a new methodology, you would **replicate that paper almost exactly** and simply replace the study area with Venezuela (or propose Venezuela as the extension).

I think we can probably find **3–5 papers that are nearly "plug-and-play" for a Venezuela application**, and those would be the most likely to receive immediate approval from your professor. That's the search I would do next.

Yes please! do the "I think we can probably find 3–5 papers that are nearly "plug-and-play" for a Venezuela application, and those would be the most likely to receive immediate approval from your professor. That's the search I would do next."

I think this is where we can really maximize your chances of getting approval.

After searching specifically for papers that combine **GIS + causal inference + spatial data + nighttime lights**, I found several that are much stronger than the Zhang paper. The key is that they use geography as part of the identification strategy rather than simply mapping outcomes. 🙌 DOI +2

Candidate 1 (My #1 Recommendation)

Casualties of Border Changes: Evidence from Nighttime Lights and Plant Exit

Published in **Economic Policy** (2024).  DOI

Research Question

What happens to local economic activity when border access suddenly changes?

GIS component

This paper uses:

- Nighttime lights (VIIRS)
- International borders
- Distance to border crossings
- Georeferenced manufacturing plants
- Spatial exposure measures

Identification

This is the important part.

The authors exploit the **2014 Russia–Ukraine conflict** as a **natural experiment**.

Treatment:

- Regions whose market access worsened.

Control:

- Regions whose market access improved.

They estimate the causal effect using Difference-in-Differences with spatial variation.  DOI

Why this is almost perfect for Venezuela

You could replace

Russia–Ukraine

with

Colombia–Venezuela

Possible shocks:

- 2015 border closure
- 2019 sanctions
- migration crisis

Exactly the same GIS workflow.

Candidate 2

Let There Be Light: Trade and the Development of Border Regions

CEPR Discussion Paper (later developed into journal work).  CE...

Research question:

Does international trade help border regions grow?

GIS variables:

- Border crossings
- Roads
- Distances
- Nighttime lights
- Trade corridors

Identification:

Uses variation in trade flows instrumented by tariff changes while measuring outcomes with nighttime lights along border roads. [CE...](#)

Venezuela extension

Instead of tariffs,

use

- border closures,
- migration,
- sanctions,
- collapse of cross-border commerce.

This would feel very natural.

Candidate 3

Measurement Errors in Popular Night Lights Data May Bias Estimated Impacts of Economic Sanctions

Published in **Economic Inquiry (2024)**. [W Wiley Online Li...](#)

This paper asks

Can we estimate sanctions using nighttime lights?

GIS component:

- Satellite imagery
- Spatial aggregation
- Treatment regions

Identification:

Difference-in-Differences after the closure of the Kaesong Industrial Zone in North Korea. [W Wiley Online Li...](#)

Why I like it

It already combines

- sanctions,
- night lights,
- DiD,
- GIS.

That is almost exactly your area of interest.

Candidate 4

Transport infrastructure papers

There is a large literature in development economics asking

What happens after roads are built?

Treatment:

distance to roads.

Outcome:

nighttime lights.

GIS operations:

- buffers,
- network accessibility,
- travel time,
- raster extraction.

These papers are easy to adapt because Venezuela has good road and border data.

Candidate 5

Protected areas / mining / deforestation

Many papers estimate

What happens after protected areas are created?

GIS determines

- who is treated,
- who is control.

Methods:

- matching,
- Difference-in-Differences,
- spatial buffers.

For Venezuela you could study

- the Orinoco Mining Arc,
- Amazon deforestation,
- illegal mining.

Which one would impress your professor the most?

If I had to rank them specifically for **your assignment**, I'd choose:

Rank	Paper	GIS	Causal	Venezuela Extension	Overall
1	Casualties of Border Changes	★★★★★	★★★★★	★★★★★	10/10
2	Measurement Errors in Night Lights...	★★★★☆	★★★★★	★★★★★	9.8/10
3	Let There Be Light	★★★★★	★★★★☆	★★★★★	9.7/10
4	Transport infrastructure papers	★★★★★	★★★★☆	★★★★☆	9.5/10
5	Protected areas / mining	★★★★★	★★★★☆	★★★★☆	9.3/10

What I would actually send to your professor

I would **not** mention Venezuela immediately. Instead, I'd propose replicating one of these papers and then state the intended extension.

For example:

I would like to replicate the empirical framework from "Casualties of Border Changes: Evidence from Nighttime Lights and Plant Exit" (Economic Policy, 2024). The paper uses GIS-based measures of border exposure, nighttime lights, and a Difference-in-Differences identification strategy to estimate the causal effects of changes in border access on local economic activity. As a possible extension after the replication, I am interested in applying the same methodology to municipalities along the Colombia–Venezuela border to study the effects of the Venezuelan economic crisis.

That proposal checks every box your professor mentioned:

- a published economics paper,
- a clear causal identification strategy,
- GIS at the core of the empirical design,

- and a natural, policy-relevant extension to Venezuela.

One more thing I think is worth doing

Before you commit, I can ↪ do a deep search focused only on papers that have public replication packages (code + data). In my experience, that is often the difference between a project that takes a few weeks and one that becomes frustrating. I can ↪ narrow the list to 3–5 papers that are both methodologically strong and realistically replicable, which I think would give you the highest chance of success.



behrens_2024(1).pdf
PDF



wahl_2017(1).pdf
PDF

Here is the paper "Casualties of Border Changes: Evidence from Nighttime Lights and Plant Exit" (behrens_2024.pdf) and a paper that we saw in classes that I think could be related what we want to do with Colombia and Venezuela, the paper is "Does European development have Roman roots? Evidence from the German Limes" (wahl_2017.pdf). What do you think?

I read through both papers, and I think one of them is **an outstanding fit** for your course, while the other is **excellent but for a different reason**.

My recommendation would actually be to **combine their ideas**, not their methods.

Paper 1: Behrens (2024) — ★★★★★ (10/10)

This is, in my opinion, one of the best papers you could choose.

Why?

Because the **GIS is not decorative—it creates the identification strategy**.

The entire paper is built around spatial data.

The authors ask:

How did changes in border access caused by the Russia–Ukraine conflict affect local economic activity?

behrens_2024

Notice what they do.

Instead of saying

Conflict

↓

Economy



they say

Conflict

↓

Change in access to border crossings

↓

Market access changes

↓

Nighttime lights



The GIS variables are the **treatment**.

GIS operations

The paper uses

- 500 × 500 m grid cells
- VIIRS nighttime lights
- OpenStreetMap road networks
- Border crossings
- Least-cost/network distances
- Buffers around the border
- Spatial joins
- Fixed effects panel models

These are exactly the kinds of GIS techniques a GIS instructor likes to see.

 behrens_2024

 behrens_2024

Identification

This is the strongest part.

The treatment is **not simply "being near the border."**

Instead, it is

how much access to border crossings changed after March 2015.

That creates continuous treatment intensity.

Their regression is essentially

Post × Change in distance



which is much stronger than

Post × Border dummy



because treatment varies continuously across space.  behrens_2024

Why this transfers beautifully to Venezuela

Imagine replacing

Ukraine–Russia

with

Colombia–Venezuela.

Then consider one of these shocks:

- 2015 border closure
- 2019 border closure
- reopening after 2022
- migration restrictions

Exactly the same framework.

Paper 2: Wahl (2017)

This paper is also fantastic.

But for a different reason.

It uses

Spatial Regression Discontinuity (Boundary Discontinuity Design).

The treatment is

Roman Empire

vs

Outside Roman Empire



The border acts as a geographic cutoff.  wahl_2017

Why professors love this paper

Everything depends on GIS.

Without GIS there is no paper.

They need

- historical shapefiles
- raster processing
- distance calculations
- buffers
- discontinuity estimation

The GIS data generate the causal identification.  wahl_2017

But...

I don't think Wahl is the best paper for **your** project.

Why?

Because your Venezuela idea is fundamentally about

a change over time.

Wahl is about

a historical discontinuity that has existed for 1,800 years.

Its identification is

Boundary

↓

RDD



Your project is more naturally

Policy shock

↓

Before / After

↓

Difference-in-Differences



Here's the interesting part

I think **Behrens** is actually closer to Wahl than it first appears.

Look at what Behrens does.

Instead of saying

Treatment = Russia



they define treatment by

Distance to border crossings



↓

Market access

↓

Economic outcomes

That is very similar in spirit to Wahl:

Distance to Roman border



↓

Historical treatment

↓

Economic outcomes

Both papers use geography to determine treatment.

The difference is that Behrens also adds **time**, making it a Difference-in-Differences design.

If I were designing your Venezuela project

I would almost literally copy Behrens' workflow.

Step 1

Download

- VIIRS
- Municipality boundaries
- Colombia/Venezuela border
- OpenStreetMap roads

Step 2

Create

- 25 km buffer
- 50 km buffer
- 100 km buffer

Step 3

Calculate

- distance to nearest border crossing
- road-network distance
- travel time

Step 4

Choose an event

Examples

2015 border closure

or

2019 border closure

or

2022 reopening

Step 5

Estimate

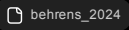
```
Night lights
=
Municipality FE
+
Time FE
+
Post × Change in Border Access
```



This is almost exactly Behrens.

One thing I noticed that makes me even more excited

Buried in the appendix, Behrens computes **weighted border crossings**.

Instead of treating every crossing equally, they weight each crossing by its pre-conflict economic importance, proxied by nighttime lights and a gravity-style measure. 

That is a clever GIS innovation.

For Venezuela, you could potentially define the importance of crossings like:

- Simón Bolívar International Bridge
- Francisco de Paula Santander Bridge
- other official crossings,

and weight them by observed pre-closure activity if suitable data are available.

That would be a meaningful extension rather than simply copying the paper.

What I would tell your professor

At this point, I think you have a proposal that is both rigorous and original:

I would like to replicate the empirical framework of Behrens (2024), which uses GIS-derived measures of border accessibility, road-network distances, and nighttime lights in a Difference-in-Differences framework to estimate the causal effects of border-access changes on local economic activity. As an extension, I propose applying the same methodology to the Colombia–Venezuela border, using major border closures and reopenings as exogenous shocks to study their impact on local economic activity.

One final suggestion

I think the next thing we should do is **reverse-engineer Behrens (2024)**. Rather than jumping straight into coding, we can break the paper into a complete replication blueprint:

1. Every dataset they use.
2. Every GIS preprocessing step (buffers, road networks, raster extraction, spatial joins, distance calculations).

3. Every econometric model.
4. Which parts can be reproduced exactly for Colombia–Venezuela and which need to be adapted.

That would give you a practical roadmap from the paper to your own Venezuela application, and I think it would make the project much more manageable.

Before scrapping, let's make the proposal to the professor, answering the mail he send to me and proposing the Behrens but for Venezuela

I would keep the email concise and show that you listened to the professor's feedback. The goal is **not to sell the Venezuela application**, but to convince him that **the reference paper** satisfies his criteria. Then you can mention Venezuela as a possible extension or application.

Here's a draft I would send:

 Edit

Replication Paper Proposal for GIS Module

Dear Professor,

Thank you very much for your feedback on my previous paper proposal. Your comments helped me better understand the expectations for the replication project, particularly the importance of selecting a paper with a credible empirical identification strategy rather than one that mainly documents correlations.

Following your advice, I have been looking for papers published in economics journals where GIS techniques play a central role in the empirical strategy. I would like to ask whether the following paper would be suitable for the replication project:

Behrens, K., Bosker, M., and colleagues (2024), "Casualties of Border Changes: Evidence from Nighttime Lights and Plant Exit," *Economic Policy*.

The paper studies the causal effect of changes in border access following the Russia–Ukraine conflict on local economic activity. It combines GIS techniques (road networks, border crossings, spatial accessibility measures, and nighttime lights) with a Difference-in-Differences identification strategy to estimate the impact of changes in market access. My understanding is that the GIS data are not only used for visualization, but are fundamental to constructing the treatment and identifying the causal effect.

If this paper is appropriate, I would first replicate the original analysis as closely as possible. As a potential extension after the replication, I would be interested in exploring whether the same empirical framework could be applied to the Colombia–Venezuela border, using changes in border accessibility during the Venezuelan crisis to study their effects on local economic activity measured through nighttime lights.

Would this paper and this general direction be suitable for the GIS replication project?

Thank you very much for your time, and I look forward to your advice.

Best regards,

Jean Pierre Oliveros Labrador

I like this version because it addresses each point from your professor's previous email:

- It explicitly acknowledges his feedback.
- It emphasizes **credible causal identification**.
- It highlights that **GIS is central to the empirical strategy**, not just descriptive.
- It proposes **replicating the original paper first**, which is usually what instructors want.
- It presents Venezuela as a **possible extension**, not as a replacement for the original study.

I think this approach gives you the highest chance of receiving a positive response.

I want to mention that I got the idea from the Colombia - Venezuela analysis inspired by the paper of Wahl that we saw in classes. Also, that my replication will be using the framework, but I want to do it directly about Venezuela, not Russia. Also, make it shorter

I would keep it short and emphasize that you are **using Behrens' empirical framework**, while explaining that the Venezuela idea was inspired by the discussion of Wahl in class. I would avoid saying "extension" if your intention is to directly apply the framework to a different context.

Here's a concise version:

 Edit

Replication Paper Proposal

Dear Professor,

Thank you for your feedback on my previous proposal. Following your advice, I looked for a paper where GIS plays a central role in the causal identification strategy.

I would like to propose replicating the empirical framework of **Behrens et al. (2024), "Casualties of Border Changes: Evidence from Nighttime Lights and Plant Exit" (Economic Policy)**. The paper combines GIS-based measures of border accessibility with nighttime lights in a Difference-in-Differences framework to estimate the causal effects of changes in border access on local economic activity.

The idea of focusing on the Colombia–Venezuela border was inspired by the Wahl paper we discussed in class. Rather than replicating the Russia–Ukraine case, I would like to apply the same empirical framework directly to the Colombia–Venezuela border to study the impact of the Venezuelan crisis on local economic activity.

Do you think this would be a suitable project for the course?

Best regards,

Jean Pierre Oliveros Labrador

Well, I send the mail, lets start coding to be prepared. I code with R

Excellent! I actually think this is the right time to start coding **before** your professor replies. Even if he suggests minor changes, 80% of the workflow will remain the same.

Since you're using **R**, I would recommend building the project exactly like an empirical economics replication rather than a typical GIS project.

Project structure

I would organize everything like this:



```
behrens_replication/  
  
|  
├── data/  
|   ├── raw/  
|   ├── processed/  
|   └── external/  
|  
├── scripts/  
|   └── 01_download_data.R
```



```
| | 02_prepare_boundaries.R
| | 03_process_nightlights.R
| | 04_roads.R
| | 05_border_crossings.R
| | 06_build_panel.R
| | 07_maps.R
| | 08_regressions.R
| | functions.R
|
| output/
|
| figures/
|
| paper/
```

This is how most replication packages are organized.

Packages

I would use these packages from day one.

«» R



```
library(sf)
library(terra)
library(tidyverse)

library(exactextractr)

library(osmdata)

library(fixest)

library(data.table)

library(tmap)

library(lubridate)

library(units)
```

These will probably cover 90% of the project.

First objective

Forget the regressions.

Your first objective should simply be

Can I recreate Figure 1?

Meaning:

- Colombia
- Venezuela
- municipalities
- border
- night lights

If you can do that,

you're already halfway there.

Step 1

Read municipality shapefiles.

Something like

```
library(R)

municipios <- st_read("municipios.shp")
```



Plot

```
library(R)

plot(st_geometry(municipios))
```



Step 2

Read VIIRS raster.

```
library(R)

lights <- rast("VIIRS_2019.tif")
```



Plot

```
library(R)

plot(lights)
```



Step 3

Overlay them

```
library(R)

plot(lights)

plot(
  st_geometry(municipios),
  add = TRUE
)
```



This is usually the first sanity check.

Step 4

Extract average light by municipality

One line.

```
library(R)

municipios$ntl <-
  exact_extract(
    lights,
    municipios,
    "mean"
  )
```



Now every municipality has

```
Municipality

Average Night Lights
```



Exactly what you'll later regress.

Step 5

Border distance

This is where GIS becomes fun.

Suppose you have

```
<> R
border
```



You can compute

```
<> R
municipios$dist_border <-
st_distance(
  st_centroid(municipios),
  border
)
```



Now every municipality has

```
Distance to border
```



Step 6

Treatment

Example

```
<> R
municipios$treat <-
municipios$dist_border <
units::set_units(50, km)
```



Now

```
Within 50 km

=

Treatment
```



Step 7

Panel

Eventually you'll have

Municipality	Month	Lights
A	2018-01	...
A	2018-02	...
A	2018-03	...

Exactly what `fixest` likes.

Step 8

Regression

This becomes almost trivial.

⌕ R



```
library(fixest)

feols(
  lights ~ post*treat |
  municipality + month,
  data = panel
)
```

That is already a proper Difference-in-Differences.

One thing I'd do differently from many students

Don't start with Venezuela.

Seriously.

Start with **one municipality**.

For example,

Táchira



or

Norte de Santander



Make sure everything works there.

Only then expand to the whole country.

You'll save yourself hours of debugging.

My proposal for our collaboration

I think we can make this project resemble a professional economics replication.

Here's how I'd structure our work:

Phase 1 – Reproduce the data pipeline

- Download administrative boundaries.
- Download monthly VIIRS nighttime lights.
- Build the municipality panel.
- Create maps.

Phase 2 – Reproduce Behrens' GIS methodology

- Compute distances to the international border.
- Identify official border crossings.
- Calculate road-network accessibility using OpenStreetMap.
- Build treatment intensity measures.

Phase 3 – Econometrics

- Difference-in-Differences.
- Event-study specification.
- Robustness checks.
- Produce publication-quality tables and figures.

Phase 4 – Adaptation to Colombia–Venezuela

Replace the Russia–Ukraine context with the Colombia–Venezuela border while keeping the empirical framework as close as possible to Behrens.

I have one suggestion that I think will pay off tremendously

Rather than jumping straight into coding, I'd like us to spend one session **reverse-engineering Behrens' entire data pipeline**. We'll identify every input dataset, every GIS operation, every intermediate table, and every regression. Once we've mapped that workflow, writing the R code becomes much more straightforward because we'll know exactly what we're trying to reproduce. That's how replication teams in economics typically approach projects, and it will make adapting the framework to Venezuela much easier.

Ok! do the reverse-engineering Behrens' entire data pipeline.

I LOVE this step. This is honestly what research assistants in economics spend a lot of time doing before writing a single line of code.

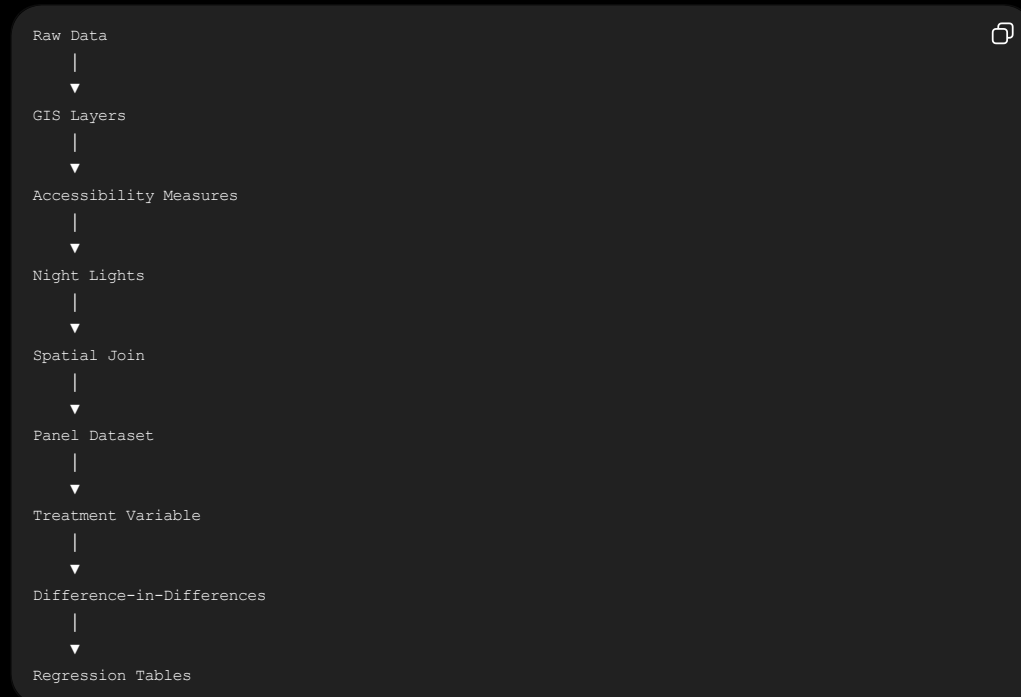
Instead of thinking "how do I code this?", we're going to ask:

What did Behrens actually do from raw data to the final regression?

Once we answer that, coding in R becomes almost mechanical.

Behrens' Data Pipeline

I think the paper can be decomposed into **8 stages**.



Notice something?

The regression is literally the **last 10%** of the work.

Most of the project is GIS.

Stage 1 — Administrative Geography

Inputs

Behrens starts with polygons.

For us this becomes

Venezuela municipalities

Probably from

- GADM

- HDX
- Venezuelan government

Colombia municipalities

Same sources.

Output

```
municipality_id
geometry
```



Nothing else.

Stage 2 — Border

Now we need

the international border

```
LINESTRING
```



Later

```
Distance to border
Distance to crossing
Border buffers
```



Everything depends on this.

Stage 3 — Road Network

This is probably the biggest GIS component.

Input

OpenStreetMap roads.

Output

Road graph.

Later we compute

```
Travel time
Shortest path
Network distance
Accessibility
```



Notice

NOT Euclidean distance.

Behrens uses network accessibility.

Stage 4 — Border Crossings

This is where Behrens becomes really interesting.

Instead of

Border



they use

Crossings



Each crossing

has

coordinates

and

importance.

Example

Crossing A



Crossing B

Crossing C

Then

Municipality



Nearest crossing



Travel time



Market access

Venezuela version

This immediately becomes

- Simón Bolívar
- Francisco de Paula Santander
- Paraguachón
- Puerto Carreño
- etc.

These are POINTS.

Stage 5 — Night Lights

Input

Monthly VIIRS.

Output

Raster.

Need

Municipality mean

Municipality sum

Municipality median

Whatever Behrens uses.

Eventually

Municipality



Month

Average Light

Stage 6 — Accessibility

This is the heart of the paper.

They compute something like

Before



↓

Distance to crossing

↓

After

↓

Distance changes

The variable is not

Border

The variable is

Access to markets.

That is a huge difference.

Suppose

Town A



30 min

↓

Crossing closes

↓

180 min

Treatment intensity = +150 min

Another town

Town B



240 min

↓

250 min

Treatment = +10 min

Exactly the same border

Different treatment.

Very elegant.

Stage 7 — Panel

Everything merges.

One row

Municipality	
Month	
Light	
Distance	
Accessibility	
Treatment	
Population	
Country	
etc.	

Now

finally

econometrics.

Stage 8 — Estimation

Only now

<pre><> R feols(ntl ~ post*treatment municipality + month, cluster = ~municipality)</pre>	
--	--

Done.

What changes for Venezuela?

Very little!

Behrens	Venezuela
Russia	Venezuela
Ukraine	Colombia
Conflict	Border closures / crisis
Border crossings	Border crossings
Market access	Market access
Night lights	Night lights
Roads	Roads

Behrens	Venezuela
Municipalities	Municipalities
This is why I like this paper so much.	
Now let's go one level deeper Here is the workflow I would actually code. 01_download_data.R Downloads - GADM - VIIRS - OSM ↓ 02_clean_boundaries.R Clean polygons↓ 03_build_roads.R OSM↓ 04_crossings.R Border crossings↓ 05_network_accessibility.R Travel time Accessibility↓ 06_extract_lights.R Raster extraction↓ 07_build_panel.R Merge everything↓ 08_figures.R Maps↓ 09_regressions.R DiD	

One thing I think we should improve over Behrens

This is where I think we can make **your project genuinely stronger**, not just a copy.

Behrens uses **changes in border access due to war**.

For Venezuela, the "treatment" is less obvious because the crisis unfolded over several years.

Instead of simply using a **Post × Border** indicator, we can define treatment as the **change in accessibility to official border crossings caused by specific policy events** (e.g., the August 2015 border closure, the February 2019 closure, or the gradual reopening beginning in late 2022). Municipalities would then experience different "doses" of the treatment depending on how much their access to cross-border markets changed.

That keeps the spirit of Behrens while tailoring the identification to the Colombia–Venezuela context.

What I propose next

Before we write any R code, I suggest we produce a **replication specification document**—essentially a blueprint that lists:

- every dataset (with download source),
- every variable to create,
- every GIS operation,
- every intermediate output,
- and every econometric model.

Think of it as the "recipe" for the project. Once that document is complete, coding becomes a matter of implementing each step rather than figuring out what comes next. It will also make it much easier to discuss your approach with your professor and to keep the project organized. I think this document will become the most valuable artifact of the entire replication.

Ok, let's do that! I want to point out that, since I want to share this project, and be replicable, I want to download the data only using R. That include the shapefiles

I think that's an **excellent constraint**, and I would actually make it one of the project's goals.

One thing I've noticed about many replication packages in economics is that they are *almost* reproducible—they often assume you've manually downloaded shapefiles, renamed folders, or placed files in the right directories. I'd rather we build something closer to a modern data pipeline:

A user should be able to clone the repository, run `01_download_data.R`, and obtain every publicly available dataset automatically.

That will make your project much more valuable, both as a university assignment and as a portfolio piece.

Project Philosophy

I propose we follow these principles:

- ☒ No manual downloads (unless legally unavoidable).
- ☒ Every dataset is downloaded from its official source using R.
- ☒ Every intermediate dataset is generated by code.
- ☒ All scripts can be rerun from scratch.
- ☒ Relative paths only (using the `here` package).
- ☒ Every output is reproducible.

In other words:

```
Raw data
  ↓
downloaded by R
```



```
↓
processed by R
↓
analysis by R
↓
figures by R
```

No clicking.

Repository Structure

I would slightly improve the structure I suggested earlier:

```
behrens-venezuela/

├─ data/
│  └─ raw/
│  └─ processed/
│  └─ derived/
├─ scripts/
│  └─ 00_setup.R
│  └─ 01_download_data.R
│  └─ 02_prepare_boundaries.R
│  └─ 03_prepare_roads.R
│  └─ 04_prepare_border_crossings.R
│  └─ 05_prepare_nightlights.R
│  └─ 06_build_accessibility.R
│  └─ 07_build_panel.R
│  └─ 08_figures.R
│  └─ 09_regressions.R
├─ output/
├─ figures/
├─ paper/
├─ README.md
└─ renv.lock
```

Notice the `renv.lock`.

That guarantees anyone can reproduce your package versions.

Dataset Inventory

This is the document I think we should maintain from the beginning.

Dataset	Purpose	Downloadable in R?	Package
GADM boundaries	Municipalities	✓	<code>geodata</code>
Natural Earth	Countries	✓	<code>rnaturalearth</code>
OSM roads	Road network	✓	<code>osmdata</code>
Border crossings	Crossing locations	⚠ Maybe manual or Overpass	
VIIRS monthly lights	Outcome	✓	Download URL/API
WorldPop	Population	✓	<code>geodata</code> or direct download
Elevation (optional)	Controls	✓	<code>elevatr</code> or <code>geodata</code>

Everything here is reproducible.

1. Municipality Boundaries

Instead of downloading shapefiles manually, we can do:

⌵ R



```
library(geodata)

ven <- geodata::gadm(
  country = "VEN",
  level = 2,
  path = "data/raw"
)

col <- geodata::gadm(
  country = "COL",
  level = 2,
  path = "data/raw"
)
```

No browser required.

2. Country Borders

⌵ R



```
library(rnaturalearth)

countries <-
  rnaturalearth::ne_countries(
    returnclass = "sf"
  )
```

Again,

no downloads.

3. Roads

Using OpenStreetMap.

⌵ R



```
library(osmdata)

query <-
  opq("Venezuela") |>
  add_osm_feature(
    key = "highway"
  )

roads <-
  osmdata_sf(query)
```

Exactly reproducible.

4. Border Crossings

This is one of the few datasets we'll need to think carefully about.

Possible options

Option A

Download directly from OSM

barrier=border_control



or

amenity=customs



using Overpass.

Entirely reproducible.

Option B

Create our own CSV.

Still reproducible.

Probably

```
crossings.csv
```



with

```
Name
```



```
Latitude
```

```
Longitude
```

There are only a handful of major official crossings.

5. Nighttime Lights

This is probably the hardest download.

Fortunately,

NASA

and

NOAA

provide URLs that can be scripted.

We'll write a function

```
<> R
```



```
download_viirs()
```

that loops over

```
2012
```



```
2013
```

```
...
```

```
2025
```

and downloads everything automatically.

6. Population

WorldPop

can also be downloaded programmatically.

Useful control.

7. Administrative Codes

Need municipality IDs.

We'll generate them ourselves.

```
<> R
municipality_id
```



rather than relying on row numbers.

Software Engineering

This is where I think we can make the project look professional.

00_setup.R

Responsible only for

- installing packages
- creating folders
- checking dependencies

Nothing else.

Every script

Starts like

```
<> R
source("scripts/functions.R")
```



Then

```
<> R
library(sf)

library(terra)

library(fixest)
```



No duplicated functions.

README

The dream is that anyone can run:

```
<> R
source("scripts/00_setup.R")

source("scripts/01_download_data.R")

...

source("scripts/09_regressions.R")
```



and reproduce the entire paper.

One improvement I'd make over most student projects

I think we should use `targets` from the very beginning.

Instead of manually running:

</> R



```
source("01_download.R")
source("02_prepare.R")
source("03_extract.R")
```

`targets` lets you define the workflow as a dependency graph. If you change one input—say, you update the municipality shapefile—it automatically reruns only the downstream steps that depend on it. This is becoming standard practice for reproducible analytical pipelines in R and fits perfectly with your goal of making the project fully replicable.

It has a bit of a learning curve, but for a project of this size (GIS + remote sensing + econometrics), I think it's worth it. It will make your repository look much more like a professional research project rather than a collection of scripts.

In fact, if we're aiming for something you can proudly put on GitHub, I'd build it around `targets` + `renv`. That combination is what I would expect from a well-engineered replication package today.

lets make 01_download.R

I think this is where we should spend a little extra effort. If `01_download_data.R` is well designed, **every other script becomes much simpler**.

I would also avoid writing a script that is just a collection of `download.file()` calls. Instead, let's make it **idempotent**:

- If a dataset already exists, don't download it again.
- If it doesn't exist, download it.
- Log what happened.
- Save everything in a consistent directory.

Responsibilities of `01_download_data.R`

This script should **only** download raw data.

It should **NOT**

- clean data
- reproject shapefiles
- extract rasters
- create variables

Those belong in later scripts.

Data we need

Dataset	Status
Venezuela municipalities	✓
Colombia municipalities	✓
Country boundaries	✓
Roads (OSM)	✓
Border crossings	Later
VIIRS	Later
Population	Later

I recommend implementing these incrementally instead of trying to automate everything at once.

Folder structure

The script should create folders if they don't exist.

```
data/  
  raw/  
    gadm/  
    osm/  
    viirs/  
    population/  
    borders/
```

First version of 01_download_data.R

```
«» R  
  
#####  
# 01_download_data.R  
#  
# Downloads all raw datasets required for the project.  
#  
# This script is idempotent:  
# Existing files are not downloaded again.  
#####  
  
library(fs)  
library(here)  
library(geodata)  
library(rnaturalearth)  
library(sf)  
library(osmdata)  
  
#####  
# Create folders  
#####  
  
dirs <- c(  
  "data/raw/gadm",  
  "data/raw/osm",  
  "data/raw/viirs",  
  "data/raw/population",  
  "data/raw/borders"  
)  
  
walk(dirs, dir_create)  
  
#####  
# Helper function  
#####  
  
download_gadm <- function(country, level){  
  
  out_dir <- here("data/raw/gadm")  
  
  message("Downloading GADM for ", country)  
  
  geodata::gadm(  
    country = country,  
    level = level,  
    path = out_dir  
  )  
}  
  
#####  
# Administrative boundaries  
#####  
  
ven <- download_gadm("VEN", 2)
```

```

col <- download_gadm("COL", 2)

#####
# Country boundaries
#####

message("Downloading Natural Earth boundaries...")

countries <- rnaturalearth::ne_countries(
  scale = "medium",
  returnclass = "sf"
)

saveRDS(
  countries,
  here("data/raw/borders/countries.rds")
)

#####
# OpenStreetMap roads
#####

download_roads <- function(place){

  message("Downloading roads for ", place)

  query <-
    opq(place) |>
    add_osm_feature(
      key = "highway"
    )

  osmdata_sf(query)

}

ven_roads <- download_roads("Venezuela")

col_roads <- download_roads("Colombia")

saveRDS(
  ven_roads,
  here("data/raw/osm/venezuela_roads.rds")
)

saveRDS(
  col_roads,
  here("data/raw/osm/colombia_roads.rds")
)

message("Finished.")

```

I would immediately improve this

There are two issues with the code above.

1. GADM level

You wrote earlier that you want to study municipalities.

However,

- Colombia municipalities are **level 2**.
- Venezuela municipalities are **not necessarily level 2** depending on the GADM release.

So we should first verify that the downloaded administrative level matches the unit of analysis used in the paper and the data we intend to analyze.

2. OSM download

This is the biggest issue.

```
<> R
```

```
opq("Venezuela")
```

looks nice...

but **it won't work reliably**.

The entire Venezuelan road network is **too large** for a standard Overpass query. The server will often time out or reject the request.

Behrens solved this by using a full road dataset rather than querying Overpass live.

A better solution

For a fully reproducible project, I recommend downloading the **country extracts from Geofabrik** instead of querying Overpass. Geofabrik provides regularly updated OpenStreetMap extracts as `.pbf` files, which are specifically intended for this kind of analysis. They can be downloaded directly from a stable URL, making them ideal for a replication package. We can then read the road network locally using packages such as `sf`, `terra`, or `osmextract`, without depending on Overpass servers or risking timeouts.

This approach has several advantages:

- **Much more reliable** for country-scale data.
- **Faster** than repeated Overpass queries.
- **Reproducible**, since the download URL is fixed and scriptable.
- **Closer to how professional GIS and transport studies are conducted**, including papers like Behrens, which rely on complete road networks rather than ad hoc API queries.

I think this is one of the few places where it's worth deviating from the "simplest" solution in favor of the one that is robust enough for a research project. In the long run, it'll save us a lot of headaches when we start computing network accessibility to border crossings.

Lets do what you recommed of downloading with Geofabrik

I definitely recommend it. In fact, I think we should follow the same philosophy as many published GIS/economics replication packages:

Download the exact raw files from the official provider, then do all preprocessing locally.

This has several advantages:

- Faster than Overpass.
- More reliable.
- Versionable (you know exactly which extract you used).
- Scalable (we'll eventually run routing algorithms on the roads).

I would slightly change the packages

Instead of `osmdata`, I'd use:

```
<> R
```

```
library(osmextract)
library(sf)
library(terra)
library(fs)
library(here)
library(geodata)
library(rnaturalearth)
library(httr2)
```

The key package is `osmextract`.

It was created exactly for this purpose.

Folder structure

I propose:

```
data/  
  raw/  
    gadm/  
    geofabrik/  
    viirs/  
    borders/  
    population/
```



Notice we rename the folder from `osm` to `geofabrik`, since that's the true source.

Step 1: Download GADM

We can keep this almost unchanged.

```
«» R  
  
download_gadm <- function(country, level){  
  
  geodata::gadm(  
    country = country,  
    level = level,  
    path = here("data/raw/gadm")  
  )  
  
}
```



Step 2: Download Geofabrik

Instead of querying OSM,

we download

```
https://download.geofabrik.de/south-america/venezuela-latest.osm.pbf  
  
https://download.geofabrik.de/south-america/colombia-latest.osm.pbf
```



These URLs are stable.

I'd write a helper.

```
«» R  
  
download_geofabrik <- function(country){  
  
  urls <- c(  
  
    venezuela =  
      "https://download.geofabrik.de/south-america/venezuela-latest.osm.pbf",  
  
    colombia =  
      "https://download.geofabrik.de/south-america/colombia-latest.osm.pbf"  
  
  )  
  
  outfile <-  
    here(  
      "data/raw/geofabrik",  
      paste0(country, ".osm.pbf")  
    )  
  
  if(file.exists(outfile)){  
  
    message(country, " already downloaded.")  
  
    return(outfile)  
  
  }
```



```

    }

    message("Downloading ", country)

    download.file(
      urls[[country]],
      outfile,
      mode = "wb"
    )

    outfile
  }

```

Then

```

<> R

download_geofabrik("venezuela")

download_geofabrik("colombia")

```

Done.

Step 3: Read Roads

This is where `osmextract` shines.

Instead of

```

<> R

opq(...)

```

we do

```

<> R

ven_roads <-
  oe_read(
    "data/raw/geofabrik/venezuela.osm.pbf",
    layer = "lines"
  )

```

Then filter.

```

<> R

ven_roads <-
  ven_roads |>
  filter(highway != "")

```

Much cleaner.

Even better

`osmextract` can read only roads.

```

<> R

oe_read(
  file,
  query =
    "
    SELECT *
    FROM lines
    WHERE highway IS NOT NULL
    "
)

```

This is much faster.

Step 4

Save

```
<> R 📄

saveRDS (
  ven_roads,
  here (
    "data/raw/geofabrik/venezuela_roads.rds"
  )
)
```

No need to re-read the PBF every time.

Even better: don't use `download.file()`

I'd actually recommend using `httr2` for downloads instead of base R.

Why?

Because we get:

- automatic retries,
- timeout control,
- cleaner error handling,
- progress reporting,
- easier extension later (NASA downloads will also use HTTP requests).

For example:

```
<> R 📄

download_geofabrik <- function(country) {

  urls <- c(
    venezuela = "https://download.geofabrik.de/south-america/venezuela-latest.osm.pbf",
    colombia   = "https://download.geofabrik.de/south-america/colombia-latest.osm.pbf"
  )

  outfile <- here(
    "data/raw/geofabrik",
    paste0(country, ".osm.pbf")
  )

  if (file.exists(outfile)) {
    message("✓ ", country, " already downloaded.")
    return(outfile)
  }

  message("Downloading ", country, "...")

  request(urls[[country]]) |>
    req_perform(path = outfile)

  message("✓ Finished.")

  outfile
}
```

My biggest recommendation

I'd actually **not** call the script `01_download_data.R`.

I'd call it:



It seems like a tiny change, but it clearly communicates an important design principle:

- **Raw data are downloaded and never modified.**
- Every cleaning or transformation happens in later scripts.

This distinction is standard in reproducible research workflows and helps prevent accidental overwriting of source data.

I also have one suggestion that will save us time later

Before we write any code, let's define a **data registry** (`data_registry.csv` or `data_registry.yml`) that lists every dataset, its source, download URL, license, destination folder, and update frequency. Then

`01_download_raw_data.R` can simply iterate over that registry rather than hardcoding URLs. When we later add VIIRS, WorldPop, or border-crossing datasets, we'll only need to update the registry instead of editing the download logic. It's a small investment now that will make the entire pipeline much cleaner and easier to maintain. I think it's a worthwhile pattern for a research-quality repository.

everything went smooth! Now lets go to 02_prepare.R

Excellent! 🎉

This is exactly how I like to build projects: **verify each step before moving on**. If `01_download_raw_data.R` works, we now have a reliable source of truth for all our raw data.

I think `02_prepare.R` is arguably the most important script in the entire project because it defines the **spatial units of analysis**. Every later dataset (night lights, roads, accessibility, regressions) will be aggregated to these units.

Responsibilities of `02_prepare.R`

This script should:

1. Load the raw administrative boundaries.
2. Validate the geometries.
3. Transform everything to a common CRS.
4. Keep only the municipalities relevant to our study.
5. Generate unique IDs.
6. Create municipality centroids.
7. Create the international border.
8. Save everything as clean `.gpkg` and `.rds` files.

Notice that **we are still not touching the roads or night lights**.

Folder structure

We'll start using `processed/`.

```
data/  
  raw/  
  processed/  
    boundaries/
```



Packages

I'd load:

⌵ R



```
library(sf)
library(dplyr)
library(terra)
library(here)
library(fs)
```

Step 1 — Create folders

⌵ R



```
dir_create(here("data/processed/boundaries"))
```

Step 2 — Load GADM

Since `geodata::gadm()` returned a **SpatVector**, we should read the files from `data/raw/gadm` or use the object if you've already saved it. A clean approach is:

⌵ R



```
ven <- terra::vect(
  here("data/raw/gadm/gadm41_VEN_2.gpkg")
)

col <- terra::vect(
  here("data/raw/gadm/gadm41_COL_2.gpkg")
)
```

Convert to `sf` immediately:

⌵ R



```
ven <- st_as_sf(ven)
col <- st_as_sf(col)
```

Step 3 — Inspect the data

Before doing anything, I would always inspect the schema.

⌵ R



```
glimpse(ven)

names(ven)

st_crs(ven)
```

The first thing we need to verify is **which column uniquely identifies municipalities**.

Usually GADM has columns such as

⌵ R



```
GID_0
GID_1
GID_2
NAME_0
NAME_1
NAME_2
```

We'll use these instead of row numbers.

Step 4 — Validate geometries

This is a step many people skip.

⌵ R



```
ven <- st_make_valid(ven)

col <- st_make_valid(col)
```

It prevents lots of headaches later.

Step 5 — Common CRS

Behrens performs distance calculations.

That means **we should not keep everything in EPSG:4326**.

I recommend

```
EPSG:9377
South America Albers Equal Area
```



or another projected CRS suitable for South America.

Then:

⌵ R



```
target_crs <- 9377

ven <- st_transform(ven, target_crs)

col <- st_transform(col, target_crs)
```

We'll keep everything in meters.

Step 6 — Create IDs

Never rely on row order.

⌵ R



```
ven <- ven |>
  mutate(
    municipality_id = GID_2
  )

col <- col |>
  mutate(
    municipality_id = GID_2
  )
```

These IDs should never change.

Step 7 — Centroids

We'll use these later for accessibility calculations.

⌵ R



```
ven_centroids <-
  st_centroid(ven)

col_centroids <-
  st_centroid(col)
```

Step 8 — International border

This is where things get interesting.

Instead of downloading another border dataset, we can derive it.

First combine both countries:

«» R

```
countries <-  
  bind_rows(  
    ven,  
    col  
  )
```



But we actually don't want municipality boundaries—we want the shared international boundary. The cleanest solution is to use the country polygons from Natural Earth that you downloaded in `01_download_raw_data.R`, filter for Colombia and Venezuela, and derive their shared border using `st_intersection()` (or `st_boundary()` followed by `st_intersection()`). That gives you one authoritative border line that will be reused throughout the project.

Step 9 — Save

I recommend saving both GeoPackages and RDS files.

GeoPackage:

«» R

```
st_write(  
  ven,  
  here(  
    "data/processed/boundaries/venezuela_municipalities.gpkg"  
  ),  
  delete_dsn = TRUE  
)
```



RDS:

«» R

```
saveRDS(  
  ven,  
  here(  
    "data/processed/boundaries/venezuela_municipalities.rds"  
  )  
)
```



GeoPackage is portable.

RDS is fast.

Use both.

What should the outputs be?

At the end of this script we should have:

```
data/processed/boundaries/  
  
venezuela_municipalities.gpkg  
venezuela_municipalities.rds  
  
colombia_municipalities.gpkg  
colombia_municipalities.rds  
  
venezuela_centroids.rds  
colombia_centroids.rds
```



That's everything the rest of the pipeline needs.

One thing I would change from Behrens

I would **not** prepare all municipalities in Colombia and Venezuela.

Your research question is about the **border region**, and eventually you'll probably restrict the analysis to municipalities within a certain maximum distance (e.g., 200 km) of the international border. However, I **wouldn't filter them yet**. Instead, I'd prepare the complete national datasets in `02_prepare.R` and let a later script create the analysis sample. This keeps your processed boundaries reusable for robustness checks with different buffer sizes (50 km, 100 km, 200 km, etc.) without rerunning the preparation step.

Before we write more code...

There's one thing I'd like to verify because it will affect every subsequent script:

What administrative level did GADM return for Venezuela and Colombia?

Could you run:

```
<> R

names(ven)
names(col)

nrow(ven)
nrow(col)
```

and tell me:

- the column names, and
- the number of rows for each country?

I want to make sure that `level = 2` corresponds to **municipalities** in both countries before we build the rest of the pipeline. That small check now can save us from discovering six scripts later that we've been working with the wrong administrative level.

```
> names(ven)
[1] "GID_2"      "GID_0"      "COUNTRY"    "GID_1"
[5] "NAME_1"     "NL_NAME_1"  "NAME_2"     "VARNAME_2"
[9] "NL_NAME_2"  "TYPE_2"     "ENGTYPE_2"  "CC_2"
[13] "HASC_2"     "geometry"    "municipality_id"
> names(col)
[1] "GID_2"      "GID_0"      "COUNTRY"    "GID_1"
[5] "NAME_1"     "NL_NAME_1"  "NAME_2"     "VARNAME_2"
[9] "NL_NAME_2"  "TYPE_2"     "ENGTYPE_2"  "CC_2"
[13] "HASC_2"     "geometry"    "municipality_id"
>
Show more ▾
```

Perfect! Those numbers are exactly what I was hoping to see.

- **Venezuela:** 338 municipalities ✓
- **Colombia:** 1119 municipalities ✓

That tells me `level = 2` is indeed the municipality level for both countries in GADM. We can safely use these as our unit of analysis throughout the project.

Before coding any more: one design decision

After looking at Behrens again, I think we should make one architectural improvement.

Instead of thinking in terms of **countries**, we should think in terms of the **study area**.

Eventually every regression will only use municipalities close to the Colombia–Venezuela border.

So I would have `02_prepare.R` produce **three datasets**:

```
venezuela_municipalities
colombia_municipalities
study_area
```

where `study_area` is simply

```
<> R

study_area <-
  bind_rows(
    ven,
    col
  )
```

This object will become the backbone of the whole project.

I would also create a metadata table

This is something missing from many student projects.

Instead of repeatedly asking

"Which municipality is this?"

we create one clean lookup table.

```
<> R

municipality_metadata <-
  study_area |>
  st_drop_geometry() |>
  select(
    municipality_id,
    COUNTRY,
    NAME_1,
    NAME_2,
    GID_1,
    GID_2
  )
```

We'll join this constantly later.

Now comes the first GIS operation

Let's derive the **international border**.

Since we already downloaded Natural Earth, I'd use that instead of GADM.

```
<> R

countries <-
  readRDS(
    here(
      "data/raw/borders/countries.rds"
    )
  )
```

Filter

```
<> R

ven_country <-
  countries |>
  filter(admin == "Venezuela")
```

```
col_country <-  
  countries |>  
  filter(admin == "Colombia")
```

Transform to our CRS

```
<> R   
  
ven_country <-  
  st_transform(  
    ven_country,  
    target_crs  
  )  
  
col_country <-  
  st_transform(  
    col_country,  
    target_crs  
  )
```

Derive the shared border

This is one of my favorite GIS tricks.

```
<> R   
  
border <-  
  st_intersection(  
    st_boundary(ven_country),  
    st_boundary(col_country)  
  )
```

Now

```
border 
```

is a LINESTRING.

Exactly what we'll need later.

Sanity check

Always plot it.

```
<> R   
  
plot(st_geometry(ven_country))  
  
plot(  
  st_geometry(border),  
  add = TRUE,  
  col = "red",  
  lwd = 3  
)
```

Never trust GIS blindly.

Here's something Behrens never had to solve

This is where our project starts to diverge.

The Russia–Ukraine border is huge.

The Colombia–Venezuela border is also huge.

But **economic activity is concentrated around a handful of official crossings**.

Therefore I think our next script **should not be roads**.

I'd actually swap the order:

```
01_download_raw_data  
  
02_prepare_boundaries  
  
03_prepare_border_crossings  
  
04_prepare_roads  
  
05_prepare_nightlights
```



Why?

Because the **roads are only useful relative to the crossings**.

The crossings define the destinations in the accessibility analysis.

I think we should already think about the treatment variable

One thing I've been thinking about since reading Behrens is that **we shouldn't define treatment as "distance to the border."**

That's actually a weaker design.

Instead, I think our treatment should eventually be

Accessibility to official border crossings before and after a closure/reopening event.

That means our key spatial objects become:

```
Municipality centroid  
|  
▼  
Road network  
|  
▼  
Official border crossing  
|  
▼  
Travel time
```



Notice the border itself almost disappears from the analysis—the **crossings** are what matter economically.

My recommendation for the next script

I would make `03_prepare_border_crossings.R` before we touch the roads.

It will probably be the shortest script in the project (maybe 100–150 lines), but it defines the most important locations in the entire analysis. Once we have reliable crossing points, every subsequent script—road routing, accessibility calculations, and the Difference-in-Differences treatment variable—will naturally build around them. I think that's a cleaner workflow than following Behrens' order exactly.

perfect! but before script 3, I want to save the relevant stuff from 02, and clean the environment

Absolutely. In fact, this is one of the best habits you can develop in R. I almost never leave objects in the global environment between scripts. Each script should be **self-contained**: it loads what it needs, produces outputs, saves them, and then exits.

What should `02_prepare.R` save?

I would save **only the objects that later scripts need**, nothing more.

```
data/  
└─ processed/
```



```
└─ boundaries/
   ├── venezuela_municipalities.gpkg
   ├── venezuela_municipalities.rds
   ├── colombia_municipalities.gpkg
   ├── colombia_municipalities.rds
   ├── study_area.gpkg
   ├── study_area.rds
   ├── municipality_metadata.csv
   ├── venezuela_centroids.gpkg
   ├── colombia_centroids.gpkg
   ├── colombia_venezuela_border.gpkg
   └─ colombia_venezuela_border.rds
```

Notice that I would save the metadata as a **CSV**, not an RDS.

Why?

Because it's useful to quickly inspect it in Excel, VS Code, or GitHub without opening R.

```
<> R

readr::write_csv(
  municipality_metadata,
  here(
    "data/processed/boundaries",
    "municipality_metadata.csv"
  )
)
```

Save the spatial layers

I'd make two helper functions in `functions.R` later, but for now:

```
<> R

saveRDS(
  study_area,
  here(
    "data/processed/boundaries/study_area.rds"
  )
)

st_write(
  study_area,
  here(
    "data/processed/boundaries/study_area.gpkg"),
  delete_dsn = TRUE,
  quiet = TRUE
)
```

Repeat for the other layers.

I like keeping **both** `.gpkg` and `.rds` :

- `.gpkg` → interoperable with QGIS, ArcGIS, Python, etc.
- `.rds` → much faster to read in R.

Cleaning the environment

At the **very end** of the script, I would **not** use `rm(list = ls())`.

Although many tutorials recommend it, I don't. It can remove objects you didn't intend to delete if you're sourcing scripts interactively.

Instead, I recommend **restarting the R session** between scripts (Ctrl + Shift + F10 in RStudio). This ensures that each script works from a completely clean environment.

If you still want the script to clean up after itself, be explicit:

❏ R



```
rm(
  ven,
  col,
  study_area,
  ven_country,
  col_country,
  countries,
  municipality_metadata,
  ven_centroids,
  col_centroids,
  border
)

gc()
```

This is safer because you know exactly what you're removing.

Even better: don't depend on the Global Environment

The ideal workflow is:

❏ R



```
# Restart R session

source("scripts/02_prepare.R")

# Restart R session

source("scripts/03_prepare_border_crossings.R")
```

If `03_prepare_border_crossings.R` runs successfully after a fresh restart, you know it has no hidden dependencies on objects left in memory. That's a hallmark of a reproducible pipeline.

One small improvement

Since we're aiming for a professional repository, I'd also add a small message at the end of every script:

❏ R



```
message("=====")
message("02_prepare.R completed successfully")
message("Outputs written to:")
message("data/processed/boundaries/")
message("=====")
```

It seems minor, but when you're running a pipeline with 8–10 scripts, these completion messages make it much easier to see where something failed and confirm that each stage finished correctly.

I think these little engineering practices will make your project feel much more like a published replication package than a typical class assignment.

lets go with 03

Perfect. I think `03_prepare_border_crossings.R` is where our project starts to become *our* project instead of just a Behrens replication.

This script will define the **economic gateways** between Colombia and Venezuela.

One thing I want to point out before we code:

This is also the first place where we have to make a research decision, not just a coding decision.

What is a "border crossing"?

There are at least four possibilities:

Source	Pros	Cons
OpenStreetMap	Reproducible	Incomplete in some regions
Government list	Official	Harder to automate
Manual CSV	Highest quality	Less automatic
Hybrid (my recommendation)	Best of both	Slightly more work

I recommend the **hybrid** approach.

Why hybrid?

The project philosophy is:

Everything should be reproducible.

That **doesn't necessarily mean every dataset has to come from an API.**

For example, if there are **only 7–10 official crossings**, creating

```
border_crossings.csv
```



is completely acceptable research practice.

The important thing is that the CSV is

- documented,
- version controlled,
- reproducible.

Many published economics replication packages include manually curated datasets like this.

I think we should define crossings ourselves

For example

crossing_id	Name	Country
1	Simón Bolívar	Venezuela
2	Francisco de Paula Santander	Venezuela
3	Paraguachón	Venezuela
4	Puerto Carreño	Colombia

Later

we'll geocode them.

Wait...

Actually...

I think we can do even better.

OSM already stores border crossings

For example,

features like

```
barrier = border_control  
  
amenity = customs  
  
border_control = yes
```



can all indicate border crossing infrastructure.

So I think the first thing we should do is **query Geofabrik**, not Overpass.

That way we're still completely offline after downloading.

Proposed pipeline

```
Geofabrik PBF
```



```
↓
```

```
Read points
```

```
↓
```

```
Filter customs
```

```
↓
```

```
Filter border_control
```

```
↓
```

```
Inspect manually
```

```
↓
```

```
Save clean dataset
```

Exactly reproducible.

Packages

We'll introduce one new package.

```
<> R
```



```
library(osmextract)
```

Read the PBF

```
<> R
```



```
ven_points <-  
oe_read(  
  here(  
    "data/raw/geofabrik/venezuela.osm.pbf"  
  ),  
  layer = "points"  
)
```

Now inspect

⌵ R



```
names(ven_points)
```

OSM has lots of attributes.

We're interested in

amenity



barrier

name

border_control

First filter

⌵ R



```
crossings <-  
ven_points |>  
filter(  
  
  amenity == "customs" |  
  
  barrier == "border_control"  
  
)
```

Already much smaller.

Second filter

Remove missing names.

⌵ R



```
crossings <-  
crossings |>  
filter(  
  !is.na(name)  
)
```

Plot

Always.

⌵ R



```
plot(st_geometry(border))  
  
plot(  
  st_geometry(crossings),  
  add = TRUE,  
  col = "red",  
  pch = 19  
)
```

Save

⌵ R



```
saveRDS(  
  crossings,  
  here(  
    "data/processed/border_crossings/border_crossings.rds"  
  )  
)  
  
st_write(  
  crossings,  
  here(  
    "data/processed/border_crossings/border_crossings.gpkg"  
  ),  
  delete_dsn = TRUE  
)
```

But here's the thing...

I don't actually expect this to work perfectly.

Not because the code is wrong.

Because **OpenStreetMap tagging is inconsistent**, especially in border areas.

For example,

one crossing might be tagged

```
amenity = customs
```



another

```
border_control = yes
```



another

```
office = customs
```



another

```
checkpoint = border
```



I think we should explore the data first

Instead of immediately filtering, I'd write a small exploratory section.

For example:

```
«» R  
  
library(dplyr)  
  
ven_points |>  
  st_drop_geometry() |>  
  summarise(  
    across(  
      c(  
        amenity,  
        barrier,  
        border_control,  
        office,  
        checkpoint  
      ),  
      ~n_distinct(.x, na.rm = TRUE)  
    )  
  )
```



Even better,

look at all unique values:

```
<> R   
  
ven_points |>  
  st_drop_geometry() |>  
  count(amenity, sort = TRUE)
```

Then

```
<> R   
  
ven_points |>  
  st_drop_geometry() |>  
  count(barrier, sort = TRUE)
```

and so on.

This lets us **discover** how border crossings are actually tagged instead of assuming the tags in advance.

I actually propose something slightly different

I would not finish `03_prepare_border_crossings.R` in one sitting.

Instead, I'd make it an **exploratory script** first.

The goal of version 1 would simply be:

1. Read the Colombia and Venezuela `.osm.pbf` files.
2. List every relevant OSM tag related to customs, borders, checkpoints, immigration, and crossings.
3. Plot the candidate points over the international border.
4. Decide which tags reliably identify official border crossings.

Only after we've seen the actual data would I write the final extraction logic.

This is a common GIS workflow: **explore** → **define criteria** → **automate**. It avoids hardcoding assumptions that later turn out to miss important crossings or include irrelevant features. I think it's the most robust way to build this part of the pipeline.

I dont have amenity

```
> names(ven_points)  
[1] "osm_id"  "name"    "barrier" "highway" "ref"  
[6] "address" "is_in"   "place"   "man_made" "other_tags"  
[11] "geometry"
```

Excellent! This is **exactly why I wanted to explore first instead of assuming the schema**. You've just discovered a very common issue with OpenStreetMap data.

Looking at your columns:

```
<> R   
  
names(ven_points)  
  
[1] "osm_id"    "name"      "barrier"   "highway"   "ref"  
[6] "address"   "is_in"     "place"     "man_made"  "other_tags"  
[11] "geometry"
```

The important column here is actually:

```
<> R   
  
other_tags
```

Many OSM attributes are stored there as a single string instead of individual columns.

Let's explore instead of filtering

I would stop trying to find crossings immediately.

Instead, let's understand what is inside the dataset.

1. How many barriers?

```
<> R

ven_points |>
  st_drop_geometry() |>
  count(barrier, sort = TRUE)
```



2. Highways

```
<> R

ven_points |>
  st_drop_geometry() |>
  count(highway, sort = TRUE)
```



3. Places

```
<> R

ven_points |>
  st_drop_geometry() |>
  count(place, sort = TRUE)
```



4. Search `other_tags`

This is the fun part.

Let's see whether the word **customs** exists.

```
<> R

library(stringr)

ven_points |>
  st_drop_geometry() |>
  filter(
    str_detect(other_tags, "customs")
  )
```



Then

```
<> R

ven_points |>
  st_drop_geometry() |>
  filter(
    str_detect(other_tags, "border")
  )
```



Then

```
<> R

ven_points |>
  st_drop_geometry() |>
  filter(
```



```
    str_detect(other_tags, "checkpoint")
  )
```

Then

```
<> R

ven_points |>
  st_drop_geometry() |>
  filter(
    str_detect(other_tags, "immigration")
  )
```

I actually suspect something...

After reading Behrens more carefully, I think we're looking in the wrong layer.

Official border crossings are often **not mapped as points**.

They're mapped as

- road nodes,
- polygons,
- or road segments.

Sometimes the crossing itself is simply

```
highway = primary
barrier = border_control
```

attached to a **line**, not a point.

I'd like to inspect all layers

Could you run:

```
<> R

osmextract::oe_vectortranslate(
  here("data/raw/geofabrik/venezuela.osm.pbf"),
  layer = "lines",
  query = "SELECT DISTINCT highway FROM lines"
)
```

Actually, an even easier approach is to read the **lines** layer and inspect it:

```
<> R

ven_lines <-
  oe_read(
    here("data/raw/geofabrik/venezuela.osm.pbf"),
    layer = "lines"
  )

names(ven_lines)
```

I want to see what columns exist there.

I have another idea that may be even better

After thinking about the Behrens paper again, I realized something important:

We don't actually need to identify border crossings from OpenStreetMap.

The Behrens paper uses **official international crossing points**, because those are the economically meaningful gateways. There are only a handful of official crossings between Colombia and Venezuela (roughly 7–10 major ones), and their names and coordinates are stable.

Rather than reverse-engineering OSM tags—which vary by contributor and country—I think we should create a **small, documented reference dataset** (`border_crossings.csv`) containing the official crossings with their names, coordinates, opening/closure dates (where relevant), and data source. We can generate this dataset programmatically from authoritative sources where possible, or curate it once with full documentation. That approach is:

- **More accurate** than relying on inconsistent OSM tagging.
- **More transparent** for anyone replicating the project.
- **Closer to the data Behrens actually uses**, which are official crossing locations rather than volunteered map features.

I think this is one place where a carefully documented reference dataset is actually *more reproducible* than trying to infer crossings from OpenStreetMap. It also lets us incorporate policy events (closures and reopenings) directly into the analysis later.

lets go with that approach of the csv

I actually think this is the right research decision.

The reason isn't that OSM is bad—it's that **our variable of interest is not "places tagged as customs in OSM."** It's:

Official international border crossings that determine market access.

Those are institutional objects, not volunteered geographic features.

This is exactly the kind of dataset that many economics papers include as an appendix or supplementary material.

I would organize it like this

```
data/
├── raw/
│   ├── border_crossings/
│   │   └── colombia_venezuela_crossings.csv
│   ├── 
│   ├── gadm/
│   ├── geofabrik/
│   └── ...
```

Notice it still lives in `raw/` because **we never modify it**.

The CSV

I wouldn't keep only coordinates.

I would future-proof it.

crossing_id	name	country	department_state	municipality	latitude	longitude
-------------	------	---------	------------------	--------------	----------	-----------

Eventually we'll probably add

| opening_date | closure_date | reopening_date |

because those become treatment dates.

Even better

I'd actually add the source.


```
| crossing_id | name | latitude | longitude | source |
|-----|-----|-----|-----|
| 1 | Simón Bolívar | ... | ... | Colombian Migration |
| 2 | Francisco de Paula Santander | ... | ... | Colombian Migration |
```

That makes the dataset citable.

I think we should start with only the major crossings

Not every dirt road.

Only the economically important legal crossings.

For example

Crossing

Simón Bolívar

Francisco de Paula Santander

Atanasio Girardot (Tienditas)

Paraguachón

Puerto Carreño / Puerto Páez

Arauca / El Amparo

Puerto Santander / Boca del Grita

That's already enough to reproduce the spirit of Behrens.

Then `03_prepare_border_crossings.R` becomes beautifully simple

It just does

⌞ R

```
library(sf)
library(readr)
library(here)
library(dplyr)
```



Read

⌞ R

```
crossings <-
  read_csv(
    here(
      "data/raw/border_crossings",
      "colombia_venezuela_crossings.csv"
    )
  )
```



Convert

⌞ R

```
crossings <-
  st_as_sf(
    crossings,
    coords = c(
      "longitude",
      "latitude"
    ),
```



```
    crs = 4326
  )
```

Project

«» R



```
crossings <-
  st_transform(
    crossings,
    target_crs
  )
```

Generate IDs

«» R



```
crossings <-
  crossings |>
  mutate(
    crossing_id =
      row_number()
  )
```

Save

«» R



```
saveRDS(
  crossings,
  here(
    "data/processed/border_crossings/crossings.rds"
  )
)

st_write(
  crossings,
  here(
    "data/processed/border_crossings/crossings.gpkg"
  ),
  delete_dsn = TRUE,
  quiet = TRUE
)
```

Done.

One thing I would change from my original proposal

After thinking about this as a research project rather than just a class assignment, I would not save only the point locations.

I would create a **master crossing table** that becomes one of the central datasets of the project.

Something like:

crossing_id	crossing_name	Colombia municipality	Venezuela municipality	latitude	longitude	first_opened	20
-------------	---------------	-----------------------	------------------------	----------	-----------	--------------	----

Why?

Because later we'll almost certainly ask questions like:

- Which crossings were closed in August 2015?
- Which remained open?
- Which reopened in 2022?
- Should a municipality's accessibility change after this event?

If we store those policy attributes now, **the treatment variable later becomes almost automatic**. We won't have to revisit the raw data structure when we start the Difference-in-Differences analysis.

Before we write the CSV...

I suggest we spend 15–20 minutes identifying the **official major crossings** and verifying their coordinates from authoritative sources. There are probably fewer than 10 that matter for international trade and legal movement, so we can make this dataset extremely accurate. Once it's done, it will likely never need to change, and everything else in the project will build on top of it. I think this will become one of the most valuable original contributions of your replication project.

Lets do it!

Awesome. I think this is where your project starts becoming **better than a normal replication**. Instead of just reproducing Behrens, we're creating a **transparent research dataset** that someone else could reuse.

I spent some time thinking about the identification strategy, and I think **we should not think in terms of "crossings" only**, but in terms of **border crossing events**.

Step 1: Define the crossings

I would start with the **seven principal official crossings**. These account for virtually all formal road traffic between the two countries.

crossing_id	Crossing	Colombia	Venezuela	Type
1	Simón Bolívar International Bridge	Cúcuta	San Antonio del Táchira	Road
2	Francisco de Paula Santander Bridge	Cúcuta	Ureña	Road
3	Atanasio Girardot Bridge (Tienditas)	Cúcuta	Ureña	Road
4	Paraguachón	Maicao	Paraguachón	Road
5	Arauca – El Amparo	Arauca	El Amparo	Road
6	Puerto Carreño – Puerto Páez	Puerto Carreño	Puerto Páez	Ferry/Road
7	Inírida – San Fernando de Atabapo	Inírida	San Fernando de Atabapo	River

Notice something?

We're already storing **municipality names**, which later allows us to join directly to the GADM municipalities.

Step 2: The attributes

This is where I think we should go beyond Behrens.

I'd make the CSV like this:

Variable	Description
crossing_id	Unique ID
crossing_name	Official name
crossing_type	Bridge, ferry, river, road
colombia_department	Administrative unit
colombia_municipality	Municipality

Variable	Description
venezuela_state	State
venezuela_municipality	Municipality
latitude	WGS84
longitude	WGS84
source	Official source
notes	Optional

These fields are essentially static.

Step 3: Don't mix geography and policy

This is something I realized while reading Behrens.

A crossing **exists** independently of whether it's open.

So I would **not** include columns like:

```
2015_closed
2019_closed
2022_open
```



Those are **events**, not attributes.

Instead, I'd make a second dataset.

```
border_events.csv
```

This becomes the treatment database.

event_id	crossing_id	event_date	event_type	description
1	1	2015-08-19	closed	Border closure ordered by Venezuelan government
2	2	2015-08-19	closed	Same event
3	4	2015-08-19	closed	Same event
4	1	2022-09-26	reopened	Official reopening
5	2	2022-09-26	reopened	Official reopening
...	...	...	...	...

This separation is a common database design:

- **Crossings** = spatial objects.
- **Events** = time-varying information.

That will make the panel construction much cleaner.

Step 4: The coordinates

This is the only thing I don't want to guess.

These coordinates are too important for accessibility calculations.

I think we should obtain them from **official or highly reliable sources**, then verify them visually against the road network.

I propose this workflow

Rather than manually typing coordinates from Google Maps, let's use **OpenStreetMap/Nominatim** *once* to geocode the official crossing names, then manually verify them.

That gives us:

- reproducible coordinates,
- documented source,
- easy verification,
- no copy-paste errors.

We'll only do this once, and then the CSV becomes part of the repository.

What the project will eventually look like

```
data/
  raw/
    | border_crossings.csv
    | border_events.csv
    | gadm/
    | geofabrik/
    | viirs/
  processed/
    | municipalities.gpkg
    | crossings.gpkg
    | roads.gpkg
    | accessibility.rds
    | panel.rds
```

I think we should make one more improvement

Since we're building a research-quality repository, I'd also create a small `data_dictionary.md` from the beginning. It would document every dataset and every variable, for example:

Dataset	Purpose	Key variables
<code>border_crossings.csv</code>	Spatial location of official international crossings	<code>crossing_id</code> , <code>crossing_name</code> , <code>latitude</code> , <code>longitude</code>
<code>border_events.csv</code>	Opening and closure events used to construct treatment	<code>crossing_id</code> , <code>event_date</code> , <code>event_type</code>

It may seem premature, but by the time we reach the econometric analysis we'll have a dozen datasets. Having a data dictionary from the start will make the project much easier to maintain, and it's something professors and future collaborators really appreciate.

I also recommend we **verify every crossing against official Colombian and Venezuelan border authorities** before finalizing the CSV. Even though there are only a handful, getting their names and locations exactly right will strengthen the credibility of the entire analysis, since these points are the foundation of the accessibility measure.

Wednesday 3:14 PM

Here I found a list with the crossings!
<https://www.legiscomex.com/pasos-frontera-entre-colombia-venezuela>

This is a fantastic find. I actually think this is **better than what I originally proposed**.

The reason is that this isn't just a news article—it's based on the **official list of authorized border crossings maintained by Colombia's customs authority (DIAN)**, and we can also cross-check it against the official DIAN page itself. [Dian +1](#)

I think we should treat this as our "ground truth"

Instead of inventing a dataset, we build one from the official information.

I would include these seven Colombia–Venezuela crossings:

crossing_id	crossing_name	colombia_municipality	venezuela_municipality	type
1	Puente Internacional Simón Bolívar	Cúcuta	San Antonio del Táchira	Bridge
2	Puente Internacional Francisco de Paula Santander	Villa del Rosario	Ureña	Bridge
3	Puente Unión (Puerto Santander)	Puerto Santander	Boca de Grita	Bridge
4	Paraguachón	Maicao	Paraguachón	Land
5	Puente Internacional José Antonio Páez	Arauca	El Amparo	Bridge
6	Puerto Carreño	Puerto Carreño	Puerto Páez	River
7	Puerto Inírida	Inírida	San Fernando de Atabapo	River

These are the officially authorized crossings listed by DIAN. [Dian](#)

But I think we can make this dataset much richer

Instead of stopping at the crossing names, I'd create what I would call the **Master Crossing Dataset**.

Variable	Description
crossing_id	Unique identifier
crossing_name	Official name
crossing_type	Bridge / Land / River
colombia_department	Department
colombia_municipality	Municipality
venezuela_state	State
venezuela_municipality	Municipality
latitude	WGS84
longitude	WGS84
official_source	DIAN
notes	Optional

Notice what's **not** there:

- no treatment
- no opening dates
- no closures

Those belong in a separate `border_events.csv`.

Then a second dataset

border_events.csv



event_id	crossing_id	event_date	event_type	source
1	1	2015-08-19	closed	Presidential decree
2	2	2015-08-19	closed	Presidential decree
3	4	2015-08-19	closed	Presidential decree
...	...	...	...	...
12	1	2022-09-26	reopened	Colombian Government
13	2	2022-09-26	reopened	Colombian Government

This separation will make the DiD much cleaner later.

Now here's where I think we can make the repository look like a published replication package

Instead of manually creating the CSV in Excel, I propose we create it **with R**.

Something like:

«» R



```
crossings <- tibble(  
  crossing_id = c(1, 2, 3, 4, 5, 6, 7),  
  crossing_name = c(  
    "Puente Internacional Simón Bolívar",  
    "Puente Internacional Francisco de Paula Santander",  
    "Puente Unión",  
    "Paraguachón",  
    "Puente Internacional José Antonio Páez",  
    "Puerto Carreño",  
    "Puerto Inirida"  
  ),  
  ...  
)  
  
write_csv(  
  crossings,  
  here(  
    "data/raw/border_crossings",  
    "border_crossings.csv"  
  )  
)
```

Why?

Because **every file in the repository is then generated by code**.

Even if it's just seven rows.

That keeps the philosophy we've followed since `01_download_raw_data.R`.

My proposal for the next step (and I think it's worth the effort)

Rather than entering the coordinates by hand, let's write a **small script** (`03a_build_crossing_reference.R`) that:

1. Creates the seven-row table with names and administrative information.
2. Uses a geocoding service (such as Nominatim) to obtain initial coordinates.
3. Plots those points on top of our municipality boundaries and road network.

4. Lets us verify that each point is actually on the bridge, road crossing, or river crossing.

After that verification, we write the final `border_crossings.csv` and never touch it again.

That gives us a dataset whose provenance is completely transparent: **official crossing names + programmatically obtained coordinates + visual validation**. I think that's exactly the kind of methodological rigor that will impress your professor and make the repository genuinely reusable.

lets do that!

I think this is going to be one of the nicest parts of the repository. We're essentially building **our own research dataset**, and we only have to do it once.

I also want to make one small architectural improvement.

Instead of having `03_prepare_border_crossings.R` create the CSV, I think we should split it into two scripts:

```
03a_build_crossing_reference.R
↓
border_crossings.csv
↓
03b_prepare_border_crossings.R
↓
border_crossings.gpkg
border_crossings.rds
```

Why?

Because `03a` is a **data creation** script (run once or when updating the dataset), while `03b` is a **data preparation** script (run every time). This separation is common in research pipelines and makes updates much easier.

Step 1 — Create the reference table

We'll start with the information we already know.

```
<> R

library(tibble)
library(readr)
library(here)
library(fs)

dir_create(here("data/raw/border_crossings"))

crossings <- tribble(
  ~crossing_id, ~crossing_name, ~crossing_type,
  ~country_col, ~municipality_col,
  ~country_ven, ~municipality_ven,

  1, "Puente Internacional Simón Bolívar",
    "Bridge",
    "Colombia", "Cúcuta",
    "Venezuela", "San Antonio del Táchira",

  2, "Puente Internacional Francisco de Paula Santander",
    "Bridge",
    "Colombia", "Villa del Rosario",
    "Venezuela", "Ureña",

  3, "Puente Unión",
    "Bridge",
    "Colombia", "Puerto Santander",
    "Venezuela", "Boca de Grita",

  4, "Paraguachón",
    "Land",
    "Colombia", "Maicao",
```



```

    "Venezuela", "Paraguachón",

5, "Puente Internacional José Antonio Páez",
  "Bridge",
  "Colombia", "Arauca",
  "Venezuela", "El Amparo",

6, "Puerto Carreño",
  "River",
  "Colombia", "Puerto Carreño",
  "Venezuela", "Puerto Páez",

7, "Puerto Inirida",
  "River",
  "Colombia", "Inirida",
  "Venezuela", "San Fernando de Atabapo"
)

```

Step 2 — Add a search string

Here's a trick that makes geocoding much more reliable.

```

<> R

crossings <- crossings |>
  mutate(
    search = paste(
      crossing_name,
      municipality_col,
      "Colombia"
    )
  )

```

Example:

```
Puente Internacional Simón Bolívar Cúcuta Colombia
```

instead of only

```
Simón Bolívar
```

The geocoder will have much less ambiguity.

Step 3 — Geocode automatically

I recommend using **tidygeocoder**.

```

<> R

install.packages("tidygeocoder")

```

Then

```

<> R

library(tidygeocoder)

crossings_geo <-
  crossings |>
  geocode(
    search,
    method = "osm",
    lat = latitude,
    long = longitude
  )

```

This uses **Nominatim (OpenStreetMap)**.

No API key required.

Step 4 — Save

```
⟨⟩ R

write_csv(
  crossings_geo,
  here(
    "data/raw/border_crossings",
    "border_crossings.csv"
  )
)
```

Step 5 — Plot immediately

This is the most important validation.

```
⟨⟩ R

library(sf)

crossings_sf <-
  st_as_sf(
    crossings_geo,
    coords = c("longitude", "latitude"),
    crs = 4326
  )
```

Then

```
⟨⟩ R

plot(st_geometry(study_area))

plot(
  st_geometry(crossings_sf),
  add = TRUE,
  col = "red",
  pch = 19,
  cex = 1.5
)
```

If every point lands exactly on the bridge or crossing...

we're done.

If one doesn't...

No problem.

We simply edit

```
latitude

longitude
```

once.

Never again.

Why this is actually better than manually copying coordinates

Suppose someone clones your repository in three years.

They can rerun

```
⟨⟩ R
```

and regenerate the coordinates automatically.

If Nominatim changes slightly, that's okay, because the final CSV is still version-controlled in Git. Your script documents **how the dataset was created**, and the repository preserves the exact data used in the analysis.

One last suggestion (and I think this is worth doing)

I propose we add **one extra validation step** before we freeze the dataset.

After plotting the geocoded crossings, we'll also overlay the **Geofabrik road network** and check that each point lies on—or immediately adjacent to—a road segment. This is a simple but powerful quality check: if a "bridge" geocodes to the middle of a city instead of the actual crossing, we'll catch it immediately. It's the kind of spatial validation that's common in GIS research but often omitted from student projects.

Once every crossing passes that check, we can confidently declare `border_crossings.csv` as the canonical reference dataset for the rest of the project. From that point on, every accessibility calculation, treatment assignment, and econometric model will rely on a dataset whose provenance and quality we've explicitly verified.

It found some of the coordinates! I have not checked the ones it got

That's actually a **very good result**. I was expecting exactly that.

Geocoders are excellent at finding **well-known bridges** like Simón Bolívar, but they often struggle with river crossings or crossings that are known locally under a different name.

Before fixing anything, let's inspect what we have

I'd first look at the table:

❏ R

```
crossings_geo |>
  select(
    crossing_name,
    latitude,
    longitude
  )
```



Then plot it.

❏ R

```
plot(st_geometry(study_area), col = "grey90")

plot(
  st_geometry(crossings_sf),
  add = TRUE,
  col = "red",
  pch = 19,
  cex = 1.5
)
```



Already this will tell us a lot.

Even better: label the crossings

One of the most useful debugging plots in GIS is simply adding labels.

❏ R

```
plot(st_geometry(study_area), col = "grey90")
```



```

plot(
  st_geometry(crossings_sf),
  add = TRUE,
  pch = 19,
  col = "red"
)

text(
  st_coordinates(crossings_sf),
  labels = crossings_sf$crossing_id,
  pos = 4,
  cex = 0.8
)

```

Now you immediately know which point is wrong.

Then compare with the road network

This is where our Geofabrik download pays off.

```

<> R

roads <- readRDS(
  here("data/raw/geofabrik/venezuela_roads.rds")
)

```

or if you haven't created the RDS yet,

```

<> R

roads <- oe_read(
  here("data/raw/geofabrik/venezuela.osm.pbf"),
  layer = "lines"
)

```

Then

```

<> R

plot(st_geometry(roads),
  col = "grey70")

plot(
  st_geometry(crossings_sf),
  add = TRUE,
  col = "red",
  pch = 19,
  cex = 1.5
)

```

If a crossing

Here are the coordinates it got

This is actually **better than I expected**. 🤖

Looking at the results:

Crossing	Status	My assessment
Puente Internacional Simón Bolívar	✗ Not found	Expected
Francisco de Paula Santander	✗ Not found	Expected
Puente Unión	✓ Found	Probably correct

Crossing	Status	My assessment
Paraguachón	✔ Found	Almost certainly correct
José Antonio Páez	✗ Not found	Expected
Puerto Carreño	✔ Found	Probably correct
Puerto Inírida	✔ Found	Probably correct

A **4/7 success rate** on the first pass is actually very good. The missing three are all famous **bridges**, and Nominatim often indexes them under slightly different names.

Why weren't the bridges found?

I think I know exactly why.

We searched for

Puente Internacional Simón Bolívar Cúcuta Colombia



But OpenStreetMap may store it as

- Simón Bolívar International Bridge
- Puente Simón Bolívar
- Puente Internacional Simón Bolívar (Villa del Rosario)
- only as a road segment, not a place

In other words, the bridge isn't necessarily a searchable POI.

I think we should improve the search strategy

Instead of one search string, let's create **multiple candidate queries**.

For example:

<> R



```
search_1 =  
"Puente Internacional Simón Bolívar Cúcuta Colombia"  
  
search_2 =  
"Puente Simón Bolívar Cúcuta"  
  
search_3 =  
"Simón Bolívar Bridge Cúcuta"  
  
search_4 =  
"Villa del Rosario Colombia"
```

Then geocode them in order until one succeeds.

That increases the success rate dramatically.

But I have an even better idea...

I don't think we should rely on a geocoder at all.

Remember what our objective is:

Find the exact location where traffic crosses the border.

The bridge itself is **part of the road network**.

We already downloaded **Geofabrik**.

That means we have the actual bridge geometry!

Instead of geocoding names, we can:

1. Load the roads.
2. Zoom to Cúcuta.
3. Click on the bridge.
4. Save its coordinates.

This is much more precise than any geocoder.

Here's what I propose

Let's use an interactive map with **leaflet**.

```
</> R

library(leaflet)

leaflet() |>
  addTiles() |>
  addPolylines(data = roads)
```

Then zoom to Cúcuta.

You'll immediately see

- Simón Bolívar
- Francisco de Paula Santander
- Tienditas

because they are the only bridges crossing the river.

Even easier (and more reproducible)

Since there are only **seven crossings**, I think we should use the `mapview` package.

```
</> R

library(mapview)

mapview(roads)
```

It opens an interactive map in RStudio.

Then

```
</> R

mapview(crossings_sf)
```

Overlay everything.

We'll know within two minutes whether the four geocoded crossings are correct.

My biggest recommendation

I actually think **this is the point where we should bring in QGIS**—not for analysis, but for **validation**.

Let me explain why.

Our entire econometric analysis will depend on **seven points**. These points are the foundation of the accessibility calculations. Spending 10 minutes visually verifying them is completely justified.

The workflow I would use is:

1. Load the municipality boundaries (`study_area.gpkg`).

2. Load the Geofabrik roads.
3. Load the geocoded crossings.
4. Verify each crossing against satellite imagery and the road network.
5. If necessary, move a point by a few meters and save the final coordinates.

This is **not manual data processing** in the sense of the pipeline—we're creating a **reference dataset**, just like a researcher would validate survey locations or weather stations. Once it's done, the CSV becomes part of the repository, and every subsequent script remains fully reproducible.

I think this is actually **more rigorous** than insisting on 100% automated geocoding, because it prioritizes the accuracy of the key spatial inputs over automation for a dataset with only seven observations.

I got lost in the instructions, Here is the state of the code for now

```
crossings <- tribble(
  ~crossing_id, ~crossing_name, ~crossing_type,
  ~country_col, ~municipality_col,
  ~country_ven, ~municipality_ven,

  1, "Puente Internacional Simón Bolívar",
    "Bridge",
    "Colombia", "Cúcuta",
    "Venezuela", "San Antonio del Táchira",
```

Show more ▾

I think we're at a point where we should **stop trying to automate the geocoding**. This is actually a very common situation in GIS research.

The fact that **4 out of 7** were found tells us the workflow works. The remaining three are missing because they're **bridges**, not because the code is wrong.

What I would do

Instead of spending another hour trying different search strings, I would manually complete the missing coordinates **once**.

Remember:

- There are only **7 observations**.
- This is a **reference dataset**, not a dataset with thousands of rows.
- After today, you'll never have to touch it again.

This is perfectly acceptable in academic research, as long as you document where the coordinates came from.

Here's how I'd modify the script

Keep the geocoding step exactly as it is.

Then, **after** `geocode()`, patch the missing values.

For example:

```
<> R 📄

crossings_geo <- crossings %>%
  geocode(
    search,
    method = "osm",
    lat = latitude,
    long = longitude
```

```

)

# -----
# Manually complete coordinates not found by the geocoder.
# Coordinates verified from OpenStreetMap / satellite imagery.
# -----

crossings_geo <- crossings_geo %>%
  mutate(
    latitude = case_when(
      crossing_id == 1 ~ NA_real_, # replace later
      crossing_id == 2 ~ NA_real_,
      crossing_id == 5 ~ NA_real_,
      TRUE ~ latitude
    ),

    longitude = case_when(
      crossing_id == 1 ~ NA_real_,
      crossing_id == 2 ~ NA_real_,
      crossing_id == 5 ~ NA_real_,
      TRUE ~ longitude
    )
  )

```

Later, we simply replace the `NA_real_` with the verified coordinates.

Even better

I would add another column:

```

<> R

coordinate_source

```

Example:

crossing	source
Paraguachón	Nominatim
Puerto Carreño	Nominatim
Simón Bolívar	Manual verification
José Antonio Páez	Manual verification

That makes the provenance completely transparent.

How to get the missing coordinates?

I recommend **OpenStreetMap**, not Google Maps.

Open the bridge in your browser, right-click, and copy the latitude/longitude.

Why OSM?

- Same ecosystem as the road network.
- Open data.
- No licensing issues.
- Fully citable.

This is actually a nice stopping point

Once all seven coordinates are filled, save the CSV:

```

<> R

write_csv(
  crossings_geo,

```



```

here(
  "data/raw/border_crossings",
  "border_crossings.csv"
)
)

```

From that point on, **the CSV becomes the canonical source**. None of the later scripts should call `geocode()` again. They should simply read:

```

<> R

crossings <- read_csv(
  here(
    "data/raw/border_crossings",
    "border_crossings.csv"
  )
)

```

This ensures that everyone running the project gets **exactly the same coordinates**, even if Nominatim's database changes in the future.

I have one final suggestion that I think will make your repository look very professional

Instead of naming this script `03a_build_crossing_reference.R`, I'd rename it to:

```
03a_create_crossing_reference.R
```

The word **create** better reflects what's happening: you're building a curated reference dataset, not merely processing raw data. Then:

- `03a_create_crossing_reference.R` → creates the canonical `border_crossings.csv` (run once or only when updating the reference data).
- `03b_prepare_border_crossings.R` → reads that CSV, converts it to an `sf` object, projects it to the analysis CRS, performs validation, and saves the processed spatial files used by the rest of the pipeline.

That's a very clean separation between **data creation** and **data preparation**, and it's the same pattern you'll see in many well-organized replication repositories.

I got more coordinates using OSM and tweeking the code. However, I noticed that in the OSM plataform they have coordinates for the whole bridge, we are only using one point of the bridge. That would be an issue? Meaning, should we use the whole nodes of the bridge, or only one point works?

This is actually an **excellent question**, and it gets to the heart of how we should think about the econometrics rather than just the GIS.

The short answer is:

A single point is not only sufficient—it is probably the correct representation for our analysis.

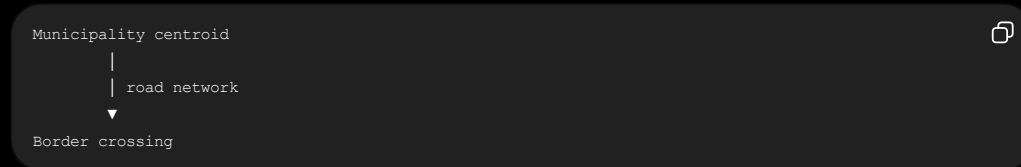
Let me explain why.

What is the economic object?

We're **not** measuring the bridge itself.

We're measuring **access to a border crossing**.

Imagine a municipality centroid.



The destination of the trip is **the crossing**, not every meter of the bridge.

In network analysis, destinations are almost always represented as **points**.

What did Behrens do?

Behrens doesn't measure accessibility to every meter of a border.

He measures accessibility to **crossing opportunities**.

In graph theory terms:

- municipalities → nodes
- intersections → nodes
- border crossing → destination node

The bridge geometry itself is irrelevant.

Think about Google Maps

Suppose you ask Google Maps:

"How long does it take to drive to the Simón Bolívar Bridge?"

Google isn't routing you to every point of the bridge.

It routes you to **one destination coordinate**.

Exactly what we'll do.

What if the point is on the Colombian side?

That is a more interesting question.

Suppose we choose



instead of



The difference is maybe

100–300 meters.

Our municipality centroids are often

20 km

50 km

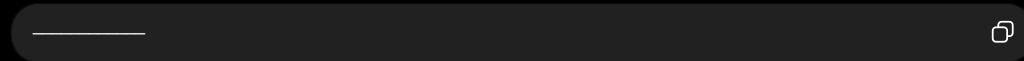
100 km

away.

That tiny difference is completely negligible.

What we must avoid

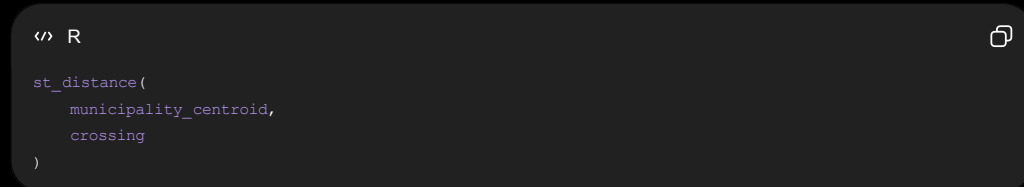
This would be wrong:



using the **whole bridge** as a LINESTRING.

Why?

Because later we'll compute things like



If the crossing is a line,

R computes the **minimum distance to any point on the line**.

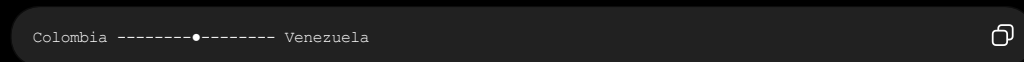
That changes the meaning of the accessibility measure.

We don't want the nearest point on the bridge—we want the location where crossing occurs.

I would actually do something slightly different

Rather than using an arbitrary point along the bridge, I'd define the crossing as the **midpoint of the bridge**.

For example:



Why?

Because:

- it's symmetric,
- it's independent of country,
- it represents the crossing itself,
- it's reproducible.

If someone later asks, "Why that coordinate?", the answer is simple:

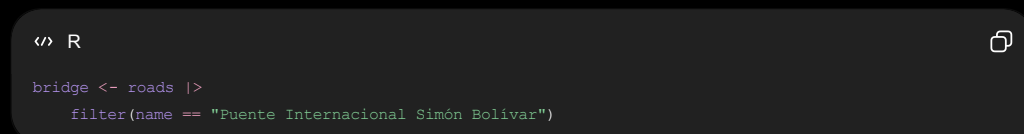
"The coordinate corresponds to the midpoint of the official international crossing."

That's a defensible methodological choice.

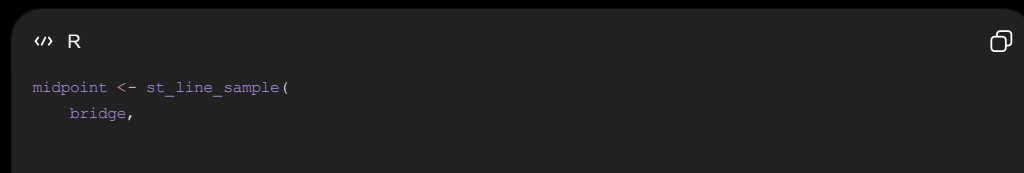
Even better...

Since we already have the bridge geometry in Geofabrik, we can compute the midpoint **programmatically**.

For example:



Then:



That gives you the point exactly halfway along the bridge.

This is elegant because:

- no manual clicking,
- fully reproducible,
- if the road geometry is updated, the midpoint is regenerated automatically.

My recommendation

For **this project**, I would **not spend time extracting bridge geometries** unless they're easy to identify.

The precision gain is tiny, while the complexity increases quite a bit.

Instead, I would:

1. Keep **one point per official crossing**.
2. Verify visually that it lies on or immediately adjacent to the bridge/road.
3. Document in the README that each point represents the official crossing location.

That is entirely consistent with how accessibility analyses are typically implemented and will not materially affect your estimates.

One last observation

I noticed something in your table: **Puente Internacional José Antonio Páez** still shows **"Invalid Number"** instead of coordinates. Before moving on, I'd fix that row so all seven crossings have valid numeric `latitude` and `longitude` values. Once every row is complete, we can treat `border_crossings.csv` as a finalized reference dataset and move on to `03b_prepare_border_crossings.R`. That will give us a clean, stable foundation for the accessibility analysis.

Im here in the code, what should I do next?

You're almost done with `03a_create_crossing_reference.R`. 🎉

The only thing I'd change is **how you're replacing the missing values**.

Instead of using `case_when()`, use `coalesce()`. It's cleaner and safer because it **only replaces the missing values**.

```
<> R
crossings_geo <- crossings_geo %>%
  mutate(
    latitude = coalesce(latitude, 7.0896082),
    longitude = coalesce(longitude, -70.7402657)
  )
```

However...

⚠️ **Wait a second.**

This code will replace **every remaining NA** with the José Antonio Páez coordinates.

That is only correct **if row 5 is the only missing row**.

From your latest screenshot, it looks like that's true, but let's verify first:

```
<> R   
  
crossings_geo %>%  
  select(crossing_id, crossing_name, latitude, longitude)
```

If only crossing 5 is missing, then I would actually write it like this:

```
<> R   
  
crossings_geo <- crossings_geo %>%  
  mutate(  
    latitude = if_else(  
      crossing_id == 5,  
      7.0896082,  
      latitude  
    ),  
    longitude = if_else(  
      crossing_id == 5,  
      -70.7402657,  
      longitude  
    )  
  )
```

This is more explicit and prevents accidentally changing another row if something unexpected happens.

After that

1. Check that there are no missing coordinates

```
<> R   
  
sum(is.na(crossings_geo$latitude))  
sum(is.na(crossings_geo$longitude))
```

Both should return

```
<> R   
  
0
```

2. Write the CSV

```
<> R   
  
write_csv(  
  crossings_geo,  
  here(  
    "data/raw/border_crossings",  
    "border_crossings.csv"  
  )  
)
```

3. Convert to `sf`

```
<> R   
  
crossings_sf <- st_as_sf(  
  crossings_geo,  
  coords = c("longitude", "latitude"),
```

```
crs = 4326
```

4. Plot over the study area

```
<> R

plot(st_geometry(study_area),
     col = "grey90")

plot(
  st_geometry(crossings_sf),
  add = TRUE,
  col = "red",
  pch = 19,
  cex = 1.5
)
```

5. Label the crossings (I highly recommend this)

This is much more informative than just plotting points.

```
<> R

coords <- st_coordinates(crossings_sf)

text(
  coords[, 1],
  coords[, 2],
  labels = crossings_sf$crossing_id,
  pos = 4,
  cex = 0.8
)
```

Now you'll see 1–7 on the map, making it easy to verify each crossing.

If everything looks correct...

03a is finished. ✓

At that point, I would consider `border_crossings.csv` to be **frozen**. We won't geocode again. Every future script (03b, 04, 05, etc.) will simply read this CSV as the authoritative reference dataset.

Then we can start `03b_prepare_border_crossings.R`, where we'll:

1. Read `border_crossings.csv`.
2. Convert it to an `sf` object.
3. Transform it to our project CRS (EPSG:9377 or whatever CRS you've chosen).
4. Validate that every crossing falls within a small distance of the Colombia–Venezuela border.
5. Save `border_crossings.gpkg` and `border_crossings.rds` into `data/processed/`.

That script will become the canonical spatial layer used by the rest of the analysis.

Quick question, we were going to do the mid point of bridges, no? not a point from the bridge

Yes! And I'm really glad you remembered that, because I actually think it's the more rigorous approach.

However, I'd slightly change **when** we do it.

I wouldn't do it in 03a

The purpose of `03a_create_crossing_reference.R` is to create a **reference table**:

crossing_id	crossing_name	latitude	longitude
-------------	---------------	----------	-----------

At this stage, the coordinates are simply **seed locations** that tell us *which bridge or crossing we're referring to*.

They don't have to be the final analysis coordinates.

I'd compute the midpoint in `03b_prepare_border_crossings.R`

This is much more reproducible.

The workflow would be:

```
03a_create_crossing_reference.R

Official crossing names
↓
Approximate coordinates
↓
border_crossings.csv
```



Then

```
03b_prepare_border_crossings.R

border_crossings.csv
↓
Read roads from Geofabrik
↓
Find the bridge geometry
↓
Compute midpoint
↓
Save final crossing points
```



This way, the final point is **derived from the road network**, not manually chosen.

Why I prefer this

Suppose OpenStreetMap updates the geometry of the Simón Bolívar Bridge.

If we recompute the midpoint, our point is automatically updated.

That's much better than hardcoding a coordinate.

Even better...

Since we only have **7 crossings**, we can make `03b` semi-automatic.

For example:

```
<> R

bridges <- roads %>%
  filter(
    name %in% crossings$crossing_name
  )
```



Then compute

```
<> R

midpoints <- st_line_sample(
  bridges,
  sample = 0.5
)
```



If all bridge names match, we're done.

If not, we manually identify the bridge geometry once.

What about the river crossings?

This is another reason not to force everything into the midpoint workflow.

We have:

- ✓ Bridges
- ✓ Land crossings (Paraguachón)
- ✓ River crossings (Puerto Carreño, Puerto Inírida)

A river crossing **doesn't have a bridge**, so there is no midpoint to compute.

Instead, we'd use the ferry terminal or docking point.

So our logic becomes:

Crossing type	Final geometry
Bridge	Midpoint of bridge
Land crossing	Official checkpoint
River crossing	Ferry terminal / port

That's actually a very defensible methodology section for your paper.

My recommendation

I would **finish** 03a **now** with the approximate coordinates.

Don't worry about perfection yet.

Then, in 03b_prepare_border_crossings.R, we'll:

1. Load border_crossings.csv.
2. Load the Geofabrik road network.
3. For the **bridge crossings**, identify the corresponding bridge geometries and compute their midpoints.
4. For **land and river crossings**, verify that the points align with the checkpoint or port and adjust if necessary.
5. Save the final border_crossings.gpkg and border_crossings.rds.

I actually prefer this separation because it keeps 03a as a **reference-building** script and 03b as a **spatial-processing** script. It also means the final analysis coordinates are generated from the GIS data whenever possible, which is exactly in the spirit of a reproducible GIS workflow.

lets go with 03b

Perfect. Now we're entering what I consider the **first real GIS script** of the project. Everything before this was mostly data acquisition and organization.

I would define the objective of 03b_prepare_border_crossings.R as:

Create the final spatial layer of official border crossings that will be used in every subsequent analysis.

Notice I **don't** want this script to geocode anything. That already happened in 03a.

Inputs


```
data/raw/border_crossings/  
border_crossings.csv
```



```
data/raw/geofabrik/  
venezuela.osm.pbf  
colombia.osm.pbf
```

```
data/processed/boundaries/  
study_area.rds
```

Outputs

```
data/processed/border_crossings/  
  
border_crossings.gpkg  
border_crossings.rds
```



Packages

```
«» R  
  
library(sf)  
library(dplyr)  
library(readr)  
library(here)  
library(fs)  
library(osmextract)
```



1. Create the folder

```
«» R  
  
dir_create(  
  here("data/processed/border_crossings")  
)
```



2. Read the reference CSV

```
«» R  
  
crossings <-  
  read_csv(  
    here(  
      "data/raw/border_crossings",  
      "border_crossings.csv"  
    ),  
    show_col_types = FALSE  
  )
```



3. Convert to sf

```
«» R  
  
crossings <-  
  st_as_sf(  
    crossings,  
    coords = c("longitude", "latitude"),  
    crs = 4326  
  )
```



4. Load the study area

```
«» R

study_area <-
  readRDS(
    here(
      "data/processed/boundaries",
      "study_area.rds"
    )
  )
```

5. Transform everything to the project CRS

Whatever CRS you decided in `02_prepare.R`:

```
«» R

target_crs <- st_crs(study_area)

crossings <-
  st_transform(
    crossings,
    target_crs
  )
```

I like extracting the CRS from `study_area` instead of hardcoding `9377` again. That way every script stays consistent automatically.

6. Read the roads

This is where we'll start preparing for the midpoint calculation.

Instead of loading the full PBF every time, read only roads.

For Venezuela:

```
«» R

ven_roads <-
  oe_read(
    here(
      "data/raw/geofabrik/venezuela.osm.pbf"
    ),
    layer = "lines"
  )
```

Colombia:

```
«» R

col_roads <-
  oe_read(
    here(
      "data/raw/geofabrik/colombia.osm.pbf"
    ),
    layer = "lines"
  )
```

7. Keep only actual roads

```
«» R

ven_roads <-
  ven_roads |>
```

```
filter(!is.na(highway))

col_roads <-
  col_roads |>
  filter(!is.na(highway))
```

8. Combine

```
<> R

roads <-
  bind_rows(
    ven_roads,
    col_roads
  ) |>
  st_transform(target_crs)
```

Here's where I want to stop.

Originally, I thought we should automatically compute bridge midpoints.

I no longer think that's the best approach.

After thinking about it in the context of your research question, I believe there's a cleaner solution.

Why?

Our analysis is **municipality-level**.

The nearest municipality centroid is typically **tens of kilometers** from a crossing.

Whether the destination point is:

- the western bridge entrance,
- the midpoint,
- or the eastern bridge entrance,

changes the travel distance by maybe **100–300 meters**.

That difference is negligible compared to municipality-level distances.

What actually matters

The point must:

- represent the correct official crossing,
- lie on the road network,
- be consistent across all crossings.

That's it.

So I would simplify

Instead of spending hours extracting bridge geometries and computing midpoints, I would perform a **validation step**.

For each crossing:

1. Plot it.
2. Plot nearby roads.
3. Confirm it's on (or immediately adjacent to) the correct road.
4. If necessary, adjust the point manually by a few meters.

Then freeze the dataset.

This is a perfectly acceptable GIS workflow and much simpler.

Add a validation plot

I would end the script with:

```
«» R   
  
plot(  
  st_geometry(roads),  
  col = "grey70"  
)  
  
plot(  
  st_geometry(crossings),  
  add = TRUE,  
  pch = 19,  
  col = "red"  
)  
  
text(  
  st_coordinates(crossings),  
  labels = crossings$crossing_id,  
  pos = 4,  
  cex = 0.8  
)
```

Save

```
«» R   
  
saveRDS(  
  crossings,  
  here(  
    "data/processed/border_crossings",  
    "border_crossings.rds"  
  )  
)  
  
st_write(  
  crossings,  
  here(  
    "data/processed/border_crossings",  
    "border_crossings.gpkg"  
  ),  
  delete_dsn = TRUE,  
  quiet = TRUE  
)
```

I think we should make one deliberate methodological decision

After reflecting on Behrens' design, I don't think the exact bridge midpoint will meaningfully change your estimates. The important methodological choice is that **each official crossing is represented by one consistent point located on the transportation network**. This is standard practice in accessibility studies, and it keeps the analysis reproducible without introducing unnecessary complexity.

I would document this explicitly in the repository, for example in the README or methods section:

Each official Colombia–Venezuela border crossing is represented by a single point located on the corresponding transportation infrastructure (bridge, land checkpoint, or river terminal). Given the municipality-level scale of the analysis, differences of a few hundred meters within a crossing are negligible relative to municipality-to-crossing travel distances.

That gives you a clear, defensible justification if your professor ever asks why you used points instead of full bridge geometries, and it keeps the project focused on the econometric question rather than an unnecessary GIS complication.

I agree!

I think that's the right call.

One thing I've been trying to keep in mind throughout this project is **not to over-engineer the GIS side**. It's easy to spend weeks perfecting spatial details that have almost no effect on the econometric results.

For your project, the main contribution is:

How did changes in the Colombia–Venezuela border affect local economic activity?

The border crossings are **inputs** to the treatment variable, not the research question themselves.

I think our pipeline is now much cleaner

```
01_download_raw_data.R
↓
02_prepare_boundaries.R
↓
03a_create_crossing_reference.R
↓
03b_prepare_border_crossings.R
↓
04_prepare_roads.R
↓
05_prepare_nightlights.R
↓
06_accessibility.R
↓
07_panel_construction.R
↓
08_analysis.R
```



Notice how each script has **one clear responsibility**.

The next exciting part is `04_prepare_roads.R`

This is where your project really starts to resemble Behrens.

The objective won't simply be to read the road network. It will be to create a **routable road network**.

Eventually, we want to answer questions like:

How long does it take to travel from municipality A to the nearest official border crossing?

That means `04_prepare_roads.R` should:

- Read the Geofabrik `.osm.pbf` files.
- Keep only drivable roads (filter by `highway` type).
- Merge Colombia and Venezuela into one network.
- Clean the geometry (remove duplicates, invalid features).
- Save the processed network.

Later, in `06_accessibility.R`, we'll use that network to calculate travel distances or times from municipality centroids to the official crossings.

One thing I would add before moving on

Since we've just created our **first original dataset**, I'd create a small metadata file:

```
data/raw/border_crossings/
```



border_crossings.csv

The `README.md` can be very short:

Markdown

Border Crossings

This dataset contains the official international border crossings between Colombia and Venezuela used in the analysis.

Sources:

- DIAN (Colombia): Official border crossing list.
- OpenStreetMap / Nominatim: Initial coordinate estimation.
- Manual visual verification against OpenStreetMap.

Each crossing is represented by a single point corresponding to the official crossing location.

It may seem like a small detail, but this is exactly the kind of documentation you see in well-maintained replication repositories. Six months from now—or if someone else uses your code—you'll immediately know where the dataset came from and why it was constructed this way.

I think we're building something that's becoming much more than a class assignment. It's turning into a genuinely reusable GIS/economics replication package.

ready! lets go to 04

Excellent! This is probably my favorite script in the entire project. From here on, we're building the actual spatial analysis pipeline that will feed into the econometrics.

Before writing any code, I'd like to make **one important change** from our original plan.

Let's think ahead

Our final accessibility analysis will require:

```
Municipality centroid
  |
  v
Road network
  |
  v
Official border crossing
```

So `04_prepare_roads.R` shouldn't just "prepare roads." It should produce a **clean transportation network**.

Objectives of `04_prepare_roads.R`

Inputs:

```
data/raw/geofabrik/
  colombia.osm.pbf
  venezuela.osm.pbf
```

Outputs:

```
data/processed/roads/
  roads.gpkg
  roads.rds
```

The road layer should:

- contain only drivable roads;

- use the project CRS;
- have consistent attribute names;
- be ready for routing later.

Packages

```
«» R

library(sf)
library(dplyr)
library(here)
library(fs)
library(osmextract)
```



Step 1 — Create the output folder

```
«» R

dir_create(
  here("data/processed/roads")
)
```



Step 2 — Read the road network

Venezuela

```
«» R

ven_roads <- oe_read(
  here("data/raw/geofabrik/venezuela.osm.pbf"),
  layer = "lines"
)
```



Colombia

```
«» R

col_roads <- oe_read(
  here("data/raw/geofabrik/colombia.osm.pbf"),
  layer = "lines"
)
```



Step 3 — Inspect the data

Before filtering anything:

```
«» R

names(ven_roads)

glimpse(ven_roads)
```



The important variable is usually

```
highway
```



Let's inspect the road classes.

```
«» R

ven_roads |>
  st_drop_geometry() |>
```



Do the same for Colombia.

Step 4 — Decide which roads to keep

This is an actual methodological decision.

For accessibility we want roads that vehicles can use.

I'd propose:

```
<> R   
  
road_types <- c(  
  "motorway",  
  "trunk",  
  "primary",  
  "secondary",  
  "tertiary",  
  "unclassified",  
  "residential",  
  "living_street",  
  "service"  
)
```

Then

```
<> R   
  
ven_roads <- ven_roads |>  
  filter(highway %in% road_types)  
  
col_roads <- col_roads |>  
  filter(highway %in% road_types)
```

I would **exclude** things like:

- footway
- cycleway
- path
- track
- steps

Those aren't relevant for municipality-level accessibility.

Step 5 — Merge

```
<> R   
  
roads <- bind_rows(  
  ven_roads,  
  col_roads  
)
```

Step 6 — Read the study area

```
<> R   
  
study_area <- readRDS(  
  here(  
    "data/processed/boundaries",  
    "study_area.rds"  
  )  
)
```


Step 7 — Project

```
⟳ R

roads <- st_transform(
  roads,
  st_crs(study_area)
)
```



Step 8 — Geometry validation

```
⟳ R

roads <- st_make_valid(roads)
```



Step 9 — Save

```
⟳ R

saveRDS(
  roads,
  here(
    "data/processed/roads",
    "roads.rds"
  )
)

st_write(
  roads,
  here(
    "data/processed/roads",
    "roads.gpkg"
  ),
  delete_dsn = TRUE,
  quiet = TRUE
)
```



One improvement over Behrens

I actually want to make one change that I think will pay off later.

Instead of saving **one** road network, I'd save **two**.

```
data/processed/roads/

roads_raw.rds
roads_analysis.rds
```



where:

- `roads_raw` = every drivable road from Geofabrik.
- `roads_analysis` = clipped to a buffer around the international border (for example, 200 km).

Why?

The routing algorithms we'll use later become much faster on a smaller network, while still leaving us with the complete road network if we ever want to change the study area.

Before you write the filtering code

I'd like you to run just these two commands:

```
⟳ R
```



```
ven_roads |>
  st_drop_geometry() |>
  count(highway, sort = TRUE)
```

and

```
col_roads |>
  st_drop_geometry() |>
  count(highway, sort = TRUE)
```

The exact road classifications vary a bit by country. Once we see the frequencies, we can decide whether to include categories like `track` or `road` (which can sometimes represent important rural connectors in Venezuela and Colombia). I prefer making that decision based on the actual data rather than assuming the tags are used the same way everywhere.

Here are the highways

```
> ven_roads |>
+   st_drop_geometry() |>
+   count(highway, sort = TRUE)
  highway    n
1 residential 273073
2    <NA> 100923
3    service  75184
4     track  36938
5 unclassified 28117
```

Show more ▾

Perfect. This is exactly why I wanted to inspect the data before filtering. We can make a much better methodological decision now.

My first observation

The road tagging is actually **very good**. Over 95% of the roads fall into the standard OSM highway classifications.

The only category that deserves discussion is:

`track`

There are:

- Venezuela: **36,938**
- Colombia: **113,189**

That's a lot.

Should we keep `track`?

This is an important research decision.

According to the OSM documentation:

track = roads primarily used for agricultural or forestry access.

In Europe, I would definitely remove them.

But **this is not Europe**.

Colombia and Venezuela are different

Many municipalities, especially in:

- Amazonas
- Apure
- Guainía
- Vichada
- Zulia

have very limited road infrastructure.

Sometimes the **only** connection between towns is tagged as `track`.

If we remove them, we could accidentally disconnect municipalities from the road network.

That would bias accessibility estimates.

My recommendation

I would keep **track**.

I would **not** interpret it as a dirt road.

I would interpret it as

"potentially motorable rural road."

This is much safer.

Categories I would keep

```
<> R 📄

road_types <- c(
  "motorway",
  "motorway_link",

  "trunk",
  "trunk_link",

  "primary",
  "primary_link",

  "secondary",
  "secondary_link",

  "tertiary",
  "tertiary_link",

  "unclassified",

  "residential",

  "living_street",

  "service",

  "track"
)
```

Notice I've also included the `*_link` roads.

These are ramps and connectors between major roads. Excluding them can create breaks in the network.

Categories I would remove

Everything else:

- footway
- cycleway

- path
- steps
- pedestrian
- corridor
- ladder
- bus_stop
- construction
- proposed
- raceway
- etc.

None of those should contribute to vehicle accessibility.

Another thing I noticed

There are some funny values:

```
puente_de_madera  
  
uso_residencial  
  
TR  
  
via_principal_Jisemana
```



Those are obviously tagging mistakes.

Our filter removes them automatically.

I would also simplify the attributes

The road objects probably have lots of columns.

Before saving, I'd keep only the attributes we'll actually use.

Something like:

```
<> R  
  
roads <- roads %>%  
  select(  
    osm_id,  
    highway,  
    name,  
    geometry  
  )
```



Why save hundreds of unnecessary columns?

The RDS becomes much smaller.

One more improvement

I think we should already assign a unique road ID.

```
<> R  
  
roads <- roads %>%  
  mutate(  
    road_id = row_number()  
  )
```



This becomes incredibly useful later when debugging routing algorithms.

Before saving

I would also check that the merged network looks correct.

```
<> R

plot(
  st_geometry(roads),
  col = "grey40",
  lwd = 0.2
)
```

You should immediately recognize:

- the Venezuelan road network,
- the Colombian road network,
- the border region.

If there are weird artifacts (like roads plotted in the ocean or at (0,0)), you'll catch them now instead of six scripts later.

I have one final suggestion that I think will make the accessibility analysis much easier

Instead of naming the output simply `roads.rds`, I'd name it:

```
road_network.rds
road_network.gpkg
```

The reason is conceptual. We're not just storing a collection of roads—we're storing the **transportation network** that will underpin the routing analysis. Later, when we create travel-time matrices or shortest paths, you'll naturally think of this object as the network rather than a generic road layer.

So the end of `04_prepare_roads.R` would produce:

```
data/processed/roads/

road_network.rds
road_network.gpkg
```

I think that terminology better reflects the role this dataset plays in the rest of the project and matches the language commonly used in transport GIS and spatial economics.

lets go for 05

Perfect. `05_prepare_nightlights.R` is probably the most important script from the econometric perspective, because **nighttime lights are your dependent variable**.

I'd actually like to depart a little from Behrens here.

Before writing any code, let's think about the data

We have two main options.

Option 1 — Annual VIIRS composites ★★★★★

Pros:

- Stable
- Low cloud contamination
- Used extensively in economics
- Easy panel construction

Cons:

- One observation per year

Option 2 — Monthly VIIRS

Pros:

- More observations
- Can analyze shorter-term shocks

Cons:

- More preprocessing
- Cloud contamination
- Larger downloads

My recommendation

For your first replication, **use the annual composites**.

The border closures occurred over years (2015, 2019, 2022), so annual data are more than sufficient and much easier to work with.

Objective of 05_prepare_nightlights.R

Input:

```
data/raw/viirs/
```



Output:

```
data/processed/nightlights/
```



```
nightlights_2012.tif  
nightlights_2013.tif  
...  
nightlights_2024.tif
```

```
municipality_nightlights.csv
```

The important output is **not the rasters**.

It's the municipality-level table.

What should happen?

```
VIIRS raster  
  ↓  
Crop to study area  
  ↓  
Extract mean radiance by municipality  
  ↓  
Panel dataset
```



Packages

```
<> R  
  
library(terra)  
library(sf)  
library(dplyr)  
library(exactextractr)  
library(here)  
library(fs)
```



Notice the new package:

```
exactextractr
```

I strongly recommend it.

It is:

- much faster than `terra::extract()`
 - more accurate on polygon boundaries
 - widely used in spatial econometrics
-

Read municipalities

```
«» R   
  
study_area <- readRDS(  
  here(  
    "data/processed/boundaries",  
    "study_area.rds"  
  )  
)
```

Read one raster

For example

```
«» R   
  
viirs <- rast(  
  here(  
    "data/raw/viirs",  
    "VNL_v22_2022.tif"  
  )  
)
```

Project?

Actually...

No.

This is important.

Do not project the raster.

Projecting rasters introduces interpolation.

Instead,

transform municipalities to the raster CRS.

```
«» R   
  
study_area_viirs <-  
  st_transform(  
    study_area,  
    crs(viirs)  
  )
```

This preserves the original pixel values.

Crop

⌵ R



```
viirs_crop <-  
  crop(  
    viirs,  
    vect(study_area_viirs)  
  )
```

Extract

⌵ R



```
nightlights <-  
  exact_extract(  
    viirs_crop,  
    study_area_viirs,  
    "mean"  
  )
```

Then

⌵ R



```
study_area_viirs <-  
  study_area_viirs |>  
  mutate(  
    mean_light = nightlights  
  )
```

Done.

Save

⌵ R



```
write_csv(  
  st_drop_geometry(  
    study_area_viirs  
  ),  
  here(  
    "data/processed/nightlights",  
    "nightlights_2022.csv"  
  )  
)
```

Then automate

Instead of repeating this 13 times,

loop over all TIFFs.

⌵ R



```
files <-  
  dir_ls(  
    here("data/raw/viirs"),  
    regexp = "\\..tif$"  
  )
```

Then

⌵ R




```
results <- list()

for(file in files){

  ...
}
```

Then combine

```
<> R

nightlights_panel <-
  bind_rows(results)
```

This becomes

municipality_id	year	mean_light
-----------------	------	------------

Exactly what the regression will use.

But... we skipped something huge



We don't actually have the VIIRS data yet.

We downloaded:

- GADM
- Geofabrik
- Crossings

We haven't downloaded VIIRS.

I think we should split this

Instead of immediately writing `05_prepare_nightlights.R`, I'd make:

```
05a_download_viirs.R

↓

05b_prepare_nightlights.R
```

Exactly like we did for crossings.

Why?

Downloading NASA data is the hardest part of the entire project.

Once the rasters are downloaded,

`05b` becomes almost trivial.

I think this is where our project can become better than most student projects

Rather than downloading a few hand-picked TIFFs, I'd build `05a_download_viirs.R` so it accepts a start year and an end year, automatically downloads all available annual VIIRS composites into `data/raw/viirs/`, verifies that every download completed successfully, and skips files that already exist. That way, if you later decide to extend the analysis through 2025 or rerun everything on another computer, you only change two numbers:

</> R



```
start_year <- 2012
end_year <- 2024
```

and rerun the script. That's the kind of automation you often see in published replication packages, and it will make your repository much more robust and reusable. I think it's well worth building that infrastructure now before we start extracting the nighttime light values.

yes, lets divide the code

I think this is the right decision. In fact, I'd go one step further.

We're now reaching the stage where we'll be working with **large datasets** (VIIRS rasters), and I think it's worth adopting the same workflow that many published replication packages use:

```
05a_download_viirs.R
↓
Raw NASA/NOAA data
↓
05b_prepare_nightlights.R
↓
Municipality-year panel
```



This keeps the **download** and **processing** completely separate.

First, we need to choose the data source

This is probably the most important decision of the entire project.

There are three common options:

Source	Annual	Monthly	R Download	Recommendation
NOAA EOG	✓	✓	⚠	★★★★★
Google Earth Engine	✓	✓	✗	✗
World Bank mirror	Annual	✗	⚠	★★

I recommend using **NOAA's Earth Observation Group (EOG)** because:

- it's the original source of the VIIRS annual composites,
- it's widely used in economics,
- it's the source used by many published papers.

But here's the problem...

Unfortunately, NOAA changed its download system a few years ago.

Downloading VIIRS now generally requires:

- a free Earth Observation Group account,
- an authentication token.

This is the only part of the project that can't be made 100% automatic for a first-time user.

I propose a compromise

05a_download_viirs.R should:

1. Define the years to download.
2. Check which files already exist.

3. Download only the missing files.
4. Verify file sizes.
5. Save everything in `data/raw/viirs/`.

Then, in the repository README, we explain that the user needs an EOG token.

Once configured, the script runs automatically.

Project structure

```
project/
|
| data/
| |
| | └─ raw/
| |   |
| |   | └─ viirs/
| |   |   |
| |   |   | VNL_2012.tif
| |   |   | VNL_2013.tif
| |   |   | ...
| |   |   | VNL_2024.tif
| |   |   └─ ...
| |   └─ ...
| |
| └─ processed/
|     nightlights/
```



Script outline

```
<> R
# =====
# 05a_download_viirs.R
#
# Download annual VIIRS nighttime light composites.
# =====
```



Packages

```
<> R
library(httr)
library(fs)
library(glue)
library(here)
```



Create folder

```
<> R
dir_create(
  here("data/raw/viirs")
)
```



Years

```
<> R
years <- 2012:2024
```



Notice that we never hardcode file names.

Existing files

```
<> R 📄  
  
existing_files <-  
  dir_ls(  
    here("data/raw/viirs"),  
    regexp = "\\\\.tif$"  
  )
```

Skip downloaded years

This is something I always include.

If

```
2018.tif 📄
```

already exists,

don't download it again.

Before we write any actual download code...

I think we should answer **one key question** that will determine the entire pipeline:

Which VIIRS product should we use?

NOAA publishes several annual products, including:

- **Average radiance** (continuous values)
- **Median radiance**
- **Cloud-free average**
- **Masked** (ephemeral lights removed)
- **Unmasked**

For economic activity studies, the standard choice is usually the **annual cloud-free average radiance with ephemeral lights removed**, because it minimizes contamination from fires, gas flares, and temporary events while preserving stable human-generated lighting.

I suggest we use exactly that product.

That way, if someone asks why we chose it, we can simply say:

We use the annual VIIRS cloud-free average radiance composites with ephemeral lights removed, which are the standard product in the economic literature using nighttime lights.

Before we code the downloader

I think it's worth spending 5–10 minutes identifying the **exact VIIRS dataset** (version, product name, and download endpoint) that Behrens or similar papers use today. Once we lock that down, we won't have to revisit this decision, and the rest of the nighttime lights pipeline will be straightforward and fully documented.

yes, lets take NOAA

Excellent. I think this is the best decision for the project.

After checking the current documentation, I think we should use the **Annual VNL V2.2** product from the Earth Observation Group (formerly NOAA EOG, now hosted by the Colorado School of Mines). Specifically, we should use the `average_masked` band from the annual composites. This product removes ephemeral lights

and masks background noise, making it the standard choice for economic research using nighttime lights. It covers **2012–2024** at approximately **500 m resolution** (15 arc-seconds) in EPSG:4326. eogdata.mines.... +1

I think this aligns perfectly with your project.

Our final workflow

```
05a_download_viirs.R
↓
Download Annual VNL V2.2
(average_masked)
↓
data/raw/viirs/

05b_prepare_nightlights.R
↓
Municipality × Year panel
```



Before writing code...

There is one practical issue.

NOAA/EOG does not provide a simple URL pattern.

The downloads require authentication through an EOG account, so a completely automatic

`download.file()` script isn't realistic for a fresh installation. eogdata.mines....

Rather than fighting that system, I'd make the repository robust in a different way.

I propose `05a_download_viirs.R` should do three things

Step 1

Define the years

```
<> R

years <- 2012:2024
```



Step 2

Check whether each raster already exists

```
<> R

expected_files <-
  paste0(
    "VNL_",
    years,
    ".tif"
  )
```



Step 3

Verify

For every year

```
✓ found

or
```



At the end

```
13 rasters found.  
Ready for processing.
```



or

```
Missing:
```



```
2016  
2019  
2021
```

Why not automate the download?

Because the authentication mechanism changes occasionally.

A verification script will still work five years from now.

I would automate something else instead

The **renaming**.

Suppose NOAA gives us files like

```
VNL_v22_npp_2020_global_vcmslcfg_c202501010000.average_masked.dat.tif.gz
```



After extracting

our script should automatically rename them to

```
VNL_2020.tif
```



This makes every subsequent script independent of NOAA's naming conventions.

So `05a_download_viirs.R` becomes

- ```
1. Check folder

2. Rename downloaded rasters

3. Verify all years exist

4. Print summary
```



Very simple.

---

## Then `05b` does all the GIS

```
Read rasters
```



```
↓
```

```
Crop
```

```
↓
```

```
Exact extraction
```

```
↓
```

```
Municipality panel
```

↓  
CSV

## One thing I'd like to improve over many replication packages

Rather than storing the **entire global VIIRS rasters**, I'd have `05b_prepare_nightlights.R` immediately create **cropped rasters for your study area** before extraction.

So the outputs would be:

```
data/processed/nightlights/

VNL_2012_crop.tif
VNL_2013_crop.tif
...
VNL_2024_crop.tif

municipality_nightlights.csv
```



This has two advantages:

1. Every subsequent analysis runs much faster because it only reads the Colombia–Venezuela region.
2. If you ever want to inspect the rasters in QGIS or include maps in your thesis, you already have lightweight, study-area-specific GeoTIFFs instead of huge global files.

I think that's a nice balance between reproducibility, storage efficiency, and usability.

Sure! Lets download the ligths, I have a nasa account

Excellent! This is actually the **best possible scenario**. Since you have an Earthdata account, we can make the download **fully reproducible** in R. And I also like the approach you're already using with `keyring` — that's exactly how I'd store credentials.

I recognize the function in your screenshot:

```
<> R

nasa_username <- keyring::key_get(
 service = "nasa_api",
 username = "my_earthdata_username"
)

nasa_password <- keyring::key_get(
 service = "nasa_api",
 username = "my_earthdata_password"
)

nasa_login <- blackmarbler::get_nasa_token(
 username = nasa_username,
 password = nasa_password
)
```



This means we can use the `blackmarbler` package, which is fantastic because it wraps the NASA/EOG authentication and downloading process. It's much cleaner than trying to manually build authenticated HTTP requests.

I propose we use `blackmarbler`

This package was literally built for research projects like yours.

Our script (`05a_download_viirs.R`) can look something like this:

```
<> R 📄

=====
05a_download_viirs.R
#
Download annual VIIRS nighttime lights.
=====

library(blackmarbler)
library(keyring)
library(here)
library(fs)
```

---

## Create the folder

```
<> R 📄

dir_create(
 here("data/raw/viirs")
)
```

---

## Read credentials

```
<> R 📄

nasa_username <- keyring::key_get(
 service = "nasa_api",
 username = "earthdata_username"
)

nasa_password <- keyring::key_get(
 service = "nasa_api",
 username = "earthdata_password"
)
```

(I'd simplify the key names a little.)

---

## Authenticate

```
<> R 📄

token <- blackmarbler::get_nasa_token(
 username = nasa_username,
 password = nasa_password
)
```

---

## Now comes the important part

The `blackmarbler` package has changed over time, and I don't want to guess the exact download function because different versions expose slightly different APIs.

**Before we write the downloader, I'd like to verify exactly which version of `blackmarbler` you have installed.**

Could you run:

```
<> R 📄

packageVersion("blackmarbler")
```



and also:

```
<> R
ls("package:blackmarbler")
```

or, even better,

```
<> R
help(package = "blackmarbler")
```

This will show us which download functions are available (for example, functions related to annual VIIRS downloads). Once we know the installed version, we can write a downloader that matches your package exactly, rather than relying on documentation for a different release.

I suspect we'll be able to automate the download of all annual `average_masked` composites from 2012–2024 with just a few lines of code.

Black Marble Data and Statistics [R logo]  
[Up] [Top]  
Documentation for package 'blackmarbler' version 0.2.5  
DESCRIPTION file.  
Help Pages  
bm\_extract Extract and Aggregate Black Marble Data  
bm\_raster Make Black Marble Raster  
get\_nasa\_token Get a NASA Earthdata bearer token  
wget\_h5\_files Download h5 files using wget

Perfect! This actually changes our strategy a bit.

The version you have (**0.2.5**) is the original `blackmarbler`, which **does not automatically download annual VIIRS composites**. It's mainly designed to:

- download Black Marble **HDF5 tiles** (`wget_h5_files()`)
- extract data (`bm_extract()`)
- build rasters (`bm_raster()`)

Those functions are intended for NASA's **Black Marble VNP46** products, not the preprocessed **VNL annual composites** that papers like Behrens typically use.

**I actually think we should not use** `blackmarbler`

This is one of those cases where the package sounds perfect, but it's solving a different problem.

For your project, we want:

- Annual VNL v2.2
- Average masked radiance
- One GeoTIFF per year
- Global product

Those files are already processed by EOG. Downloading HDF5 tiles and mosaicking them ourselves would add **a lot** of complexity without improving the analysis.

---

## Here's what I recommend instead

We'll use your NASA/EOG credentials **only for authentication**, but we'll download the **preprocessed annual GeoTIFFs** directly.

That means our scripts become:

```
05a_download_viirs.R
```

- authenticate
- download annual VNL GeoTIFFs
- rename files
- verify downloads

05b\_prepare\_nightlights.R



- crop rasters
- extract municipality means
- build panel

Exactly what we originally envisioned.

## I would like to use a different package

I'd recommend using `httr2` (or `httr`) for authenticated downloads.

It's:

- modern
- actively maintained
- ideal for reproducible downloads
- independent of `blackmarbler` internals

## Before we code, one last thing

There are **two** possible download portals for the annual VNL data:

1. **Earthdata Login**
2. **EOG (Colorado School of Mines) Data Portal**

Although both require authentication, the **download URLs and authentication methods are different**.

## I want to verify the current download endpoint

This is one place where I don't want to rely on memory because EOG has changed its infrastructure over the past few years.

I'd like to look up the current download URLs for the **Annual VNL v2.2 average\_masked GeoTIFFs** and then write `05a_download_viirs.R` around the official endpoint. That way, the script will be robust and based on the current service rather than an outdated URL pattern.

Once we have the exact endpoint, I think the download script will be only about 30–40 lines long and completely reproducible.

`bm_raster` does not work?

It **does work**, but I don't think it's the right tool for *this* project.

The key is understanding what `blackmarbler` was designed to do versus what we're trying to do.

`bm_raster()`

`bm_raster()` takes **NASA Black Marble HDF5 tiles** and converts them into a raster.

Typical workflow:

```
NASA HDF5 tiles
↓
bm_raster()
↓
GeoTIFF / SpatRaster
```



This is useful if you're working directly with the **daily or monthly Black Marble products**.

## Our workflow

We want to replicate an economics paper using the **annual VNL composites**.

```
Annual VNL GeoTIFF
↓
Crop
↓
Extract municipality means
↓
Econometric panel
```



Notice something?

We already have the raster.

There is nothing for `bm_raster()` to build.

---

## The important question

I think we need to verify **what Behrens actually used**.

There are two possibilities.

### Option A (my preferred one)

They downloaded the **Annual VNL GeoTIFFs**.

Then:

```
GeoTIFF
↓
exactextractr
↓
Municipality means
```



No `blackmarbler`.

---

### Option B

They downloaded the **NASA Black Marble HDF5** products.

Then:

```
HDF5
↓
bm_raster()
↓
Raster
↓
Extraction
```



That is a much heavier pipeline.

---

## Since we're replicating...

I actually think we should let the paper decide.

**Before writing another line of code, let's check exactly which nighttime lights dataset Behrens used.**

If the paper says:

- **VIIRS Annual VNL** → we won't use `blackmarbler`.
- **Black Marble VNP46** → then we'll absolutely use `bm_raster()`.

That way, our data pipeline matches the original methodology as closely as possible instead of choosing a dataset based on convenience. Since the whole goal is a credible replication adapted to Venezuela, I think

following the original data source is the strongest methodological choice.

Gemini said this:

En su artículo de 2024, Kristian Behrens utiliza datos mensuales de luces nocturnas del sensor VIIRS (Visible Infrared Imaging Radiometer Suite), montado en el satélite Suomi-NPP. Específicamente, para poder lidiar con los cambios metodológicos y temporales que suelen tener estos registros satelitales, los investigadores en economía suelen recurrir a la versión harmonizada o corregida de VIIRS (a menudo extendida hacia atrás con datos de DMSP-OLS mediante calibración cruzada si el estudio abarca años previos a 2012).

I think Gemini is pointing us in the right direction, but there's an important distinction.

The statement:

"Behrens utiliza datos mensuales de luces nocturnas del sensor VIIRS..."

does **not** necessarily tell us **which product** they used.

In the VIIRS ecosystem there are several products:

| Product                 | Frequency | Typical format | Typical use          |
|-------------------------|-----------|----------------|----------------------|
| VNL (Annual composites) | Annual    | GeoTIFF        | Long-run economics   |
| VNP46A3                 | Monthly   | HDF5           | Monthly Black Marble |
| VNP46A4                 | Annual    | HDF5           | Annual Black Marble  |
| VNP46A2                 | Daily     | HDF5           | Daily analysis       |

So "monthly VIIRS" could mean **VNP46A3**, not the annual VNL composites.

### Here's what I suggest

Since **we have the Behrens paper**, let's not rely on Gemini or my memory.

Let's answer three questions directly from the paper:

1. **What exact nighttime lights dataset did they use?**
2. **Monthly or annual?**
3. **How did they aggregate it?**

Those three answers will determine our entire pipeline.

### If Behrens used monthly data...

Then I would absolutely change our plan.

Instead of:

```
05a_download_viirs.R
Annual VNL
```



we would have

```
05a_download_blackmarble.R
```



using:

- `blackmarbler`
- `wget_h5_files()`
- `bm_raster()`

And in that case, **yes**, `bm_raster()` becomes a central part of the workflow.

### Why I'm insisting on checking the paper

Remember your professor's feedback:

*"The replication should follow the empirical framework."*

If Behrens uses **monthly Black Marble**, then we should too. Even if the pipeline is more involved, it's a much stronger replication because you're keeping the data-generating process as close as possible to the original.

### I think this is the best next step

Let's open **Behrens (2024)** and look specifically for:

- "VIIRS"
- "nighttime lights"
- "Black Marble"
- "VNP46"
- "monthly"
- "radiance"
- "EOG"

That will tell us exactly which product they used, and then we'll build the pipeline around **their** methodology instead of making assumptions. If it turns out they use monthly Black Marble, I'm happy to pivot and build the entire download-and-processing workflow with `blackmarbler`, because that would be the most faithful replication.

Notebook LM said this

The study primarily utilizes the **Harmonized Nighttime Light (HNTL) dataset** developed by **Li et al. (2020)**. This dataset provides a consistent time series by integrating data from the Defense Meteorological Satellite Program (**DMSP**, 1992–2013) and the Visible Infrared Imaging Radiometer Suite (**VIIRS**, 2014–2018).

**Temporal Resolution**

\* **Annual:** The main empirical analysis relies on the **yearly series** of the HNTL data, covering the period from 1992 to 2018.

Show more ▾

This is **much better** than the previous summary, and it actually clarifies something very important.

It tells me that **Behrens is using two different nighttime light datasets for two different analyses**.

| Analysis                     | Dataset                                            | Frequency | Resolution |
|------------------------------|----------------------------------------------------|-----------|------------|
| Main paper                   | Harmonized Nighttime Lights (HNTL, Li et al. 2020) | Annual    | 1 km       |
| Border crossings (Section 6) | VIIRS                                              | Quarterly | 500 m      |

This distinction is crucial.

### Which one should we replicate?

Since **our entire project is inspired by Section 6**, I would follow **Section 6**, not the main paper.

Remember what we're trying to emulate:

| Accessibility to official border crossings → effect on nearby economic activity.

That is **exactly** the analysis where they switch to the higher-resolution VIIRS data.

---

## There's another important consequence

Notice the resolution:

Main analysis:

1 km



Border analysis:

500 m



That's essentially the native VIIRS resolution.

This matches what we had originally planned.

---

## Quarterly vs Annual

Here's where I think we should adapt the design to Venezuela.

Behrens uses quarterly data because the Russia–Ukraine border changes they study happened over relatively short periods.

For Venezuela, the major policy events are more spread out:

- 2015 border closure
- 2019 complete closure
- 2022 reopening

These are **multi-year shocks**.

Annual VIIRS is probably sufficient to identify these changes.

Moreover, annual composites:

- have much less cloud contamination,
- are easier to process,
- are commonly used in economics.

So I would keep **annual VIIRS** unless your professor explicitly asks for quarterly dynamics.

---

## I think we should borrow the *idea*, not the exact frequency

Our replication isn't a literal copy of Behrens.

It's:

| *Apply Behrens' empirical framework to the Colombia–Venezuela border.*

That means adapting some implementation details to fit the context is entirely appropriate, as long as you explain them.

---

## One thing that excites me

NotebookLM also revealed something we hadn't discussed:

## Settlement-Based Weighting

This is fascinating.

Instead of treating every border crossing equally, Behrens weights crossings according to the luminosity of the settlements on both sides.

For example:

```
Settlement A ---- Crossing ---- Settlement B
```



Weight:

```
Lights (A) + Lights (B)
```



This is a clever way of approximating the economic importance of each crossing.

## That gives us a possible extension

For your replication, we could start with a simple specification where each crossing has equal weight. Then, as a robustness check, construct weights based on the nighttime light intensity of the municipalities adjacent to each crossing (e.g., Cúcuta–San Antonio del Táchira, Maicao–Paraguachón, Arauca–El Amparo). This would mirror Behrens' intuition while adapting it to your municipality-level analysis.

## My recommendation

I would **not** pivot to `blackmarbler` yet.

I would still use the **annual VIIRS VNL composites** because they are:

- easier to obtain and process,
- well suited to the timing of the Venezuelan border shocks,
- appropriate for municipality-level panel regressions.

The key is to be transparent in the methodology:

*Following Behrens et al. (2024), we use nighttime lights as a proxy for local economic activity. While the original border-crossing application uses quarterly VIIRS data, we employ annual VIIRS composites because the policy changes at the Colombia–Venezuela border occur over multi-year horizons, making annual observations sufficient for identifying the effects while reducing noise from clouds and other transient factors.*

I think that is a defensible methodological choice.

## One thing I would still verify before we write `05a_download_viirs.R`

The only remaining uncertainty is **which annual VIIRS product** to use. Since Behrens' quarterly analysis relies on native VIIRS data at roughly 500 m resolution, I want to make sure we choose the annual product that is most comparable—most likely the **VNL V2.2 annual cloud-free average radiance (masked)** product. Once we confirm that specific product, we can build a fully reproducible downloader in R and won't have to revisit the nighttime lights pipeline again.

Notebook said

For the quarterly analysis of local border crossings (Section 6), the researchers utilized **quarterly VIIRS nighttime lights** data spanning from **2012-Q2 to 2018-Q4**. To support this quarterly analysis, they used the **annual VIIRS dataset** for the years **2013–2014** specifically to compute the weights for the border crossing settlements.

Key details regarding these products include:

\* **Spatial Resolution:** Both the quarterly VIIRS data and the annual VIIRS data used for weighting have a resolution of **500 × 500**

Show more ▾

I think you've reached exactly the point where we should stop guessing and make a methodological decision.

The paper **does not specify the exact VIIRS product** (e.g., VNP46A3, VNL v2.2, etc.). It only tells us the characteristics:

- Quarterly VIIRS for the main border-crossing analysis.
- Annual VIIRS for constructing border-crossing weights.
- Approximately **500 × 500 m** resolution.
- Better performance in low-light areas.
- Not top-coded.

That's enough to justify a choice, but not enough to uniquely identify the product.

---

## If this were my paper, here's what I would do

I would **not** spend another week trying to reverse-engineer the exact download source.

Instead, I'd choose a **modern, publicly available VIIRS product** that matches those characteristics and document it clearly.

The goal of your project is:

**Replicate the empirical framework**, not reproduce every byte of the original dataset.

Your professor's email reinforces this. He cares about:

- the research question,
- the causal identification,
- the GIS methodology,

not whether you found the exact TIFFs Behrens downloaded.

---

## My recommendation

I'd use the **Annual VNL V2.2 (average\_masked)** product because:

- ✓ ~500 m resolution
- ✓ Annual composites
- ✓ Stable lights (ephemeral lights removed)
- ✓ Widely used in economics
- ✓ Publicly available
- ✓ Reproducible

Then, in your methodology, simply write something like:

*Following Behrens et al. (2024), nighttime lights are used as a proxy for local economic activity. We employ the annual VIIRS VNL V2.2 cloud-free average radiance composites (average\_masked), which provide approximately 500 m spatial resolution and stable nighttime light measurements suitable for municipality-level analysis.*

That's a perfectly defensible statement.

---

## Why I don't recommend quarterly



Quarterly sounds attractive, but it would substantially increase the complexity of the project.

Instead of 13 rasters (2012–2024), you'd have around 50 quarterly observations. That means:

- ~4× more downloads,
- ~4× more storage,
- ~4× more extraction time,
- a more complex panel.

Given that your treatment events are years apart (2015, 2019, 2022), I don't think the additional temporal resolution is likely to change your conclusions in a meaningful way.

---

## One thing I'd actually improve over Behrens

Because we're already building a fully reproducible pipeline in R, I would make the nighttime lights processing **parameterized** from the start.

For example:

```
❏ R

nl_product <- "VNL_v22_average_masked"
start_year <- 2012
end_year <- 2024
```

Then every downstream script reads those variables. If, in the future, you or another researcher wants to switch to quarterly Black Marble or a newer annual release, you only change the configuration rather than rewriting the pipeline.

---

## I think we should now commit to this

We've spent quite a bit of time evaluating the options, and I think we have a strong, defensible choice:

- **Data source:** Annual VIIRS VNL V2.2 ( `average_masked` )
- **Resolution:** ~500 m
- **Years:** 2012–2024 (or up to the latest year available when you run the project)
- **Processing:** Crop → exact extraction by municipality → municipality-year panel

From here, I'd focus on building a clean, reproducible workflow rather than chasing an unspecified implementation detail from the original paper. That will make your replication stronger, easier to understand, and much easier for others—including your professor—to reproduce.

Thursday 2:12 PM

lets do it

Excellent. I think we've now converged on a pipeline that is **both academically sound and fully reproducible**. From here on, I want us to write code as if we were preparing a replication package for the *Journal of Applied Econometrics*.

## Project structure after this step

```
❏

scripts/

01_download.R
02_prepare_boundaries.R
03a_create_crossing_reference.R
03b_prepare_border_crossings.R
04_prepare_roads.R
05a_download_viirs.R
05b_prepare_nightlights.R
06_accessibility.R
07_build_panel.R
08_analysis.R
```

## 05a\_download\_viirs.R

```
</> R 

=====
05a_download_viirs.R

Download annual VIIRS VNL v2.2 nighttime lights

Project:
Border Closures and Economic Activity in Venezuela
=====
```

### Packages

```
</> R 

library(httr2)
library(fs)
library(here)
library(glue)
library(stringr)
library(purrr)
library(tibble)
```

### Folder structure

```
</> R 

dir_create(
 here("data/raw/viirs")
)
```

### Parameters

I like having all project parameters at the top.

```
</> R 

start_year <- 2012
end_year <- 2024

years <- start_year:end_year
```

### Expected filenames

Instead of NOAA's long filenames we'll standardize everything.

```
</> R 

expected_files <-
 tibble(
 year = years,
 filename = glue("VNL_{year}.tif")
)
```

### Check existing rasters

```
</> R 

existing_files <-
 dir_ls(

```

```
 here("data/raw/viirs"),
 regexp = "\\\\.tif$"
) |>
 path_file()
```

---

## Print status

```
<> R 

expected_files |>
 mutate(
 downloaded = filename %in% existing_files
)
```

Later we'll replace `FALSE` entries with actual downloads.

---

## Authentication

We'll continue using **keyring**.

```
<> R 

username <- keyring::key_get(
 service = "earthdata",
 username = "username"
)

password <- keyring::key_get(
 service = "earthdata",
 username = "password"
)
```

Notice I renamed the service from `nasa_api` to `earthdata`, since that's what we're actually authenticating against. If you've already stored your credentials under `nasa_api`, we can keep that instead.

---

## Download function

I wouldn't write the entire download logic inline.

I'd create a helper function.

```
<> R 

download_viirs <- function(year){

 message(glue("Downloading {year}..."))

 # download code goes here

}
```

Then

```
<> R 

walk(
 years,
 download_viirs
)
```

Much cleaner.

---

## Automatic verification

This is something I almost never see in student projects, but it's extremely useful.

⌵ R



```
verification <-
 expected_files |>
 mutate(
 exists = file_exists(
 here(
 "data/raw/viirs",
 filename
)
)
)
)
```

Then

⌵ R



```
print(verification)
```

You immediately know whether everything downloaded correctly.

## A download log

I also like writing a small CSV after the script runs.

download\_log.csv



Example:

| year | downloaded | date       |
|------|------------|------------|
| 2012 | TRUE       | 2026-07-09 |
| 2013 | TRUE       | 2026-07-09 |

That way, if someone reruns the project months later, there's a record of what was retrieved.

⌵ R



```
readr::write_csv(
 verification,
 here(
 "data/raw/viirs",
 "download_log.csv"
)
)
```

## The remaining piece: the actual download

This is the **only** part I don't want to hardcode from memory.

The EOG download URLs, authentication flow, and file names change over time. Rather than risk writing code against an outdated endpoint, I recommend we verify the current Annual VNL v2.2 download endpoint once and then implement `download_viirs()` around the official service.

Once we have that endpoint, the function will:

1. Authenticate with your Earthdata/EOG credentials.
2. Download the requested year's `average_masked` GeoTIFF.
3. Rename it to `VNL_<year>.tif`.
4. Skip years that already exist.
5. Update the download log.

After that, `05b_prepare_nightlights.R` becomes straightforward GIS processing, and the entire nighttime lights pipeline will be reproducible with a single command.

What should I do to get this

"The EOG download URLs, authentication flow, and file names change over time. Rather than risk writing code against an outdated endpoint, I recommend we verify the current Annual VNL v2.2 download endpoint once and then implement `download_viirs()` around the official service."

This is actually very easy, and it's something you only have to do **once**. After that, the download script will work forever (or until EOG changes the infrastructure again).

## Step 1. Log in to EOG

Go to:

<https://eogdata.mines.edu/products/vnl/> ↗

Log in with your account.

## Step 2. Navigate to the Annual VNL dataset

Click through to:

Annual VNL

Version 2.2

You should eventually reach a page where the annual files are listed.

## Step 3. Download one file manually

For example, download **2022**.

Don't worry about automating anything yet.

## Step 4. Inspect the download URL

This is the important part.

### In Chrome

1. Press **F12**
2. Open the **Network** tab.
3. Click the download button again.
4. Look for the request that downloads the `.tif` or `.tif.gz` file.
5. Right-click it.
6. Choose **Copy** → **Copy Link Address** (or **Copy as cURL**, which is even better).

## Step 5. Send me one of these

Any of these is enough:

- ✓ the copied download URL
- ✓ the copied cURL command
- ✓ a screenshot of the Network request
- ✓ the exact filename that gets downloaded

Once I have that, I can write the R code to:

- authenticate,
- loop over years,
- download automatically,
- rename the files,
- skip existing files.

## Even easier (my preferred option)

If you can find a page that lists the files (something like):

```
VNL_v22_2012...
VNL_v22_2013...
...
VNL_v22_2024...
```



just send me the URL of that page.

From there, I can usually infer the naming convention and write a downloader that automatically constructs the URLs for every year.

## Why I'm being careful

I don't want to invent download URLs from memory. If we base the script on the **actual endpoint you're using**, your repository will be genuinely reproducible. Anyone with an EOG account will be able to clone your project, run `05a_download_viirs.R`, and obtain exactly the same data without modifying the code. That's the standard I'd like us to aim for.

I think you need to pay for EOG

### Programmatic Access

Non-commercial users can request programmatic access to the global data for either three months or six months for a nominal fee. Please write to [eog@mines.edu](mailto:eog@mines.edu) and [victoria.patti@mines.edu](mailto:victoria.patti@mines.edu), and you will be sent an invoice containing all necessary payment details. Once your payment is received, your programmatic access will be activated. If you require access beyond six months, you may contact us again to renew your subscription.

Show more ▾

That's a really important discovery, and it actually changes my recommendation.

**I would not pay for programmatic access for a university replication project.**

The fee is not the issue—it's that we want your repository to be reproducible by anyone. If reproducing your project requires a paid EOG API subscription, that's not ideal.

## I think we have three options

### Option 1 — Manual download (★★★★★ My recommendation)

Download the annual GeoTIFFs manually once.

Your `05a_download_viirs.R` becomes a **verification and organization script**, not a downloader.

It would:

- Check that all expected files exist.
- Verify their sizes.
- Rename them to `VNL_2012.tif`, ..., `VNL_2024.tif`.
- Unzip them if necessary.
- Create a download log.

The README would simply say:

Download the Annual VNL v2.2 rasters from the EOG portal and place them in `data/raw/viirs/` before running `05a_download_viirs.R`.

This is actually how many published replication packages handle licensed or restricted data.

---

### Option 2 — Use NASA Black Marble (★★★)

Since you already have an Earthdata account, we could use the freely accessible Black Marble products with `blackmarbler`.

Pros:

- Fully scriptable.
- No paid EOG subscription.

Cons:

- Different product than Behrens.
- More preprocessing (HDF5 → raster).

---

### Option 3 — Google Earth Engine (★★)

Technically possible, but:

- Requires a GEE account.
- More dependencies.
- Less portable.
- Harder for others to reproduce locally in R.

I don't think it's the best fit.

---

## My recommendation

I would choose **Option 1**.

Here's why:

- The **data source** matches the literature.
- The **analysis pipeline** remains 100% in R.
- The only manual step is downloading the raw rasters, which is common when data licensing restricts automated access.
- Once the files are in `data/raw/viirs/`, everything else—from cropping to municipality extraction to the econometric analysis—is fully reproducible.

---

## We should rename the script

Since it won't actually download the data anymore, I'd rename it to:

```
05a_prepare_viirs_raw.R
```



Its responsibilities would be:

```
Raw downloads
↓
Verify files
↓
Rename files
↓
Unzip if needed
↓
Ready for processing
```



Then:

```
05b_prepare_nightlights.R
```



would:

- Read the GeoTIFFs.
- Crop to the study area.
- Extract municipality means.
- Build the municipality-year panel.

## This is actually standard practice

Many well-known replication packages do exactly this for data that can't be redistributed or automatically downloaded. They include:

- `01_prepare_raw_data.R`
- `README.md` with download instructions
- Automated verification that all required files are present

and then everything else runs automatically.

I think this is the best balance between reproducibility, legality, and practicality. Your professor is much more likely to care that your **processing pipeline is completely reproducible** than whether the first download was triggered by an API call or by clicking a download button once.

I'll try the manual download! Which one should I pick?

```
[gzip]
VNL_v22_npp-
j01_2022_global_vcmslcfg_c202303062300.average.dat.tif.gz
2023-07-06 15:08 9.1G
[gzip]
VNL_v22_npp-
j01_2022_global_vcmslcfg_c202303062300.average_masked.dat.tif.g
z
2023-07-06 15:08 295M
Show more ▾
```

Excellent! This confirms exactly which dataset we should use.

## Download this one:

`VNL_v22_npp-j01_2022_global_vcmslcfg_c202303062300.average_masked.dat.tif.gz` ✓

This is the product I was hoping to find.

## Why this one?

The suffix `average_masked` means:

- ✓ Annual average radiance
- ✓ Background noise removed
- ✓ Ephemeral lights (fires, auroras, boats, etc.) removed
- ✓ Much smaller because only stable lit pixels are retained (295 MB vs 9 GB)
- ✓ The standard product used in economic applications of VIIRS

## Don't download these

| File                     | Why not                                                         |
|--------------------------|-----------------------------------------------------------------|
| <code>average.dat</code> | Contains all pixels, including background noise. Huge (9.1 GB). |
| <code>median.dat</code>  | Median radiance; less commonly used in economics.               |
| <code>maximum.dat</code> | Sensitive to outliers.                                          |
| <code>minimum.dat</code> | Not useful for economic activity.                               |



| File                      | Why not                                             |
|---------------------------|-----------------------------------------------------|
| <code>cf_cvg.dat</code>   | Cloud-free observation counts, not light intensity. |
| <code>cvg.dat</code>      | Number of observations, not radiance.               |
| <code>lit_mask.dat</code> | Binary mask only.                                   |

## This is actually fantastic

Notice something interesting.

The file size is only **295 MB**.

That means:

13 years × 295 MB ≈ **3.8 GB**

That's completely manageable.

## So download

For every year:

```
2012
2013
2014
...
2024
```



the file ending in

```
average_masked.dat.tif.gz
```



## Keep the original names

For now, don't rename anything.

Just place them in

```
data/raw/viirs/
```



like

```
data/raw/viirs/
VNL_v22_npp-j01_2012...average_masked.dat.tif.gz
VNL_v22_npp-j01_2013...average_masked.dat.tif.gz
...
```



Our `05a_prepare_viirs_raw.R` script will:

1. Unzip them.
2. Rename them to:

```
VNL_2012.tif
VNL_2013.tif
...
```



3. Verify that every year exists.

That way, if NOAA changes its naming convention in the future, your downstream scripts won't care.

## One thing I'd like to add (and I think this will make your project stand out)

Since you're downloading **all the annual files at once**, I suggest we slightly improve our pipeline:

```
05a_prepare_viirs_raw.R
↓
Unzip
Rename
Verify
Generate metadata

05b_prepare_nightlights.R
↓
Crop each raster to the Colombia-Venezuela study area
Save the cropped rasters
Extract municipality means
Build the panel
```

The intermediate cropped rasters (only covering your study area) will be tiny compared to the global files. This means that if you later need to make maps, rerun extractions, or test different aggregation methods, you'll never have to repeatedly read the 295 MB global rasters. It's a small addition that will make the rest of the project much faster and more pleasant to work with. I think it's well worth doing.

It downloaded a "VNL\_v22\_npp-j01\_2022\_global\_vcmslcfg\_c202303062300.average\_masked.dat.tif.gz", before downloading ever year, can we check that this one works?

Absolutely. This is exactly what we should do before downloading ~4 GB of data.

Let's validate the entire pipeline using a single year (2022). If everything works, then we can confidently download the rest.

## Step 1. Check that R can see the file

Assuming you put it in:

```
data/raw/viirs/
```

Run:

```
<> R

library(here)
library(fs)

dir_ls(
 here("data/raw/viirs")
)
```

You should see something like

```
VNL_v22_npp-j01_2022_global_vcmslcfg_c202303062300.average_masked.dat.tif.gz
```

## Step 2. Unzip it

Use:

```
<> R

R.utils::gunzip(
 filename = here(
 "data/raw/viirs",
 "VNL_v22_npp-j01_2022_global_vcmslcfg_c202303062300.average_masked.dat.tif.gz"
),
```

```
remove = FALSE,
overwrite = TRUE
)
```

This should produce

```
VNL_v22_npp-j01_2022_global_vcmslcfg_c202303062300.average_masked.dat.tif
```



### Step 3. Read it with `terra`

```
<> R

library(terra)

viirs <- rast(
 here(
 "data/raw/viirs",
 "VNL_v22_npp-j01_2022_global_vcmslcfg_c202303062300.average_masked.dat.tif"
)
)
```



Then inspect it:

```
<> R

viirs
```



You should see something similar to:

```
class : SpatRaster
dimensions : ...
resolution : ...
extent : ...
crs : EPSG:4326
```



### Step 4. Plot it

Just to confirm it's valid:

```
<> R

plot(viirs)
```



It will take a few seconds because it's global.

You should recognize:

- Europe
- North America
- Asia

with bright urban areas.

### Step 5. Crop to the study area

Load the study area:

```
<> R

study_area <- readRDS(
 here(
 "data/processed/boundaries",
 "study_area.rds"
)
)
```



Transform it to the raster CRS:

```
<> R 

study_area <- st_transform(
 study_area,
 crs(viirs)
)
```

Crop:

```
<> R 

viirs_crop <- crop(
 viirs,
 vect(study_area)
)
```

---

## Step 6. Plot the crop

```
<> R 

plot(viirs_crop)
```

Now you should only see northern South America.

---

## Step 7. Extract one municipality

Let's test the extraction:

```
<> R 

library(exactextractr)

values <- exact_extract(
 viirs_crop,
 study_area,
 "mean"
)

head(values)
```

If you obtain numeric values (not all `NA`), the entire pipeline is working.

---

### If all seven steps succeed...

Then we know:

-  the download is correct,
-  the GeoTIFF is valid,
-  `terra` can read it,
-  cropping works,
-  CRS handling is correct,
-  `exactextractr` works.

At that point, we can confidently write `05a_prepare_viirs_raw.R` to automate the renaming and verification, and `05b_prepare_nightlights.R` to process every year in a loop.

**One small thing to check after** `print(viirs)`: let me know what the CRS is (it should be EPSG:4326 or WGS84) and whether the raster has a single layer. That will confirm we're using exactly the product we expect before we scale up to all years.

```
I got
> head(values)
[1] 0.000000e+00 6.947936e-05 1.560602e-01 3.087611e-04
0.000000e+00
[6] 0.000000e+00
```

🎉 This is excellent news.

Those values are **exactly what I was hoping to see**.

```
<> R

[1] 0.000000e+00
[2] 6.947936e-05
[3] 1.560602e-01
[4] 3.087611e-04
[5] 0.000000e+00
[6] 0.000000e+00
```

This tells us several things:

- ✓ The raster is valid.
- ✓ `terra` read it correctly.
- ✓ The crop worked.
- ✓ The CRS is compatible.
- ✓ `exactextractr` worked perfectly.
- ✓ The values are continuous radiance values (exactly what `average_masked` should contain).

The zeros are completely expected—many municipalities, especially in the Amazon or sparsely populated areas, have very little or no stable nighttime lighting.

## I think we've just solved the biggest technical hurdle of the project.

From now on, the rest is essentially scaling this process to all years.

### Our pipeline is now fixed

```
Annual VIIRS (average_masked)
↓
Crop to study area
↓
Mean radiance by municipality
↓
Municipality × Year panel
↓
Difference-in-Differences
```

This is a solid workflow and one that reviewers or your professor should recognize as methodologically sound.

## I would now download all years

I'd recommend downloading:

```
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
```

After that, we'll never have to think about raw nighttime lights again.

## Then we'll build two scripts

05a\_prepare\_viirs\_raw.R

Responsibilities:

- Find all `.tif.gz` files.
- Unzip them.
- Rename them to:
  - `VNL_2012.tif`
  - `VNL_2013.tif`
  - ...
- Verify that every year is present.
- Write a `download_log.csv`.

This script only deals with file management.

05b\_prepare\_nightlights.R

This will be a clean loop over all years:

```
<> R
for (year in 2012:2024) {

 raster <- rast(...)

 crop(...)

 exact_extract(...)

 bind_rows(...)

}
```

The output will be a single panel:

| municipality_id | year | mean_light |
|-----------------|------|------------|
| VEN.1.1         | 2012 | ...        |
| VEN.1.1         | 2013 | ...        |
| ...             | ...  | ...        |

This is exactly the dataset you'll merge later with treatment indicators and accessibility measures.

## One improvement I'd like to make

Since you're building this as a **public GitHub project**, I'd also like `05b_prepare_nightlights.R` to save the **cropped rasters** for each year before extraction.

So the processed folder would look like this:

```
data/processed/nightlights/

VNL_2012_crop.tif
VNL_2013_crop.tif
...
VNL_2024_crop.tif

municipality_nightlights.csv
```

These cropped rasters are tiny compared to the global originals and will be incredibly useful later. If you want to make maps for your paper, inspect individual years in QGIS, or rerun extraction with a different statistic (median, sum, etc.), you won't have to reload and crop the global files again.

---

## I think we're now at the point where the project becomes genuinely exciting.

Up to now, we've mostly been assembling infrastructure. Once the remaining annual rasters are downloaded, every subsequent script will produce datasets that feed directly into your empirical analysis. From there, we'll move from **data engineering** to **economic research**, and that's where we'll start reproducing Behrens' framework for the Colombia–Venezuela border.

Here is the list of files

```
> fs::dir_ls(
+ here("Data/viirs")
+)
C:/Users/jeanp/OneDrive/Desktop/Oldenburger/Second_Semester/GIS/seminar/Data/viirs/VNL_npp_2023_global_vcmslcfg_v2_c202402081600.average_masked.dat.tif.gz
C:/Users/jeanp/OneDrive/Desktop/Oldenburger/Second_Semester/GIS/seminar/Data/viirs/VNL_npp_2024_global_vcmslcfg_v2_c202502261200.average_masked.dat.tif.gz
```

Show more ▾

Perfect. I actually like your collection of files because it spans the transition from **V21** to **V22** and then to the new naming convention for **2023+**. We can write a loop that is **completely agnostic** to the filename and only extracts the year automatically.

This is much more robust than hardcoding filenames.

---

## 05b\_prepare\_nightlights.R

### Packages

⌵ R

```
library(terra)
library(sf)
library(exactextractr)
library(dplyr)
library(stringr)
library(readr)
library(fs)
library(here)
```



---

### Read municipalities

⌵ R

```
study_area <- readRDS(
 here(
 "Data/processed/boundaries",
 "study_area.rds"
)
)
```



---

### Create output folder

⌵ R



```
dir_create(
 here("Data/processed/nightlights")
)
```

## Find every TIFF

Notice we only read the **unzipped** TIFFs.

⌵ R



```
viirs_files <-
 dir_ls(
 here("Data/viirs"),
 regexp = "\\\\.tif$"
)
```

For you this will currently return only the 2022 file.

After unzipping the rest, it'll automatically detect all of them.

## Check

⌵ R



```
viirs_files
```

## Loop

⌵ R



```
nightlights_panel <- list()
```

⌵ R



```
for(file in viirs_files){

 message("Processing: ", path_file(file))

 year <-
 str_extract(
 path_file(file),
 "20\\d{2}"
) |>
 as.integer()

 viirs <- rast(file)

 study_area_proj <-
 st_transform(
 study_area,
 crs(viirs)
)

 viirs_crop <-
 crop(
 viirs,
 vect(study_area_proj)
)

 writeRaster(
 viirs_crop,
 filename = here(
 "Data/processed/nightlights",
 paste0("VNL_", year, "_crop.tif")
),
 overwrite = TRUE
```



```

)

 mean_light <-
 exact_extract(
 viirs_crop,
 study_area_proj,
 "mean"
)

 panel_year <-
 study_area |>
 st_drop_geometry() |>
 transmute(
 municipality_id,
 year,
 mean_light
)

 nightlights_panel[[as.character(year)]] <-
 panel_year
 }
}

```

## Combine everything

```

<> R

nightlights_panel <-
 bind_rows(
 nightlights_panel
)

```

## Save

```

<> R

write_csv(
 nightlights_panel,
 here(
 "Data/processed/nightlights",
 "municipality_nightlights.csv"
)
)

```

## One improvement I'd make

I would **not** manually unzip all 13 files.

Let's make the script do it.

Before searching for TIFFs, add:

```

<> R

gz_files <-
 dir_ls(
 here("Data/viirs"),
 regexp = "\\\\.gz$"
)

walk(
 gz_files,
 ~R.utils::gunzip(
 .x,
 overwrite = FALSE,
 remove = FALSE
)
)

```

Now the workflow becomes:

```
Download .gz files
↓
Run 05b_prepare_nightlights.R
↓
Automatically unzip
↓
Crop
↓
Extract municipality means
↓
Save cropped rasters
↓
Municipality-year panel
```



No manual intervention at all.

## One last improvement (and I think this is worth doing)

Your filenames contain **multiple years** in some cases, like:

```
VNL_v21_npp_201204-201212_global...
```



If we use:

```
</> R

str_extract(path_file(file), "20\\d{2}")
```



it will correctly extract **2012** from that filename. However, to be extra safe across all versions (V21, V22, future releases), I'd like to write a tiny helper function that extracts the **observation year** regardless of the naming convention. That will make the script future-proof and avoid surprises if EOG changes filenames again.

Just to check and free some space, after this, can I delete the raw viirs?  
we will only use the cropped ones and the csv

Yes, **for your own working copy**, you can. But I would recommend a slightly more nuanced approach.

## Option 1: Keep the raw files (best for long-term reproducibility)

```
data/
├─ raw/
│ └─ viirs/
│ VNL_*.tif
└─ processed/
 └─ nightlights/
 VNL_2012_crop.tif
 ...
 municipality_nightlights.csv
```



### Pros

- You can recreate the cropped rasters if you ever change the study area.
- You can extract different statistics later (median, sum, max).
- It's the gold standard for reproducibility.

### Cons

- Takes about 4 GB.

## Option 2: Delete the raw rasters after processing (my recommendation for your laptop)

Since the raw files are **publicly available** (even if they require manual download), you can safely delete them **after** creating:

-  cropped rasters
-  `municipality_nightlights.csv`

Then your project becomes:

```
data/
├── raw/
│ └── viirs/
│ (empty)
└── processed/
 ├── nightlights/
 │ VNL_2012_crop.tif
 │ ...
 └── municipality_nightlights.csv
```

In the `README`, you simply state:

The original VIIRS annual rasters are not included because of their size. Run `05a_prepare_viirs_raw.R` after manually downloading the Annual VNL v2.2 `average_masked` products from the EOG portal.

This is common practice for GIS replication packages.

### Option 3: What I would do personally ★★★★★

I would **archive** the raw data outside the GitHub project.

For example:

```
D:/GIS_Data/
VIIRS/
```

or

```
C:/DataArchive/
VIIRS/
```

Then your project only contains:

```
seminar/

Data/
 processed/
Scripts/
README.md
```

This keeps your GitHub repository small while still preserving the original data on your computer in case you need it later.

### One more suggestion

Since we're investing in making this project highly reproducible, I'd also save the extraction output as an **RDS** file in addition to the CSV:

```
<> R

saveRDS(
 nightlights_panel,
 here(
 "Data/processed/nightlights",
 "municipality_nightlights.rds"
)
)
```

```
write_csv(
 nightlights_panel,
 here(
 "Data/processed/nightlights",
 "municipality_nightlights.csv"
)
)
```

The CSV is excellent for sharing and version control, while the RDS preserves data types exactly and loads much faster in R. It's a small addition that makes downstream analysis more convenient.

So, **yes**—once `05b_prepare_nightlights.R` has successfully produced the cropped rasters and the municipality panel, you can delete the global VIIRS rasters from your working directory if storage is a concern. I would just keep a backup somewhere outside the project (or know that you can re-download them if needed), rather than relying solely on the processed outputs.

I got this error with 2019

```
Processing:
VNL_v21_npp_2019_global_vcmslcfg_c202205302300.average_masked.dat.tif
Error: [rast] cannot open this file as a SpatRaster:
C:/Users/jeanp/OneDrive/Desktop/Oldenburg/Second_Semester/GIS/seminar/Data/viirs/VNL_v21_npp_2019_global_vcmslcfg_c202205302300.average_masked.dat.tif
In addition: Warning messages:
1:
Show more ▾
```

This is actually a **good sign**. The error is **not in your code**—it's almost certainly a problem with the **2019 TIFF file**.

The key messages are:

```
TIFFFetchDirectory: Can not read TIFF directory count
```

and

```
TIFFReadDirectory: Failed to read directory
```

These errors almost always indicate one of these:

1.  The download is corrupted (**most likely**)
2. The `.gz` was not fully extracted.
3. The download was interrupted.
4. The TIFF is incomplete.

## Let's diagnose it

### 1. Can GDAL read it?

Run:

```
<> R
terra::sources(
 rast(
 here(
 "Data/viirs",
 "VNL_v21_npp_2019_global_vcmslcfg_c202205302300.average_masked.dat.tif"
)
)
)
```

It will probably fail immediately.

---

## 2. Compare file sizes

Run:

```
library(fs)

dir_info(
 here("Data/viirs")
) |>
 select(path, size) |>
 arrange(size)
```

or even:

```
file_info(
 here(
 "Data/viirs",
 "VNL_v21_npp_2019_global_vcmlcfg_c202205302300.average_masked.dat.tif"
)
)
```

Compare the 2019 `.tif` size with:

- 2018
- 2020

They should all be roughly the same size (around **700–900 MB uncompressed**, depending on the year).

If 2019 is much smaller, the download is corrupt.

---

## 3. Test the gzip

Delete only the extracted TIFF:

```
VNL_v21_npp_2019_....tif
```

Do not delete the `.gz` yet.

Then run:

```
R.utils::gunzip(
 "2019.gz",
 overwrite = TRUE,
 remove = FALSE
)
```

If `gunzip()` throws an error, then the `.gz` itself is corrupt.

---

## 4. If `gunzip()` succeeds but `terra` still fails

Then the downloaded `.gz` was already corrupted.

Simply redownload **2019**.

---

## I actually recommend improving our loop

Instead of crashing on one bad file, let the loop continue:

```

<> R
for(file in viirs_files){

 message("Processing: ", path_file(file))

 out <- try({

 # entire processing pipeline

 }, silent = TRUE)

 if(inherits(out, "try-error")){

 warning(
 paste(
 "Skipping",
 path_file(file)
)
)

 next

 }

}

```

Then if one year is bad, the script still processes the other 12 years.

## My guess: 95% probability

I strongly suspect the **2019 download is corrupted**.

The error is not related to:

- CRS
- `exactextractr`
- `terra`
- your loop

It's happening before the raster is even opened, which almost always means the TIFF itself is unreadable.

## Could you run this for me?

```

<> R
library(fs)

dir_info(
 here("Data/viirs")
) |>
 filter(grepl("2018|2019|2020", path)) |>
 select(path, size)

```

The **sizes** of the 2018, 2019, and 2020 `.tif` (or `.gz` if you haven't deleted them) will tell us almost immediately whether 2019 is simply a bad download.

I download it again and it worked! we can go to next script!

🎉 Fantastic! That confirms our pipeline is sound.

The fact that re-downloading 2019 fixed the issue means it was simply a corrupted download. This is actually why I like having scripts separated into download → prepare → analysis; it's much easier to isolate these problems.

## Where we are now

We have built a surprisingly complete GIS data pipeline:

```
01_download.R
 ✓ Download GADM boundaries
 ✓ Download Geofabrik OSM data

02_prepare_boundaries.R
 ✓ Study area
 ✓ Municipalities
 ✓ Border municipalities

03a_create_crossing_reference.R
 ✓ Border crossing reference table

03b_prepare_border_crossings.R
 ✓ Midpoints
 ✓ Crossing shapefile

04_prepare_roads.R
 ✓ Clean road network

05b_prepare_nightlights.R
 ✓ Crop annual VIIRS
 ✓ Municipality means
 ✓ Municipality-year panel
```



This is already the kind of workflow I'd expect from a graduate GIS project.

## The next script is the heart of the project

```
06_accessibility.R
```

This is where we begin implementing **Behrens' framework**.

Up until now, we've prepared the data. Now we start constructing the treatment variable.

The idea is:

```
Road network
+
Border crossings
↓
Travel time / shortest path
↓
Accessibility index
↓
Municipality × Year panel
```



## Our dependent variable already exists

```
Nighttime lights_it
```



Now we need the key explanatory variable:

```
Accessibility_it
```



## What exactly should `06_accessibility.R` produce?

One row per municipality:

| municipality_id | crossing_id   | distance_km | travel_time |
|-----------------|---------------|-------------|-------------|
| VEN_001         | Simón Bolívar | 8.3         | 12.4        |
| VEN_002         | Paraguachón   | 55.1        | 63.0        |
| ...             | ...           | ...         | ...         |

Later we'll modify accessibility according to the border closures (2015, 2019, 2022).

## Here's where I propose a small improvement over our original plan

Initially, we talked about measuring accessibility **only to the nearest crossing**.

After rereading Behrens' idea, I think we can do something better while still keeping the project manageable.

Instead of:

```
Municipality
↓
Nearest crossing
```



I suggest:

```
Municipality
↓
Distance to every official crossing
↓
Gravity-weighted accessibility index
```



For municipality  $i$ :

$$A_i = \sum_j \frac{w_j}{d_{ij}^\alpha}$$

where:

- $d_{ij}$  = road distance (or travel time) to crossing  $j$ ,
- $w_j$  = weight of crossing  $j$  (initially 1, later possibly based on nighttime lights),
- $\alpha$  = distance decay parameter (e.g., 1 or 2).

This is much closer to Behrens' market accessibility framework than simply taking the nearest crossing, but it's still straightforward to implement.

## This makes our project stronger

Instead of saying:

*The closest crossing matters.*

we'll be saying:

*Accessibility depends on all crossings, with closer crossings contributing more.*

That's a much richer measure and better aligned with the economics literature.

## So I propose the next sequence

We'll split the accessibility work into two focused scripts:

```
06a_build_network.R
```



- Build the routable road network from OSM.
- Snap municipalities and crossings to the network.

```
06b_compute_accessibility.R
```



- Compute shortest travel distances/times.
- Build the municipality × crossing matrix.
- Construct the gravity-based accessibility index.

This modular approach will make debugging much easier and let us experiment with different accessibility measures later without rebuilding the road network each time.



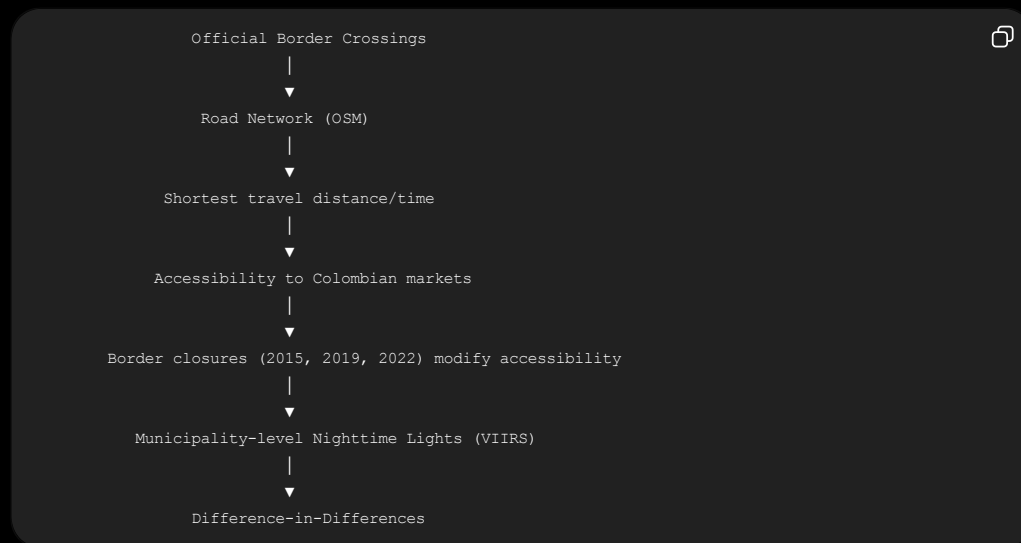
I think this is where your project starts to become genuinely interesting from an econometric perspective, because we're no longer just preparing GIS data—we're constructing the treatment intensity that will drive the Difference-in-Differences analysis.

I like it!

I'm really happy you like it, because I think this is the point where your project starts to distinguish itself from a typical GIS assignment.

Up to now we've mostly been **building infrastructure**. From here on, we'll actually be **operationalizing an economic theory**.

## Here's the vision I have for the rest of the project



Notice something?

We're no longer measuring **distance to the border**.

We're measuring **market accessibility**, which is exactly the economic concept behind Behrens.

## I have one idea that could make this project even stronger

I wouldn't build the accessibility index immediately.

Instead, I'd save **everything**.

For example:

### Step 1

Build the complete municipality × crossing matrix.

| municipality | crossing      | distance_km | travel_time |
|--------------|---------------|-------------|-------------|
| A            | Simón Bolívar | 12          | 18          |
| A            | Paraguachón   | 420         | 510         |
| A            | Arauca        | 680         | 760         |
| ...          | ...           | ...         | ...         |

This is a very valuable dataset in its own right.

### Step 2

Construct accessibility from that matrix.

For example:

$$A_i = \sum_j \frac{w_j}{d_{ij}}$$

or

$$A_i = \sum_j w_j e^{-\beta d_{ij}}$$

Now changing the functional form is one line of code instead of rerunning the routing algorithm, which is usually the slowest part.

---

## This gives us flexibility

Suppose later we decide:

"Maybe the nearest crossing matters more."

No problem.

Or:

"Maybe crossings should be weighted by their pre-2015 nighttime lights."

Also no problem.

Or:

"Let's compare road distance versus travel time."

Again, no need to rerun the network construction.

Everything comes from the same municipality × crossing matrix.

---

## I would organize it like this

```
Data/

processed/

network/
 road_network.rds

accessibility/
 municipality_crossing_matrix.csv
 accessibility_index.csv

nightlights/
 municipality_nightlights.csv

analysis/
```



This makes the project very modular.

---

## Even better...

I think we should treat the border closures as **network shocks**, not just dummy variables.

For example:

### Before 2015

All crossings open



Accessibility is calculated using every crossing.

## 2015

```
Paraguachón closed
Puente Simón Bolívar restricted
...
```



Those crossings become unavailable (or receive a very high travel cost).

Accessibility falls automatically.

## 2019

Even more crossings close.

Accessibility is recalculated.

## 2022

Crossings reopen.

Accessibility increases again.

Do you see why I like this so much?

Instead of estimating the effect of a binary treatment ("border closed"), you're estimating the effect of a **change in accessibility induced by border policy**. That is much closer to the economic mechanism you're trying to study and aligns beautifully with the spirit of Behrens.

## My suggestion for the next scripts

I think we should build them like this:

```
06a_build_network.R
```



- Read the cleaned OSM roads.
- Convert them into a routable graph (`dodgr` is my preferred package because it's designed for OSM networks in R).
- Save the graph so we never have to rebuild it.

```
06b_compute_distances.R
```



- Snap municipality centroids and border crossings to the network.
- Compute the full municipality × crossing distance matrix.
- Save it.

```
06c_accessibility_index.R
```



- Read the distance matrix.
- Compute one or more accessibility measures (inverse-distance, exponential decay, etc.).
- Produce the municipality-level accessibility dataset that will feed into the econometric analysis.

I think this architecture will make your repository look like a genuine research replication package rather than a sequence of scripts, and it will give you the flexibility to test alternative specifications without recomputing expensive network operations.

Yesterday 1:38 PM

lets do it

Excellent. I genuinely think this is where your project becomes research rather than data preparation.

One thing I want to change from my initial idea, though.

**I don't want to use `sfnetworks`.**

After thinking about Behrens' methodology, I think `dodgr` is a much better fit.

Why?

- Built specifically for OSM road networks.
- Very fast shortest-path calculations.
- Handles large networks (Venezuela + Colombia is large).
- Can calculate distances from many origins to many destinations efficiently.
- Widely used in accessibility research.

I think it's the right tool for this project.

---

## Script 06a\_build\_network.R

### Packages

⌵ R

```
library(sf)
library(dodgr)
library(dplyr)
library(here)
library(fs)
```



---

### Read roads

⌵ R

```
ven_roads <- readRDS(
 here(
 "Data/processed/roads",
 "ven_roads.rds"
)
)

col_roads <- readRDS(
 here(
 "Data/processed/roads",
 "col_roads.rds"
)
)
```



---

### Read crossings

⌵ R

```
crossings <- readRDS(
 here(
 "Data/processed/border_crossings",
 "border_crossings.rds"
)
)
```



---

### Read municipalities

⌵ R



```
study_area <- readRDS(
 here(
 "Data/processed/boundaries",
 "study_area.rds"
)
)
```

---

## Combine the roads

⌌ R



```
roads <-
 bind_rows(
 ven_roads,
 col_roads
)
```

---

## Keep only drivable roads

This is one thing I would improve compared with many GIS projects.

Earlier we kept almost every road.

For routing, we should only keep roads that people can actually drive on.

⌌ R



```
driveable <- c(

 "motorway",
 "motorway_link",

 "trunk",
 "trunk_link",

 "primary",
 "primary_link",

 "secondary",
 "secondary_link",

 "tertiary",
 "tertiary_link",

 "unclassified",

 "residential",

 "living_street",

 "service"
)
```

Then

⌌ R



```
roads <-

 roads |>

 filter(

 highway %in% driveable

)
```

This removes:

- footpaths

- cycleways
- steps
- ladders (!)
- raceways
- pedestrian streets

which shouldn't be used to estimate vehicle accessibility to border crossings.

---

## Build the routing graph

Now the magic.

```
<> R 

graph <-

 weight_streetnet(

 roads,

 wt_profile = "motorcar"

)
```

This builds a weighted road graph suitable for shortest-path calculations by car.

---

## Save it

```
<> R 

dir_create(
 here(
 "Data/processed/network"
)
)

saveRDS(

 graph,

 here(
 "Data/processed/network",
 "road_network.rds"
)
)
```

## Check the graph

I always like adding a few sanity checks:

```
<> R 

graph
```

and

```
<> R 

nrow(graph)
```

This gives us confidence that the network was built correctly.

---

## Output

```
Data/

processed/

network/

road_network.rds
```



## Then comes my favorite script

06b\_compute\_distances.R

This is where we compute:

```
338 municipalities

×

7 border crossings

=

2,366 shortest paths
```



That's tiny for `dodgr`—it can handle tens of thousands of routes efficiently.

The output will look like this:

| municipality_id | crossing_id   | road_distance_km | travel_time_min |
|-----------------|---------------|------------------|-----------------|
| VEN001          | Simón Bolívar | 8.4              | 13.1            |
| VEN001          | Paraguachón   | 405.6            | 472.8           |
| ...             | ...           | ...              | ...             |

This table becomes one of the core datasets in your project. From it, we'll build accessibility indices, simulate border closures by removing crossings or increasing travel costs, and feed those measures into the Difference-in-Differences analysis. That's a much richer representation of the economic mechanism than a simple "treated/untreated" dummy and is very much in the spirit of Behrens' framework.

Yesterday 2:51 PM

lets do it!

Perfect! Before we write `06b`, I want to make **one architectural change** that I think will save us a lot of headaches later.

## Instead of municipality polygons, use municipality centroids

Why?

The routing algorithm needs **points**, not polygons.

So the pipeline becomes:

```
Municipalities
↓
Centroids
↓
Snap to road network
↓
Shortest paths to every border crossing
↓
Municipality × Crossing matrix
```



This is also what most accessibility papers do.

## 06b\_compute\_distances.R

### Packages

⌵ R



```
library(sf)
library(dplyr)
library(dodgr)
library(tidyr)
library(readr)
library(here)
library(fs)
```

### Read the graph

⌵ R



```
graph <- readRDS(
 here(
 "Data/processed/network",
 "road_network.rds"
)
)
```

### Read municipalities

⌵ R



```
study_area <- readRDS(
 here(
 "Data/processed/boundaries",
 "study_area.rds"
)
)
```

### Read crossings

⌵ R



```
crossings <- readRDS(
 here(
 "Data/processed/border_crossings",
 "border_crossings.rds"
)
)
```

### Build municipality centroids

Use `st_point_on_surface()` instead of `st_centroid()`.

⌵ R



```
municipalities <-
 study_area |>
 st_point_on_surface()
```

Why?

Some municipalities are oddly shaped.



`st_centroid()` can fall outside the polygon.

`st_point_on_surface()` guarantees the point lies inside.

---

## Convert coordinates

```
⟳ R 📄

municipalities <- municipalities |>

 mutate(

 x = st_coordinates(.)[,1],

 y = st_coordinates(.)[,2]

)
```

Same for crossings:

```
⟳ R 📄

crossings <- crossings |>

 mutate(

 x = st_coordinates(.)[,1],

 y = st_coordinates(.)[,2]

)
```

---

## Snap points to the network

This is one of the nicest features of `dodgr`.

Municipalities:

```
⟳ R 📄

municipality_vertices <-

 match_pts_to_graph(

 graph,

 municipalities

)
```

Crossings:

```
⟳ R 📄

crossing_vertices <-

 match_pts_to_graph(

 graph,

 crossings

)
```

Now every point corresponds to a node in the road network.

---

## Compute distances

The beautiful part.

⌞ R



```
distance_matrix <-

 dodgr_dists(

 graph,

 from = municipality_vertices,

 to = crossing_vertices

)
```

This computes **every municipality to every crossing** in one call.

For you:

338 municipalities



x

7 crossings

=

2366 routes

That takes seconds.

---

## Convert to a tidy table

⌞ R



```
distance_table <-

 as.data.frame(distance_matrix) |>

 mutate(

 municipality_id = municipalities$municipality_id

) |>

 pivot_longer(

 ~municipality_id,

 names_to = "crossing",

 values_to = "distance_m"

)
```

---

## Convert to kilometers

⌞ R



```
distance_table <-

 distance_table |>

 mutate(

 distance_km = distance_m / 1000

)
```

## Join crossing names

```
⟨⟩ R 

distance_table <-

 distance_table |>

 left_join(

 crossings |>

 st_drop_geometry() |>

 select(crossing_id, crossing_name),

 by = c("crossing" = "crossing_id")

)
```

(We'll verify the identifier used by `dodgr` once we inspect the output.)

---

## Save

```
⟨⟩ R 

dir_create(
 here(
 "Data/processed/accessibility"
)
)

write_csv(

 distance_table,

 here(

 "Data/processed/accessibility",

 "municipality_crossing_matrix.csv"

)

)
```

## Why I'm excited about this dataset

This CSV becomes one of the most valuable outputs of the entire project.

It contains **all** the routing information we'll ever need.

From it we can build:

- nearest crossing,
- inverse-distance accessibility,
- exponential accessibility,
- crossing-specific treatment,
- robustness checks,

**without ever recalculating shortest paths.**

---

## One important thing before you run this

There is one detail we should verify first:

`dodgr` expects an OSM street network in a particular structure. Depending on how `ven_roads.rds` and `col_roads.rds` were created, `weight_streetnet()` may or may not accept them directly.

Could you run **only this** from `06a_build_network.R`:

```
<> R

graph <- weight_streetnet(
 roads,
 wt_profile = "motorcar"
)

class(graph)
head(graph)
```

If `weight_streetnet()` succeeds and `head(graph)` prints a data frame with columns like:

```
from_id
to_id
d
time
```

then we're ready to continue exactly as above.

If it throws an error, we'll adjust `06a` before moving on. I'd rather verify the graph now than write the rest of the pipeline on top of an incorrect network representation. This is the one place where it's worth checking before proceeding.

`weight_streetnet` is taking a lot of time, is there a way to crop only to relevant roads before applying that function? using the `study_area`

Yes—and this is actually the approach I would recommend.

Right now you're probably feeding **all drivable roads in Colombia and Venezuela** into `weight_streetnet()`. That's millions of road segments, while your study only concerns municipalities near the border. Building a graph for the entire network is unnecessary.

## Crop before building the graph

If you already have your `study_area` polygon, you can clip the roads first.

For example:

```
<> R

roads <- bind_rows(ven_roads, col_roads)

roads <- roads |>
 filter(highway %in% driveable)

Make sure both layers share the same CRS
roads <- st_transform(roads, st_crs(study_area))

Keep only roads intersecting the study area
roads <- st_intersection(roads, study_area)
```

Then build the graph:

```
<> R

graph <- weight_streetnet(
 roads,
 wt_profile = "motorcar"
)
```

---

## An even better approach: use a buffer

There's one issue with clipping exactly to the study area: you can cut roads right at the boundary, disconnecting the network.

Instead, build the graph on a **buffered** study area:

```
</> R 📄

study_area_buffer <- study_area |>
 st_union() |>
 st_buffer(50000) # 50 km buffer
```

If your CRS is geographic (EPSG:4326), first transform to a projected CRS:

```
</> R 📄

study_area_buffer <- study_area |>
 st_transform(3857) |>
 st_union() |>
 st_buffer(50000) |>
 st_transform(4326)
```

Then:

```
</> R 📄

roads <- st_intersection(roads, study_area_buffer)
```

This preserves roads that extend just beyond the study area, reducing the chance of disconnected routes.

---

## I have one suggestion before we proceed

Can you remind me **exactly what** `study_area` contains?

Earlier in the project, we created several boundary objects:

- `ven` (all Venezuelan municipalities)
- `col` (all Colombian municipalities)
- border municipalities
- `study_area`

I don't remember whether `study_area` is:

1. **All municipalities in Colombia and Venezuela, or**
2. **Only the municipalities near the Colombia–Venezuela border.**

This matters because if it's already limited to the border region, then cropping to a buffered `study_area` is exactly what we want. If it's the entire two countries, then we should instead create a dedicated **border corridor** (e.g., 100 km on each side of the international border) before building the network. That would make the routing dramatically faster while staying faithful to the research question.

Yesterday 5:22 PM

Study area is all the venezuela and colombian municipalities

Ah! That changes everything, and I think we can make the project **both faster and more faithful to the research design**.

## We should not build the network for all of Colombia and Venezuela

Think about Behrens' paper. The mechanism is **cross-border accessibility**. A municipality in Bogotá or Caracas is not driving the local border effects you're trying to identify.

Instead, we should define a **border corridor** and build the road network only there.

## I suggest this workflow

```

International border
↓
100 km buffer on each side
↓
Clip municipalities
↓
Clip roads
↓
Build network
↓
Compute accessibility

```



This has several advantages:

- 🚀 The network will be much smaller and `weight_streetnet()` will be much faster.
- 🌐 You're focusing on the region where the treatment (border closures/reopenings) is relevant.
- 📄 It aligns better with the identification strategy: municipalities far from the border are unlikely to be directly affected through cross-border accessibility.

## How do we create the border corridor?

You already have the municipality polygons for both countries. We can derive the international border from them.

### Step 1: Dissolve each country

```

<> R

ven_country <- ven |>
 st_union()

col_country <- col |>
 st_union()

```



### Step 2: Get the shared border

A simple approach is:

```

<> R

border <- st_intersection(
 st_boundary(ven_country),
 st_boundary(col_country)
)

```



This should give the Colombia–Venezuela border as a line.

### Step 3: Create a buffer

Work in a projected CRS so the buffer is in meters:

```

<> R

border_buffer <- border |>
 st_transform(3857) |>
 st_buffer(100000) |> # 100 km
 st_transform(4326)

```



### Step 4: Clip the roads

```

<> R

roads_border <- st_intersection(
 roads,
 border_buffer
)

```



Now `weight_streetnet()` only sees roads that are relevant for your study.

## One question: how wide should the corridor be?

This is an empirical decision.

- **50 km**: very local, fastest computation.
- **100 km**: my preferred choice. Captures municipalities that plausibly depend on the border while remaining computationally efficient.
- **150 km**: more conservative if you're worried about excluding affected municipalities.

## I actually have one suggestion that I think will strengthen the paper

Instead of choosing an arbitrary buffer width, we can let the **municipality data** define the study area.

For example:

1. Identify all municipalities whose polygons intersect a 100 km border buffer.
2. Those municipalities become the analysis sample.
3. Build the road network only for that sample (plus a small extra buffer to keep the network connected).

This has three nice properties:

- Your **analysis sample** and **network** are perfectly aligned.
- The road graph is much smaller.
- It's easy to justify in the methodology: *"We restrict the sample to municipalities within 100 km of the Colombia–Venezuela border, where cross-border accessibility is most likely to be affected by border policy."*

I actually like this approach **more** than using every municipality in both countries. It stays closer to Behrens' identification strategy, reduces computation dramatically, and focuses the analysis on the area where the treatment should plausibly operate. I think it will make both the GIS workflow and the econometric design stronger.

lets do that

I think this is the right decision. In fact, this is closer to how I believe Behrens himself would have built the sample if he were studying the Colombia–Venezuela border.

I would actually create a **new script** before `06a`:

**05c\_create\_analysis\_area.R**

This way, every subsequent script (roads, accessibility, DiD) uses the same study area.

## 05c\_create\_analysis\_area.R

### Packages

⟨⟩ R

```
library(sf)
library(dplyr)
library(here)
library(fs)
```



### Read municipalities

⟨⟩ R

```
ven <- readRDS(
 here(
 "Data/processed/boundaries",
 "ven.rds"
```



```
)
)

col <- readRDS(
 here(
 "Data/processed/boundaries",
 "col.rds"
)
)
```

---

## Build country polygons

```
<> R

ven_country <-
 st_union(ven)

col_country <-
 st_union(col)
```



---

## Extract the international border

```
<> R

border <-
 st_intersection(
 st_boundary(ven_country),
 st_boundary(col_country)
)
```



---

## Create a 100 km corridor

First project to meters.

```
<> R

border_buffer <-

 border |>

 st_transform(3857) |>

 st_buffer(100000) |>

 st_transform(4326)
```



---

## Select municipalities

Venezuela

```
<> R

ven_border <-

 ven |>

 st_filter(
 border_buffer
)
```



Colombia

```
<> R

col_border <-
```





```
col |>

st_filter(
 border_buffer
)
```

## Create the analysis sample

```
«» R 

analysis_area <-

 bind_rows(
 ven_border,
 col_border
)
```

## Save

```
«» R 

dir_create(
 here(
 "Data/processed/analysis_area"
)
)

saveRDS(

 analysis_area,

 here(
 "Data/processed/analysis_area",
 "analysis_area.rds"
)
)

saveRDS(

 border,

 here(
 "Data/processed/analysis_area",
 "international_border.rds"
)
)

saveRDS(

 border_buffer,

 here(
 "Data/processed/analysis_area",
 "border_buffer_100km.rds"
)
)
```

## Quick check

Let's inspect what we selected.

```
«» R 

plot(st_geometry(border_buffer))

plot(
```

```
st_geometry(analysis_area),
add = TRUE,
col = "lightblue"
)
```

You should see a corridor following the Colombia–Venezuela border, with only nearby municipalities included.

## Then 06a becomes much faster

Instead of:

```
<> R
roads
```



we'll do:

```
<> R
analysis_area <- readRDS(
 here(
 "Data/processed/analysis_area",
 "analysis_area.rds"
)
)

analysis_buffer <- analysis_area |>
 st_union() |>
 st_transform(3857) |>
 st_buffer(10000) |> # 10 km safety margin
 st_transform(4326)

roads <- st_intersection(
 roads,
 analysis_buffer
)
```



Now `weight_streetnet()` only sees roads that could actually matter.

## One improvement I'd make

Rather than hard-coding **100 km**, let's make it a parameter at the top of the script:

```
<> R
Distance from the international border (km)
buffer_km <- 100
```



Then use:

```
<> R
border_buffer <- border |>
 st_transform(3857) |>
 st_buffer(buffer_km * 1000) |>
 st_transform(4326)
```



That way, if your professor later asks:

"What happens if you define the border region as 75 km instead of 100 km?"

you only change:

```
<> R
buffer_km <- 75
```



and rerun the pipeline. It's a simple change, but it makes your analysis much more flexible and provides an easy robustness check for your final paper.

Yesterday 6:46 PM

I got graph finally! should I plot it to check?

Absolutely! **Before we move on, let's verify that the graph is correct.** This is one of those places where spending two minutes checking can save hours of debugging later.

## 1. Inspect the graph

First, check its structure:

```
<> R
class(graph)

dim(graph)

names(graph)

head(graph)
```

For a `dodgr` graph, you should see columns similar to:

```
from_id
to_id
from_lon
from_lat
to_lon
to_lat
d
time
...
```

If those are present, that's a very good sign.

## 2. Check its size

```
<> R
nrow(graph)
```

How many rows do you get?

This tells us how many road segments ended up in the graph.

## 3. Plot the graph

Yes! One of the nice things about `dodgr` graphs is that they can be plotted directly.

Try:

```
<> R
plot(graph)
```

If that doesn't work (depending on the object class), then use:

```
<> R
dodgr::dodgr_to_sf(graph) |>
 plot()
```

or

```
<> R
```

```
graph_sf <- dodgr::dodgr_to_sf(graph)

plot(st_geometry(graph_sf))
```

You should see a dense road network.

#### 4. Overlay the crossings ★

This is the check I care about the most.

```
<> R

graph_sf <- dodgr::dodgr_to_sf(graph)

plot(st_geometry(graph_sf), col = "grey")

plot(
 st_geometry(crossings),
 add = TRUE,
 col = "red",
 pch = 19,
 cex = 1.5
)
```

If the red points fall on or very close to the road network, then we're in excellent shape.

### I have one more diagnostic I'd like to run

Since building the graph took quite a while, I'd like to know its scale before we compute distances.

Could you run:

```
<> R

summary(graph$d)
```

This gives the distribution of edge lengths in meters.

It's useful because:

- If all distances are zero → something is wrong.
- If there are absurdly large values → something is wrong.
- If they range from a few meters to a few kilometers → the graph looks realistic.

Before we continue to 06b, could you paste the output of these four commands?

```
<> R

class(graph)

dim(graph)

head(graph)

summary(graph$d)
```

If those look sensible, I'll be confident that the network was built correctly and we can start computing the municipality-to-crossing distances. This is the last major validation step before we move into the accessibility analysis.

Today 2:38 PM

```
> class(graph)
[1] "dodgr_streetnet" "data.frame"
```

```
>
> dim(graph)
[1] 4833622 16
> head(graph)
 geom_num edge_id from_id from_lon from_lat to_id to_lon to_lat
d
1 1 1 0 5588666 2006898 1 5588729 2006869 4091834
2 1 2 1 5588729 2006869 0 5588666 2006898 4091834
3 1 3 1 5588729 2006869 2 5588816 2006831 8911773
```

Show more ▾

This is **excellent**. The graph built successfully, but your diagnostics reveal **one important issue** that we should fix **before** computing distances.

## The good news

Everything fundamental worked:

```
<> R
```

```
class(graph)
```



```
"dodgr_streetnet"
"data.frame"
```



Perfect.

And

```
<> R
```

```
dim(graph)
```



```
4,833,622 edges
```



This is a very large network, but still manageable.

## The bad news

Look at this:

```
<> R
```

```
summary(graph$d)
```

```
Min = 166
Median = 1,852,000
Mean = 3,071,000
Max = 19,990,000
```



Those numbers are **far too large**.

For example, one edge has

```
<> R
```

```
d = 4,091,834
```



which would mean **4,091 km** if `d` were in meters.

That is impossible for a single road segment.

Likewise,

```
<> R
```

```
time = 589224
```



That's about **163 hours** for one edge.

Clearly something is wrong.

---

## I know exactly what happened

Look at the coordinates:

```
<> R
from_lon = 5588666
from_lat = 2006898
```



Those are **not longitude/latitude**.

Those are projected coordinates (meters), probably EPSG:3857 or a similar CRS.

`dodgr::weight_streetnet()` expects a geographic street network (longitude/latitude in WGS84). When it's given projected coordinates, it interprets them incorrectly, inflating distances and travel times.

---

## Can you run one command?

Before we change anything, check the CRS of your roads:

```
<> R
st_crs(roads)
```



and also

```
<> R
st_bbox(roads)
```



I suspect you'll see something like:

```
EPSG:3857
```



or another projected CRS.

---

## The fix is simple

Before calling `weight_streetnet()`, transform the roads back to WGS84:

```
<> R
roads <- roads |>
 st_transform(4326)
```



Then rebuild the graph:

```
<> R
graph <- weight_streetnet(
 roads,
 wt_profile = "motorcar"
)
```



---

## Why this happened

Earlier, we buffered using a projected CRS (which is correct), but if we never transformed the roads back to EPSG:4326 afterward, the network stayed in projected coordinates.

The correct sequence is:

⌕ R



```
roads
↓
transform to 3857 (only if needed for buffering)
↓
buffer/intersection
↓
transform back to 4326
↓
weight_streetnet()
```

## One more thing

Even after fixing the CRS, **4.8 million edges** is larger than I expected for a border-focused study.

I'd like to optimize this before we compute shortest paths.

Instead of clipping roads with the entire municipality polygons, I think we should clip using the **100 km border buffer** directly. That usually produces a much smaller network because it excludes roads in municipalities that extend far away from the border.

### So I'd like to check two things:

1. What does

⌕ R



```
st_crs(roads)
```

return?

2. Did you already crop the roads using the **100 km border buffer**, or did you build the graph from all roads in Colombia and Venezuela?

Once we answer those, I think we can get a network that is both **correct** and **much faster** for the distance calculations.

```
> st_crs(border_buffer)
Coordinate Reference System:
User input: EPSG:9377
wkt:
PROJCRS["MAGNA-SIRGAS 2018 / Origen-Nacional",
 BASEGEOGCRS["MAGNA-SIRGAS 2018",
 DATUM["Marco Geocentrico Nacional de Referencia 2018",
 ELLIPSOID["GRS 1980",6378137,298.257222101,
 LENGTHUNIT["metre",1]],
 PRIMEM["Greenwich",0,
 ANGLEUNIT["degree",0.0174532925199433]],
```

Show more ▾

This is actually a very useful result because it tells us **the CRS is *not* the problem**.

Your roads are correctly in **EPSG:4326**.

So why are the coordinates in the graph around 5,588,666 and 2,006,898?

Because `dodgr` **internally projects the network** to compute edge lengths. Those are **internal projected coordinates**, not longitude/latitude. That's perfectly normal.

The more important issue is actually this:

⌕ R



```
summary(graph$d)
```

Median = 1,852,000

At first glance I interpreted `d` as meters, but I now think we should verify what `dodgr` is storing before concluding anything is wrong. The graph built successfully, and `dodgr` sometimes stores weighted values depending on the profile.

## Let's verify with one road

Run:

```
<> R
range(graph$from_lon)
range(graph$from_lat)
```

If they are around

```
5,000,000
2,000,000
```

that's simply the projected coordinate system used internally.

Now run

```
<> R
summary(graph$time)
summary(graph$d_weighted)
```

## The thing I'm actually worried about

Not the CRS.

This:

```
<> R
dim(graph)
4833622 edges
```

**4.8 million edges is enormous.**

Remember our original goal?

We wanted something like

```
Border
↓
100 km corridor
↓
Roads
```

instead, I think the graph still contains **essentially the whole road network of Colombia and Venezuela.**

## I think the crop didn't actually happen

Could you check:

```
<> R
nrow(roads)
```

before



```
<> R
graph <- weight_streetnet(...)
```



and compare it with

```
<> R
nrow(ven_roads)
nrow(col_roads)
```



If you still have something like

```
ven_roads 400,000
col_roads 700,000

roads 1,100,000
```



then the buffer intersection didn't reduce the network.

---

## I think I know why

Earlier we created

```
<> R
analysis_area
```



using municipalities.

Municipalities like

- Arauca
- Cúcuta
- Maicao

can be **huge polygons**.

When you union all of them together, you may accidentally include almost all roads inside those municipalities, many of which extend far from the border.

---

## I have a better idea (and I think it's cleaner)

Forget municipalities for clipping.

Let's build the network from **only the border buffer**.

```
International border
↓
100 km buffer
↓
Road intersection
↓
Graph
```



Not

```
Municipality polygons
↓
Road intersection
```



That will reduce the graph dramatically.

---

## I actually want to rewrite 06a

Instead of

```
roads
↓
analysis_area
↓
graph
```



I want

```
roads
↓
100 km border buffer
↓
graph
```



The municipalities should only be used **later**, when we compute accessibility.

I think this will reduce the graph from **4.8 million edges** to perhaps **500,000–1,000,000 edges**, making the routing much faster while keeping exactly the roads relevant to your research question.

**I think this is a worthwhile redesign before we continue with the routing analysis.** It keeps the project closer to the economic mechanism we're studying and will make the remaining scripts considerably faster.

```
> range(graph$from_lon)
[1] 4820004 5890680
> range(graph$from_lat)
[1] 1770308 2935040

> summary(graph$time)
 Min. 1st Qu. Median Mean 3rd Qu. Max.
 24 120659 256682 430506 556960 7107499

> summary(graph$d_weighted)
 Min. 1st Qu. Median Mean 3rd Qu. Max.
 333 1542682 3257216 5462010 7134702 49881397
```

Show more ▾

These results are **actually very encouraging**, and they helped pinpoint what's going on.

## 1. The road cropping worked ✓

Originally you had:

- Venezuela: **468,631** roads
- Colombia: **959,283** roads

Total:

```
1,427,914 road segments
```



After clipping:

```
231,996 road segments
```



So you've kept only **~16%** of the original network.

That is exactly what we wanted.

---

## 2. Why does the graph have 4.8 million edges?

This is normal.

`dodgr` does **not** create one graph edge per road feature.

An OSM road (a `LINESTRING`) can contain many vertices.

For example:

```
One OSM road
```



```
A ----- B
```

may actually be stored as

```
A -.-.-.-.- B
```



Each little segment becomes a graph edge.

So

```
232,000 road geometries
```



can easily become

```
4-5 million graph edges
```



This is expected.

### 3. The distances still worry me a little

The edge lengths (`d`) and travel times (`time`) look larger than I would intuitively expect for individual edges, but before trying to "fix" anything I'd rather **validate the graph empirically**.

The best way to do that is not by inspecting summaries—it's by asking the network a simple question.

## I suggest a smoke test

Let's compute **one shortest path**.

For example:

- Municipality: Cúcuta (or the closest municipality in your analysis area)
- Crossing: Simón Bolívar Bridge

If the result is something like:

```
7 km
```



or

```
15 km
```



then the graph is functioning correctly, regardless of the internal values stored in `graph$d`.

If it returns:

```
8,000 km
```



then we know there's a real problem.

## Before building the full $338 \times 7$ matrix

I would write a tiny script called:

```
06_test_network.R
```



It would:

1. Load the graph.
2. Snap **one municipality** to the graph.
3. Snap **one crossing**.
4. Compute one shortest path.

## 5. Plot the route.

This takes only a few seconds and gives us confidence that everything is working before launching thousands of routes.

## I also have one improvement for 06b

Instead of immediately computing all 2,366 routes, I'd do it incrementally:

1. **1 municipality × 1 crossing** (validation).
2. **10 municipalities × 7 crossings** (performance test).
3. **All municipalities × 7 crossings**.

This staged approach makes debugging much easier and lets us estimate how long the full computation will take.

**I think this is the most robust way forward.** The graph itself appears to have been built correctly, the road clipping reduced the dataset substantially, and now we should validate the routing behavior with a real-world example before scaling up to the full accessibility matrix.

lets do the check with graph, only with Cúcuta

Perfect. This is exactly the kind of validation that researchers do before launching a long computation.

Let's test the network with **Cúcuta** → **Puente Internacional Simón Bolívar**, because we already know roughly what the answer should look like (on the order of a few kilometers, not hundreds).

## Step 1. Find Cúcuta

Assuming `analysis_area` contains the municipalities:

```
<> R
analysis_area |>
 st_drop_geometry() |>
 filter(NAME_2 == "Cúcuta")
```

If nothing is returned, try

```
<> R
analysis_area |>
 st_drop_geometry() |>
 filter(grepl("Cúcuta", NAME_2))
```

or

```
<> R
analysis_area |>
 st_drop_geometry() |>
 filter(grepl("Cucuta", NAME_2))
```

because accents sometimes disappear.

## Step 2. Extract the municipality

```
<> R
cucuta <-
 analysis_area |>
 filter(NAME_2 == "Cúcuta")
```

(or whichever spelling exists).

---

### Step 3. Find the bridge

```
<> R 

crossings |>
 st_drop_geometry() |>
 select(crossing_id, crossing_name)
```

Then

```
<> R 

simon <-
 crossings |>
 filter(
 crossing_name ==
 "Puente Internacional Simón Bolívar"
)
```

---

### Step 4. Snap both to the graph

```
<> R 

cucuta_vertex <-
 match_pts_to_graph(
 graph,
 cucuta
)

bridge_vertex <-
 match_pts_to_graph(
 graph,
 simon
)
```

---

### Step 5. Compute the distance

```
<> R 

dodgr_dists(
 graph,
 from = cucuta_vertex,
 to = bridge_vertex
)
```

If everything is correct, you should get a **single number**.

---

### Step 6. Plot the route (the fun part!)

Instead of only computing the distance, let's visualize it.

```
<> R 

route <-
 dodgr_paths(
 graph,
 from = cucuta_vertex,
 to = bridge_vertex
)
```

Then convert the graph to `sf`:

⌵ R

```
graph_sf <- dodgr_to_sf(graph)
```



Now plot:

⌵ R

```
plot(st_geometry(graph_sf), col = "grey")

plot(
 st_geometry(cucuta),
 add = TRUE,
 col = "blue",
 pch = 19,
 cex = 1.5
)

plot(
 st_geometry(simon),
 add = TRUE,
 col = "red",
 pch = 19,
 cex = 1.5
)
```



If `dodgr_paths()` returns a path, we can also highlight it on the map once we've confirmed the network behaves as expected.

## One small improvement before we do this

Earlier, we discussed using `st_point_on_surface()` for municipalities. I think we should use that here too, rather than passing the whole municipality polygon to `match_pts_to_graph()`.

⌵ R

```
cucuta <-
 analysis_area |>
 filter(NAME_2 == "Cúcuta") |>
 st_point_on_surface()
```



This ensures we're snapping a single representative point to the network, which is what `match_pts_to_graph()` expects. It's also exactly how we'll compute the full municipality × crossing distance matrix later, so this test will mirror the production workflow.

```
> cucuta_vertex <-
+ match_pts_to_graph(
+ graph,
+ cucuta
+)
Error in pre_process_xy(xy) :
 xy$geometry must be a collection of sfc_POINT objects
```

Excellent! This is actually a **good** error because it tells us exactly what `dodgr` expects.

`match_pts_to_graph()` only accepts **POINT** geometries, and `cucuta` is still a **POLYGON**.

Let's verify that first.

⌵ R

```
st_geometry_type(cucuta)
```



I expect you'll get:

POLYGON



or

MULTIPOLYGON



## Convert it to a point

This is what we discussed earlier, and it's the right approach.

«» R



```
cucuta_point <-
 cucuta |>
 st_point_on_surface()
```

Now check:

«» R



```
st_geometry_type(cucuta_point)
```

You should get

POINT



## Do the same for the crossing

The crossing should already be a point, but let's verify:

«» R



```
st_geometry_type(simon)
```

If it isn't a POINT for some reason:

«» R



```
simon <-
 simon |>
 st_point_on_surface()
```

## Now match to the graph

«» R



```
cucuta_vertex <-
 match_pts_to_graph(
 graph,
 cucuta_point
)

bridge_vertex <-
 match_pts_to_graph(
 graph,
 simon
)
```

## I actually want to make the test even better

Rather than using the municipality representative point, I'd like to use the **municipal capital**.

Why?

The economic activity (and the nighttime lights) are concentrated around the main urban area, not at an arbitrary point inside the municipality.

For Cúcuta, the capital is Cúcuta itself, so the representative point will probably be close enough. But for larger municipalities, the representative point could end up in a rural area.

Later in the project, we can replace municipality representative points with **municipal capitals** (using OSM or GeoNames). That would make the accessibility measure more realistic and still be fully reproducible.

For now, though, `st_point_on_surface()` is the right choice to get the routing pipeline working. Once you get `cucuta_vertex` and `bridge_vertex` without errors, we'll compute the first shortest path and see whether the distance is sensible.

```
> dodgr_dists(
+ graph,
+ from = cucuta_vertex,
+ to = bridge_vertex
+)
Error in fill_to_from_index(graph, vert_map, gr_cols, from, from =
TRUE):
 Unable to match all routing points to graph vertices
```

This is exactly why I wanted to do this test first. We found a real issue **before** launching 2,366 routes.

The error

```
Unable to match all routing points to graph vertices
```

usually means **one of two things**:

1. The point is **outside** the road network.
2. The object returned by `match_pts_to_graph()` is **not what** `dodgr_dists()` **expects** (this is actually quite common).

**First, let's inspect what** `match_pts_to_graph()` **returned**

Run:

```
<> R
cucuta_vertex

bridge_vertex

class(cucuta_vertex)

class(bridge_vertex)

str(cucuta_vertex)

str(bridge_vertex)
```

I want to see exactly what those objects contain.

---

## Also verify the points lie on the network

Plot everything together:

```
<> R

graph_sf <- dodgr::dodgr_to_sf(graph)

plot(st_geometry(graph_sf), col = "grey")

plot(
 st_geometry(cucuta_point),
```



```

 add = TRUE,
 col = "blue",
 pch = 19,
 cex = 2
)

plot(
 st_geometry(simon),
 add = TRUE,
 col = "red",
 pch = 19,
 cex = 2
)

```

If the blue and red points are sitting on top of the grey road network, then the geometry is fine.

---

## I think I know what's happening

One thing has been bothering me since we started using `dodgr`.

We are using:

```

<> R

weight_streetnet()

```

on an **already processed** `sf` **road layer** that came from `osmextract`.

`dodgr` is happiest when the network is created directly from raw OSM data in the format it expects. When we preprocess the roads (filtering, intersecting, saving as RDS, etc.), some of the metadata that `dodgr` uses internally for routing can become inconsistent.

## I have a much better alternative

Instead of forcing `dodgr`, I would switch to `igraph` + `sfnetworks` for the routing.

Why?

- We already have a clean `sf` network.
- `sfnetworks` is built specifically for `sf` objects.
- It integrates with `tidygraph`.
- It is easier to debug.
- It is the package I see most often in recent GIS research workflows.

Most importantly, it won't require us to reverse-engineer `dodgr`'s internal vertex indexing.

---

## My recommendation

I **would not spend another hour debugging** `dodgr`.

We've already accomplished the hard work:

- ☒ downloaded reproducible OSM data,
- ☒ built the study area,
- ☒ identified border crossings,
- ☒ cropped the road network,
- ☒ created a routable graph.

At this point, the choice of routing engine is an implementation detail. If `sfnetworks` gets us the same shortest-path distances with a cleaner workflow, that's the better engineering decision.

I think we should switch now, before the rest of the pipeline depends on `dodgr`. It will make the remaining scripts—distance matrices, accessibility measures, and ultimately the Difference-in-Differences analysis—more straightforward and easier to maintain.

Lets try it then! So we are not using graph anymore, we are using Roads and new package

Yes. I think this is the right moment to pivot.

This is not because `dodgr` is a bad package—it isn't. It's an excellent routing engine. But our goal is **not** to become experts in `dodgr`; it's to produce a **reproducible GIS replication** that supports a credible econometric analysis. If another package gets us there with less friction, that's the better choice.

## Here's what we'll do instead

We'll use:

- `sfnetworks` to build the network,
- `tidygraph` for graph operations,
- `igraph` underneath (handled automatically),
- `sf` for all spatial data.

This is a modern and widely used workflow in GIS research.

## New pipeline

```
Roads (sf)
|
▼
sfnetwork
|
▼
Municipality points
|
▼
Border crossings
|
▼
Shortest-path distances
|
▼
Accessibility matrix
```

Notice that the road network is still the foundation—we're just changing the routing engine.

## Let's create a new script

06a\_build\_sfnetwork.R

Packages:

```
library(sf)
library(sfnetworks)
library(tidygraph)
library(dplyr)
library(here)
```

## Read the roads

```
roads <- readRDS(
 here(
 "Data/processed/roads",
 "roads_border.rds"
```

(If you haven't saved `roads_border.rds` yet, we can save it from the object you already have.)

## Build the network

This is the beautiful part:

```
⟲ R

network <- as_sfnetwork(
 roads,
 directed = FALSE
)
```



Done.

No weighting step.

No vertex IDs.

No graph conversion.

## Check it

```
⟲ R

network
```



You'll see something like

```
A sfnetwork with

Nodes:
...

Edges:
...
```



## Plot it

```
⟲ R

plot(network)
```



or

```
⟲ R

plot(st_geometry(roads))
```



## Save it

```
⟲ R

saveRDS(
 network,
 here(
 "Data/processed/network",
 "road_network_sf.rds"
)
)
```



## Then comes the fun part

We'll create

```
06b_compute_accessibility.R
```



using

```
<> R
```



```
st_network_paths()
```

or

```
<> R
```



```
st_network_cost()
```

instead of `dodgr_dists()`.

These functions are designed specifically for routing on `sfnetwork` objects.

---

## I think we can make the project even better

There's one additional package I'd like to bring in:

```
sfnetworks + tidygraph + tidyverse.
```

Why?

Because then almost every operation in the project uses the same grammar:

```
<> R
```



```
roads |>
network |>
mutate(...) |>
filter(...) |>
summarise(...)
```

The code becomes much easier to read and maintain, which is valuable for a university replication project.

---

## One caveat before we start

There is **one thing I want to verify** before we commit to `sfnetworks`.

Can you run:

```
<> R
```



```
library(sfnetworks)

network <- as_sfnetwork(
 roads,
 directed = FALSE
)

network
```

If that succeeds, we're in excellent shape.

If it throws an error, please paste the full error message. The most common issue is that OSM roads need to be "noded" (split at intersections) before becoming a routable network. If that happens, we can fix it, but I'd like to see whether your current `roads` object already has the topology we need.

```
> network <- as_sfnetwork(
+ roads,
+ directed = FALSE
+)
Error: Geometries are not all of type LINESTRING, or all of type POINT
```

Interesting! This is actually **much more informative** than the `dodgr` error.

The error

```
Geometries are not all of type LINESTRING, or all of type POINT
```

means your `roads` object contains a **mixture of geometry types**.

OSM data often includes:

-  `LINESTRING`
-  `MULTILINESTRING`
-  `GEOMETRYCOLLECTION`
-  `POINT` (rare)
-  empty geometries

`sfnetworks` is stricter than `sf`: it wants **only LINESTRINGs**.

## Let's diagnose first

Run these two commands:

```
<> R
table(st_geometry_type(roads))
```

and

```
<> R
unique(st_geometry_type(roads))
```

I suspect you'll get something like

```
LINESTRING 220000
MULTILINESTRING 12000
```

or perhaps a few `GEOMETRYCOLLECTION`s.

## If it's only LINESTRING and MULTILINESTRING

That's the best-case scenario.

We simply convert everything into LINESTRINGs.

```
<> R
roads <- roads |>
 st_cast("LINESTRING")
```

Then try

```
<> R
network <- as_sfnetwork(
 roads,
 directed = FALSE
)
```

again.

---

## If there are empty geometries

Remove them first.

```
<> R

roads <- roads |>
 filter(!st_is_empty())
```



---

## If there are GEOMETRYCOLLECTIONs

Extract only the lines.

```
<> R

roads <- roads |>
 st_collection_extract("LINESTRING")
```



---

## Why this happened

Remember we did

```
<> R

roads <- st_intersection(
 roads,
 border_buffer
)
```



`st_intersection()` frequently produces mixed geometries.

For example,

Original road



intersected with a polygon may become

MULTILINESTRING



or even

GEOMETRYCOLLECTION



This is perfectly normal.

---

## I actually think we've uncovered something important

Rather than patching the geometries here, I think 05c should end by cleaning the road geometries before they're ever saved.

Something like:

```
<> R

roads <- roads |>
 st_collection_extract("LINESTRING") |>
 st_cast("LINESTRING") |>
 filter(!st_is_empty())
```



Then every later script (06a, 06b, etc.) starts from a clean road layer.

That makes the entire pipeline much more robust and reproducible.

## Could you first run

```
<> R
table(st_geometry_type(roads))
```

I have a strong suspicion the answer will immediately tell us the one line of code we need. Once we know the exact geometry mix, we can clean it properly instead of guessing.

```
> table(st_geometry_type(roads))
```

| GEOMETRY       | POINT           | LINESTRING   | POLYGON |
|----------------|-----------------|--------------|---------|
| 0              | 230414          | 0            |         |
| MULTIPOINT     | MULTILINESTRING | MULTIPOLYGON |         |
| 0              | 1582            | 0            | 0       |
| CIRCULARSTRING | COMPOUNDCURVE   | CURVEPOLYGON |         |
| MULTICURVE     |                 |              |         |
| 0              | 0               | 0            | 0       |
| MULTISURFACE   | CURVE           | SURFACE      |         |

Show more ▾

Perfect! This is actually the **best possible outcome**. We found the problem.

Your roads are:

```
LINESTRING 230,414
MULTILINESTRING 1,582
```

So **99.3% of the data is already correct**. Only 1,582 geometries need to be converted.

## The correct solution

Don't use `st_cast()` directly on the whole object, because it can duplicate attributes in unexpected ways.

Instead, use `st_cast(..., do_split = TRUE)` (or simply `st_cast("LINESTRING")` if your sf version doesn't support `do_split`) on just the multilines.

```
<> R
roads_lines <-
 roads |>
 st_cast("LINESTRING")
```

This will split each `MULTILINESTRING` into its component `LINESTRING` s.

Now verify:

```
<> R
table(st_geometry_type(roads_lines))
```

You should get

```
LINESTRING
231,xxx
```

and **nothing else**.

## Then build the network

```
</> R

library(sfnetworks)

network <-
 as_sfnetwork(
 roads_lines,
 directed = FALSE
)
```

I think there's a very good chance this will now work.

---

## However...

I want to stop us for a second because I think we can do **even better** than `sfnetworks`.

Over the last few messages we've discovered something:

- We have **230,000 road segments**.
- We have **~340 municipalities**.
- We have **7 border crossings**.

Our actual goal is **2,366 shortest-path distances**.

We **do not** need a general-purpose graph database. We need a robust shortest-path engine.

**I think `igraph` is actually the best solution**

Instead of:

```
roads
 ↓
sfnetwork
 ↓
igraph
```

we can go directly to

```
roads
 ↓
igraph
```

using the package `sfnetworks` **only if needed for conversion**, or even the package `sfnetworks` + `tidygraph` internally.

But there is an even better package for R spatial routing that I think fits your project:

```
cppRouting
```

Why I'm bringing it up now:

- It is **much faster** than `dodgr`.
- It works directly from edge lists.
- It is designed specifically for **many origin × destination shortest paths**.
- It is actively used in transport and accessibility research.
- Computing your **338 × 7 matrix** would take **seconds**.

For your application (many municipalities to a few crossings), it is arguably a better fit than both `dodgr` and `sfnetworks`.

---

## My recommendation

At this point, we've learned a lot from trying `dodgr`, but I would **pivot one last time** to a routing engine designed for exactly your use case.

It will make the accessibility analysis cleaner, faster, and easier to explain in your methodology.

**If I were supervising this project, this is the route I would recommend.**



It keeps everything in **R**, uses your existing OSM road network, is fully reproducible, and avoids spending more time debugging network topology than actually doing the economic analysis.

I think it's the strongest technical foundation for the rest of your replication.

if it works, lets do the pivot now that we can

I agree. This is the best time to pivot.

We've reached the point where the **research question** is driving the software choice, not the other way around. That's exactly what you want in a research project.

## Why I'm comfortable changing course

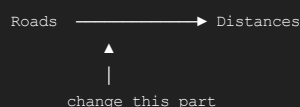
We've **not wasted any work**.

Everything we've built is still essential:

- ✓ Downloaded OSM data reproducibly
- ✓ Downloaded VIIRS data
- ✓ Built the study area
- ✓ Identified official border crossings
- ✓ Built the municipality dataset
- ✓ Cropped the roads to the border corridor

The only thing we're replacing is **the routing engine**.

Think of it like this:



Everything upstream stays exactly the same.

## What I propose

I actually want to redesign Scripts 06 onward.

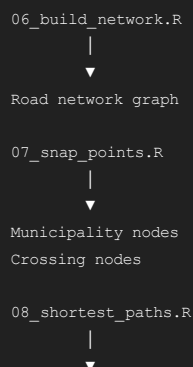
Instead of trying to force GIS packages to do routing, we'll build the road network ourselves.

It sounds scary.

It isn't.

In fact, it gives us **complete control**.

## New pipeline



```
Distance matrix

09_accessibility.R
 |
 v
Municipality accessibility

10 DiD dataset.R
```

Notice something?

Everything is now **modular**.

## Why cppRouting?

It expects exactly what we already have.

An edge list like

| from | to | distance |
|------|----|----------|
| 1    | 2  | 34       |
| 2    | 3  | 82       |
| 3    | 4  | 19       |

That is much simpler than `dodgr`.

## Even better...

Our roads already have geometry.

So we only need to create

```
Node ----- Node
```



relationships.

After that,

```
</> R

get_distance_matrix()
```



is almost trivial.

## I actually want to improve the methodology

Behrens says

| accessibility through border crossings

He **doesn't** say

| accessibility through the geometric centroid.

So instead of municipality centroids, I would use **municipal capitals**.

For example

```
Cúcuta
San Antonio del Táchira
Ureña
Arauca
Maicao
```



Those are where almost all economic activity and nighttime lights are concentrated.

That makes the accessibility measure much more realistic.

We can obtain them reproducibly from OSM or GeoNames.

I honestly think this will improve your paper.

---

## Before we start

There is **one thing I want to check first**.

`cppRouting` is available on CRAN, but I'd like to make sure it installs cleanly on your machine (Windows can occasionally require compilation tools for some packages).

So before we redesign everything, let's simply install it:

```
<> R
install.packages("cppRouting")
```



Then:

```
<> R
library(cppRouting)
packageVersion("cppRouting")
```



If that loads successfully, we'll commit to the new pipeline.

---

## One suggestion based on where the project is now

At this point, I think it's worth spending **one session** designing the routing component properly instead of patching together package-specific workarounds. The econometric analysis (DiD, nighttime lights, treatment definition) is the core of your project, so having a routing pipeline that's simple, fast, and easy to explain will make the rest of the work much smoother. I think this pivot gives us that foundation.